INSTRUCTOR KIT · PHP AND WORDPRESS DEVELOPMENT · COMPLETE WORKBOOK

PHP and WordPress Development

Your working notebook for all 18 sessions of PHP and WordPress Development. Read the key ideas, study the worked examples, do the guided labs, predict before you run, and check yourself with the self-check quizzes.

18 sessions · 31 lessons | Write-in exercises & self-checks | Name: _____

CONTENTS

PART 1: THEMES · SESSIONS 1 TO 8

S01 **WordPress for Developers and Your Local Environment**

- 1.1 WordPress for Developers
- 1.2 Your Local Development Environment

S02 **PHP the WordPress Way and the Content Model**

- 1.3 PHP the WordPress Way
- 2.1 The Content Model

S03 **Core Architecture, the Database and Hooks**

- 2.2 Core Architecture & the Database
- 2.3 The Hooks System

S04 **Theme Anatomy and Building a Block Theme · Checkpoint 1**

- 3.1 Theme Anatomy & the Template Hierarchy
- 3.2 Building a Block Theme

S05 **Patterns, Style Variations and Child Themes**

- 3.3 Patterns, Style Variations & the Site Editor
- 3.4 Child Themes & Customizing Existing Themes

S06 **The Loop and Custom Queries**

- 4.1 The Loop & Template Tags
- 4.2 Custom Queries

S07 **functions.php: Supports, Assets, Menus & Widgets**

4.3 functions.php: Supports, Assets, Menus & Widgets

S08 Part 1 Project and Assessment

PART 2: PLUGINS TO LAUNCH · SESSIONS 9 TO 18

S09 Your First Plugin and Custom Post Types

5.1 Your First Plugin

5.2 Custom Post Types, Taxonomies & Meta

S10 Security and the Settings API

5.3 Security in WordPress

5.4 Admin Pages, Settings API & Options

S11 Custom Tables, Shortcodes and Block Setup

5.5 Shortcodes, $wpdb & Custom Tables

6.1 Block Development Setup

S12 Static and Dynamic Blocks

6.2 Static Blocks: Attributes, Controls & Supports

6.3 Dynamic Blocks & Block Bindings

S13 The Interactivity API and Using the REST API · Checkpoint 2

6.4 The Interactivity API

7.1 Using the REST API

S14 Custom Endpoints, External APIs and Cron

7.2 Custom REST Endpoints

7.3 External APIs, Transients & WP-Cron

S15 Debugging, Testing and Performance

8.1 Debugging & Testing

8.2 Performance & Caching

S16 Distribution and Deployment

8.3 Internationalization, Accessibility & Distribution

8.4 Deployment & Maintenance

S17 Final Project Kickoff and Studio I

9.1 Final Project: Launch the Pages & Co. Website

S18 Final Project Studio II, Showcase and Next Steps

9.1 Final Project: Launch the Pages & Co. Website

9.2 Next Steps: Headless, AI in Core & Your Career

PHP AND WORDPRESS DEVELOPMENT · PARTICIPANT WORKBOOK

Part 1: Themes

Sessions 1 to 8. Modules 1 to 4: set up a WordPress 7.1 development site with Docker, WP-CLI, debugging and Mailpit, write PHP the WordPress way, learn the content model, the database and hooks, then build the Pages & Co. block theme with theme.json, templates, patterns and style variations, a child theme, and a classic/hybrid theme with the Loop, custom queries and functions.php. Runs on its own as a short course and ends with the Part 1 project and assessment.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 1: WordPress for Developers and Your Local Environment

Your workbook for Session 1: the environment check, a diagnostic, the key ideas of Lessons 1.1 and 1.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 1.1–1.2 | Session 1 · 3 hours | Name: _____

SESSION GOALS

- Find out whether your computer is ready: `check_environment.sh` ends with READY, and you know what to do if it does not.
- Describe WordPress as a developer sees it: a PHP and MySQL application you extend through hooks, themes, plugins, blocks and APIs.
- Follow one request through core: load, then query, then template; and say which folder is yours and why core is not.
- Use the course site: WP-CLI through the `wp` shortcut, debug.log, Mailpit, and the commands that start, stop and reset it.
- Start the Pages & Co. plan and the health check you can rerun whenever something about your setup changes.

START HERE: IS YOUR COMPUTER READY?

ENVIRONMENT CHECK

- ☐ My project folder `pagesco-dev` is in my home folder (on Windows: inside WSL), and the course's `labs/` folder is inside it, next to compose.yaml
- ☐ The site is running: `docker compose up -d` in the project folder
- ☐ In the project folder: `bash labs/module-01/l1_0-orientation/check_environment.sh`
- ☐ The last line says `READY for PHP and WordPress Development` (or `READY WITH NOTES`). My WordPress and PHP versions: _____
- ☐ If it said NOT READY, I wrote my plan here (from the install clinic) and will finish it before Session 2:

Five messages to know by heart: **docker: command not found** = install Docker · **Cannot connect to the Docker daemon** = start Docker Desktop · **no configuration file provided** = you are not in the project folder · **Permission denied** in `wordpress/` = the Linux and WSL fix of Lesson 1.2 · **There has been a critical error** = read `wordpress/wp-content/debug.log`.

DIAGNOSTIC: PREDICT, THEN CHECK

Six questions on what this course builds on: PHP and MySQL Foundations, the terminal, the web. Circle your answer before the class reveals it. Nobody collects this.

Missed one? The skill and where it comes back: **D1** PHP arrays (PHP Foundations Module 3; every WordPress function that takes an array of arguments (Lesson 1.3)) · **D2** escaping output (PHP Foundations Lesson 4.2; esc_html() and friends in Lessons 1.3 and 5.3) · **D3** SQL (MySQL Foundations; WordPress's tables in Lesson 2.2 and $wpdb in Lesson 5.5) · **D4** folders and the terminal (Lesson 1.2 (every docker compose command) and every lab) · **D5** HTTP status codes (PHP Foundations Lesson 1.1; rewrite rules and 404 templates in Lessons 2.2 and 3.1) · **D6** callbacks (PHP Foundations functions; hooks in Lesson 2.3 work exactly like this). For practice at home: `labs/module-01/l1_0-orientation/diagnostic_starter.php` has PHP answer D1, D2 and D6 on your own site, one small gap each.

D1

What does this PHP print?

```php
$books = ['Dune' => 14.99, '1984' => 9.99, 'Emma' => 7.99];
echo count($books), ' ', array_key_first($books);
```

A) 3 Dune

B) 3 Emma

C) 2 Dune

D) 3 14.99

D2

A visitor typed Maya into a form. What reaches the browser from this line?

```
echo 'Hello, ', htmlspecialchars('<b>Maya</b>');
```

A) Hello, Maya (in bold)

B) `Hello, <b>Maya</b>`

C) `Hello, <b>Maya</b>`

D) Hello, (the tags are removed)

D3

Which query lists the titles of the books under $10, cheapest first?

A) `SELECT title FROM books ORDER BY price < 10;`

B) `SELECT title WHERE price < 10 FROM books;`

C) `SELECT title FROM books WHERE price < 10 ORDER BY price;`

D) `SELECT * FROM books HAVING price < 10 DESC;`

D4

Your terminal is in your home folder; the course project is ~/pagesco-dev. Which starts the site?

A) `docker compose up -d`

B) `docker compose up -d pagesco-dev`

C) `sudo docker compose up -d`

D) `cd pagesco-dev`, then `docker compose up -d`

D5

A browser asks a server for a page that does not exist. Which status code should come back?

A) 404 Not Found

B) 200 OK

C) 301 Moved Permanently

D) 500 Internal Server Error

D6

What does this PHP print?

```php
function shout($t) { return strtoupper($t) . '!'; }
echo implode(' ', array_map('shout', ['Dune', 'Emma']));
```

A) Dune! Emma!

B) DUNE! EMMA!

C) shout shout

D) An error: a function name cannot be a string

LESSONS

1.1 WordPress for Developers

BY THE END YOU CAN

- Describe WordPress 7.1 as a developer sees it: a PHP and MySQL application with themes, plugins, blocks and APIs as extension points
- Explain the difference between WordPress.org (the software) and WordPress.com (a hosting service), and pick the right one for a project
- Trace a front-end request from index.php to the finished HTML, naming the core files on the way
- Say where your code belongs (themes, plugins, must-use plugins in wp-content/) and why you never edit core files
- Map what you know from PHP Foundations and MySQL Foundations to the way WordPress does the same jobs
- Describe the Pages & Co. project and plan how you will work through this course

KEY IDEAS

- **WordPress, to a developer, is a PHP application on MySQL** that builds every page on request, designed to be extended without being edited: **hooks**, **themes**, **plugins**, **blocks** and **APIs**.
- **WordPress.org** is the open-source software and its community; **WordPress.com** is one company's hosting, where your own plugins and themes depend on the plan. The course is self-hosted.
- **One request:** index.php → wp-blog-header.php → wp-load.php → wp-config.php → wp-settings.php (core, must-use plugins, plugins, theme, `init`) → `wp()` (the main query) → template-loader.php → HTML. **Load, then query, then template.**
- **wp-content/ is yours** (themes, plugins, mu-plugins, uploads, debug.log); everything else in the root is core. Everything else WordPress knows is in the database.
- **Never edit core** (or a theme or plugin you did not write): updates replace those files without asking. `wp core verify-checksums` catches an edited core file.
- **Deprecated** messages name the version and the replacement: check how old a tutorial is before you copy from it.
- **Inversion of control:** core runs the main program and calls your code through hooks.

WORKED EXAMPLE: THREE CLUES THAT A SITE RUNS WORDPRESS

You can tell a WordPress site from the outside, and your own site is the best place to learn the clues. In the WP-CLI container shell (`docker compose run --rm wpcli sh`), ask for the home page and look for the

theme's folder in its HTML:

```
curl -s http://localhost:8080/ | grep -o 'wp-content/themes/[a-z0-9-]*' | sort -u
```

```
wp-content/themes/twentytwentyfive
```

Every stylesheet and script of the theme is loaded from `wp-content/themes/<theme folder>/`, so the folder name shows in the page. Core's own files come from `wp-includes/`; count the lines that mention it:

```
curl -s http://localhost:8080/ | grep -c 'wp-includes/'
```

```
22
```

And the response headers carry the address of the REST API, which every WordPress site has at `/wp-json/` (Module 7):

```
curl -sI http://localhost:8080/ | grep -i '^link'
```

```
Link: <http://localhost:8080/wp-json/>; rel="https://api.w.org/"
```

Three clues, none of them a secret: a site that hides them is still a WordPress site. (On a site that is not your own, View Source in the browser shows the same paths.)

GUIDED LAB

- From the project folder: `cp labs/module-01/l1_1-wordpress-for-developers/core_tour_starter.sh wordpress/`, then `docker compose run --rm wpcli sh core_tour_starter.sh`. Open `wordpress/core_tour_starter.sh` in VS Code.
- Step 1 is done for you. TODOs 2 to 5: replace each `echo "TODO"` line with a grep (or a find) that prints the line its comment describes. Run again after each one.
- For each step write one sentence: what does this line of core do on every request?

- Done when your output matches the "Expected when done" comment at the end of the file. Then `rm wordpress/core_tour_starter.sh`.
- Finished early? `wp_paths_starter.php`, then `debug_old_tutorial.php` in the same folder (run both with `wp eval-file --use-include`).

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site: `wp ...` commands from the project folder, the others in the WP-CLI container shell (`docker compose run --rm wpcli sh`). A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (A page asked for with curl, from the WP-CLI container.)

```
curl -s http://localhost:8080/sample-page/ | grep -o '<title>.*</title>'
```

P2

Predict the output, then run it. (WP-CLI loads WordPress. How far does it get?)

```
wp eval 'echo did_action( "init" ), " ", did_action( "template_redirect" ), "\
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Meet WordPress and Plan Pages & Co. (lesson page, "Project: Meet WordPress and Plan Pages & Co."). In class: Step 3 and the start of Step 4. At home: Steps 1 and 2 (Playground) and the rest of Step 4.

- ☐ Step 1: Playground started; WordPress and PHP versions it runs: _____
- ☐ Step 2: a page with a heading and a paragraph; one surprise in its block markup
- ☐ Step 3: `pagesco-plan.txt` with at least ten addresses (the lesson's seven plus three of mine)
- ☐ Step 4: at least eight features, each marked content, theme or plugin, with a reason

SELF-CHECK

Q1

Pages & Co. updates WordPress from 7.1.3 to 7.1.4. Which of these folders does the update REPLACE?

A) `wp-content/plugins/pagesco-books/`

B) `wp-content/themes/pagesco/`

C) `wp-includes/`

D) `wp-content/uploads/`

Q2

From the WP-CLI container, on a fresh course site: what does this print?

```
curl -s http://localhost:8080/ | grep -o '<title>.*</title>'
```

Q3

A front-end request travels through core. Which of these happens FIRST?

A) template-loader.php picks a template from the theme

B) wp() parses the URL and runs the main query

C) the init action fires, after plugins and the theme are loaded

D) the theme's template prints the HTML

Q4

`wp core verify-checksums` prints `Warning: File doesn't verify against checksum: wp-includes/functions.php`. What happened, and what do you do?

A) The download was damaged: reinstall WordPress from scratch

B) WordPress 7.1 changed functions.php: nothing to do

C) A plugin is outdated: update all plugins

D) Someone edited a core file: restore it and put the change in a plugin or theme, on a hook

Q5

With pretty permalinks, which file does the web server hand `/books/dune/`, `/about/` and almost every other front-end URL?

A) `wp-config.php`

B) `index.php` in the WordPress root

C) `wp-settings.php`

D) A file named after the URL, such as `books/dune.php`

Q6

In PHP Foundations you printed a visitor's input with `htmlspecialchars()`. Which WordPress function do you use instead when printing text into HTML?

JOURNAL

Start your journal with two questions. Why do you want to learn WordPress development, and what do you hope to build by the end of this course, besides Pages & Co.? Who is it for, which two or three features does it need, and which of them will be theme work and which plugin work?

1.2 Your Local Development Environment

BY THE END YOU CAN

- Install Docker and start the course's Docker Compose project: WordPress, MySQL, WP-CLI and Mailpit
- Install WordPress from the command line with WP-CLI and use the wp shortcut for everyday tasks
- Fix the Linux and WSL file-ownership problem so your editor can save into wp-content/
- Find and read PHP errors on the page and in wp-content/debug.log, and catch WordPress email in Mailpit
- Start, stop and reset the site safely, and know when wp-env or WordPress Playground is the better tool

KEY IDEAS

- **The course site is a Docker Compose project**, pagesco-dev: **wordpress** (Apache, PHP 8.5, WordPress 7.1), **db** (MySQL 9.7), **wpcli** (on demand) and **mailpit**.
- **Your `wordpress/` folder IS the WordPress root** inside the containers (`/var/www/html`): paths in messages translate one to one. The database lives in the `db_data` volume.
- **Linux and WSL:** run the containers as your user (`.env` + `compose.override.yaml` , before the first start), so you and WordPress own the same files.
- `wp` **is the shortcut** for `docker compose --progress quiet run --rm wpcli wp` ; commands run in the WordPress root; `docker compose run --rm wpcli sh` opens a shell there.
- **WP_DEBUG, WP_DEBUG_LOG and WP_DEBUG_DISPLAY are on**: errors show in the page and in `wp-content/debug.log` . Read every message from the left: level, message, file, line.
- **Mailpit** catches every email at http://localhost:8025; the must-use plugin `mailpit-smtp.php` sends mail there.
- `down` **keeps your work,** `down -v` **deletes the database.** wp-env (`npx @wordpress/env` , never `npx wp-env`) and Playground are good second tools.

WORKED EXAMPLE: WHERE THE COMPOSE SETTINGS END UP

compose.yaml sets `WORDPRESS_DB_NAME` , `WORDPRESS_DEBUG` and `WORDPRESS_CONFIG_EXTRA` . Where do they end up? Search the wp-config.php the image wrote, from the WP-CLI container shell:

```
grep -n "getenv_docker('WORDPRESS_DB_NAME'\|getenv_docker('WORDPRESS_DEBUG'\|WORDP
```

```
44:define( 'DB_NAME', getenv_docker('WORDPRESS_DB_NAME', 'wordpress') );
116:define( 'WP_DEBUG', !!getenv_docker('WORDPRESS_DEBUG', '') );
127:if ($configExtra = getenv_docker('WORDPRESS_CONFIG_EXTRA', '')) {
```

wp-config.php does not contain your values: it reads them from the container's environment every time it runs (`getenv_docker()`), and runs the WORDPRESS_CONFIG_EXTRA lines with `eval()`. Ask the running site for one of them:

```
wp eval 'echo getenv( "WORDPRESS_DB_NAME" ), " ", DB_NAME, "\n";'
```

```
wordpress wordpress
```

The environment variable and the constant `DB_NAME` agree. That is why a change to compose.yaml takes effect after `docker compose up -d` (which recreates the container with the new environment) and why you never need to edit wp-config.php for the course settings.

GUIDED LAB

- From the project folder: `cp labs/module-01/l1_2-local-environment/site_report_starter.sh wordpress/`, then `docker compose run --rm wpcli sh site_report_starter.sh`.
- For each TODO, try the command alone first (`wp option get admin_email`, `wp help theme list`), then put it in the script and run the script again.
- Finished early? The store-hours folder lab (copy `store_hours_starter/pagesco-hours.php` into `wordpress/wp-content/mu-plugins/`, then run `test_store_hours.sh`), then `debug_health_check.php`.

QUESTION	YOUR ANSWER
Why does the title print as `Pages & Co.`?	
Which TODO needed a second option to be reliable, and why?	
Which of these could the admin screens show you, and how fast?	

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site: `wp ...` commands from the project folder, the others in the WP-CLI container shell (`docker compose run --rm wpcli sh`). A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (SQL on WordPress's own tables.)

```
wp db query "SELECT option_value FROM wp_options WHERE option_name = 'blogname
```

P4

Predict the output, then run it. (An email with no address.)

```
wp eval 'var_dump( wp_mail( "", "No address", "Hello" ) );'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Dev Site Health Check (lesson page, "Project: The Dev Site Health Check"). In class: Steps 1 to 3. At home: Step 4.

- ☐ Step 1: `docker compose ps`, the site, the admin login, Mailpit, `wp core version` all fine
- ☐ Step 2: `wordpress/pagesco-check.php` typed from the lesson
- ☐ Step 3: `wp eval-file pagesco-check.php` says "All checks passed."; the email is in Mailpit
- ☐ Step 4: with Mailpit stopped one check FAILED; started again; the script deleted; `mu-plugins/` holds only mailpit-smtp.php

SELF-CHECK

Q7

You want to throw your site away and start over with a blank WordPress. Which command deletes the database?

A) `docker compose stop`

B) `docker compose down`

C) `docker compose down -v`

D) `docker compose restart`

Q8

What does this print on the course site, exactly?

```
wp option get blogname
```

Q9

wp-content/debug.log says `PHP Warning: Undefined variable $pagesco_name in /var/www/html/wp-content/mu-plugins/pagesco-debug-demo.php on line 8`. Where is that file on YOUR computer?

A) `/var/www/html/wp-content/...` on your computer's disk

B) `wordpress/wp-content/mu-plugins/pagesco-debug-demo.php` in your project folder

C) Inside the db container

D) Nowhere: it only exists while the container runs

Q10

In a new terminal you type `wp core version` and get `no configuration file provided: not found`. What is wrong?

A) You are not in the project folder: Docker Compose found no compose.yaml here or above

B) WordPress is not installed yet

C) Docker is not running

D) The wp shortcut is missing

Q11

Most Docker tutorials map `8080:80`. Why does the course project use `8080:8080` (Apache listens on 8080 inside the container too)?

A) Port 80 is reserved for MySQL

B) Docker Desktop cannot publish port 80

C) It makes the site faster

D) WordPress calls its own address, http://localhost:8080, from inside the container (WP-Cron, Site Health), and that must reach Apache

Q12

What does this print on the course site? Name one thing that value changes.

```
wp eval 'echo wp_get_environment_type(), "\n";'
```

JOURNAL

Write your own cheat card for the development site: the five commands you expect to type most, each with one line saying what it does, and where to look first when (a) a page shows an error, (b) an email does not

arrive, (c) the site will not start. Then note which part of the setup was hardest for you and what finally made it work.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Where does your own code go in WordPress, and why never in core?

E2

Name the four services of the course project and the job of each.

E3

A page says "There has been a critical error on this website." Where do you look first, and with which command?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the Pages & Co. plan (Playground Steps 1 and 2, the rest of Step 4) and the health check (Step 4)
- ☐ Lesson 1.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 1.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ 1.2 "Before the Next Lesson": the health check passes and `wp` works in a new terminal; mailpit-smtp.php stays
- ☐ If my check was not READY: follow my plan and run the check again
- ☐ Optional: `diagnostic_starter.php`, `wp_paths_starter.php`, the store-hours lab, `debug_health_check.php`
- ☐ Bring to Session 2: a READY laptop, the project folder with `labs/`, `docker pull composer:2` done at home, my plan and journal, this workbook

GLOSSARY

TERM	MEANING
core	The WordPress files themselves (`wp-admin/`, `wp-includes/`, the root `wp-*.php` files); replaced by every update
`wp-content/`	The site's own folder: themes, plugins, must-use plugins, uploads, debug.log
theme / plugin	How the site looks / what the site can do
must-use plugin	A PHP file in `wp-content/mu-plugins/`: runs on every request, no activation
hook (action, filter)	A named moment where core calls your function (action) or lets it change a value (filter); Lesson 2.3
main query	The database query `wp()` runs from the address of a request
WordPress.org / WordPress.com	The open-source software / one company's hosting of it
Docker Compose	Starts several containers together from compose.yaml
bind mount	A folder on your disk that appears inside a container (`./wordpress:/var/www/html`)

TERM	MEANING
volume	Storage Docker keeps for a container (`db_data`: the database)
WP-CLI (`wp`)	WordPress's command-line tool; the course runs it in the wpcli container
`debug.log`	`wp-content/debug.log`: every PHP message and `error_log()` line while WP_DEBUG_LOG is on
Mailpit	A fake mail server with a web inbox (http://localhost:8025)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 2: PHP the WordPress Way and the Content Model

Your workbook for Session 2: a warm-up on Session 1, the key ideas of Lessons 1.3 and 2.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 1.3–2.1 | Session 2 · 3 hours | Name: _____

SESSION GOALS

- Write PHP the WordPress way: the coding standards (checked by PHPCS), a prefix on every global name, and core's functions instead of your own helpers.
- Sanitize what comes in and escape what goes out, with the function that fits each place, and make visible strings translatable.
- Pass callbacks in WordPress's shapes, write functions that take an array of arguments, and handle WP_Error results.
- Read the content model as a developer: post types, statuses, terms, attachments, revisions, roles and capabilities, options and block markup.
- Stock the Pages & Co. site with its staff, pages and News posts, with the IDs every later lesson uses.

WARM-UP: SESSION 1 REVISITED

FIVE QUESTIONS, NO NOTES

Answer from memory, then check with the class.

W1

Which folder of the WordPress root is yours, and why is nothing else?

W2

Put them in order: the main query, `init`, the template.

W3

`docker compose down` and `docker compose down -v`: what survives each?

W4

A page says "There has been a critical error on this website." Where do you look first?

W5

What does `wp option get blogname` print on the course site, and why?

Site check, from the project folder: `wp core version` and `wp plugin list --status=must-use --fields=name,status`. ☐ WordPress 7.1 and only mailpit-smtp listed. Homework: ☐ the health check said "All checks passed." ☐ `pagesco-plan.txt` is in my course folder ☐ `docker image ls composer:2` lists the image.

LESSONS

1.3 PHP the WordPress Way

BY THE END YOU CAN

- Write PHP that follows the WordPress coding standards, and check it with PHPCS and the WordPress ruleset
- Choose the right escaping function for each output context (esc_html, esc_attr, esc_url, wp_kses_post) and the right sanitizing function for each input
- Make strings translatable with __(), esc_html__(), _n() and sprintf(), with a text domain and translator comments
- Pass callbacks in every form WordPress accepts, and write functions that take an array of arguments merged with wp_parse_args()
- Handle WP_Error results, and use core's globals ($wpdb, $post, $wp_query) and classes without fighting them

KEY IDEAS

- **WordPress code is a guest** in a PHP process shared with core, the theme and every plugin: house rules make that work.
- **The WordPress Coding Standards** (tabs, spaces inside parentheses, Yoda conditions, single quotes, `array()`, DocBlocks); **PHPCS** with the WordPress ruleset checks them and also finds unescaped output; `phpcbf` fixes the mechanical part.
- **Prefix every global name** (`pagesco_`): functions, global variables, constants, hooks, options, meta keys, handles; the text domain is the folder slug. A namespace is a prefix.
- **Sanitize early, escape late:** `wp_unslash()` then `sanitize_text_field()` / `sanitize_email()` / `absint()` (or validate); at the echo `esc_html()`, `esc_attr()`, `esc_url()`, `wp_kses_post()`.
- **Translatable strings:** `__()` returns, `_e()` echoes, `esc_html__()` for templates, `_n()` for plurals, `sprintf()` placeholders with a `/* translators: */` comment.
- **Callbacks** come as a name, a closure, an arrow function, `array( $obj, 'method' )` or a static method; named ones can be unhooked.
- **Arrays of arguments** merged with `wp_parse_args()`; **failures as WP_Error**, checked with `is_wp_error()`; read core's globals, never assign them; values from the database are strings, so cast instead of using strict types.

WORKED EXAMPLE: THE SLASHES WORDPRESS ADDS

WordPress adds a backslash before every quote in `$_GET`, `$_POST` and `$_COOKIE` (`wp_magic_quotes()`, Lesson 1.3). You can watch the round trip without a form: `wp_slash()` does what WordPress does on the way in, `wp_unslash()` undoes it.

```
wp eval '$pagesco_in = wp_slash( "O'\''Reilly" ); var_dump( $pagesco_in, wp_unslas
```

```
string(9) "O\'Reilly"
string(8) "O'Reilly"
```

Nine characters on the way in, eight after `wp_unslash()`: the backslash is real, not a display artifact. Store the first value and the apostrophe in O'Reilly grows a backslash in the database; that is why every input line in WordPress code starts with `wp_unslash()` and then a `sanitize_*()` function: `sanitize_text_field( wp_unslash( $_POST['name'] ) )`. (The `'\''` in the command is the shell's way of writing one apostrophe inside single quotes.)

GUIDED LAB

- From the project folder: `cp labs/module-02/l1_3-php-the-wordpress-way/signup_form_starter.php wordpress/`, then `wp eval-file --use-include signup_form_starter.php`. Open `wordpress/signup_form_starter.php` in VS Code.
- TODOs 1 to 4 sanitize (and validate) the form; TODO 5 checks the email; TODOs 6 and 7 escape the two HTML lines. Run after each one.
- For each field write which function you chose and why that one:

- Done when your output matches the "Expected when done" comment at the end of the file. Then `rm wordpress/signup_form_starter.php`.
- Finished early? `event_line_starter.php`, then `debug_book_badge.php` in the same folder (run both the same way).

PREDICT, THEN RUN

Write your prediction on the line first, then run it from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (A URL without a protocol, and an ampersand.)

```
wp eval 'echo esc_url( "example.com/books?a=1&b=2" ), "\n";'
```

P2

Predict the output, then run it. (A plural with zero, and no sprintf().)

```
wp eval 'echo _n( "%s book", "%s books", 0 ), "\n";'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Book Card Helper (lesson page, "Project: The Book Card Helper"). In class: Step 1. At home: Steps 2 to 4.

- ☐ Step 1: `wordpress/pagesco-practice/book-card.php` typed, every comment read
- ☐ Step 2: `render-cards.php`: three cards, "1,178 pages", a second Dune card with an h2 and no price
- ☐ Step 3: the break test: no working script, link or attribute in the output
- ☐ Step 4: PHPCS says "book-card.php: no coding standards problems."; a copy of book-card.php in my notes; `pagesco-practice/` deleted

SELF-CHECK

Q1

A template prints a book cover: `<img src="..." alt="...">`. The alt text is the book's title as staff typed it. Which function goes around the title?

A) `esc_html()`

B) `esc_attr()`

C) `esc_url()`

D) `wp_kses_post()`

Q2

From the project folder, on the course site: what does this print?

```
wp eval 'echo esc_html( esc_html( "Tom & Jerry" ) ), "\n";'
```

Q3

What reaches the page from `$h = '<h2>' . _e( 'Staff picks', 'pagesco' ) . '</h2>'; echo $h;`?

A) `<h2>Staff picks</h2>`

B) Nothing: PHP stops with a fatal error

C) "Staff picks" first, then an empty `<h2></h2>`

D) `<h2>_e(Staff picks)</h2>`

Q4

Which of these needs NO `pagesco_` prefix?

A) A class inside the namespace `PagesCo\Books`, such as `PagesCo\Books\Plugin`

B) A global function that formats prices

C) A variable defined at the top level of a plugin file

D) An option your plugin saves in wp_options

Q5

What does this print?

```
wp eval 'var_dump( absint( "-7 copies" ), sanitize_key( "Staff Picks!" ) );'
```

Q6

`$id = wp_insert_post( array( 'post_title' => '' ) );` **What is `$id`, and how do you learn why it failed?**

A) A WP_Error: call `$id->get_error_message()`

B) false: read debug.log

C) An exception: wrap the call in try/catch

D) `0`: pass `true` as the second argument to get a WP_Error with the message

JOURNAL

Make two columns: "habits I keep from PHP Foundations" and "habits I change for WordPress". Put at least four items in each (strict types, naming, escaping, error handling, arrays...). For one habit you are changing, write why WordPress does it differently in your own words. Which rule felt most like going backwards, and has the reason behind it changed how you feel about it?

2.1 The Content Model

BY THE END YOU CAN

- Explain why posts, pages, media, revisions, templates and navigation menus are all posts with a different post_type, and read a WP_Post object
- Choose between posts and pages, use post statuses, and organize content with categories, tags and terms
- Describe how roles, capabilities and users fit together, and check capabilities rather than roles in code
- Find the option behind a Settings screen, and read the block markup the editor stores in post_content with parse_blocks()
- Create users, pages and posts from the command line, with authors, categories, dates and revisions

KEY IDEAS

- **Everything is a post:** posts, pages, attachments, revisions, templates, patterns and block-theme menus are rows of `wp_posts`, told apart by `post_type` (16 types on a fresh site); PHP sees a row as a WP_Post.
- **Posts** are dated and filed under categories and tags; **pages** are timeless and hierarchical. `post_status` decides who sees a row; new content defaults to **draft**, and a draft has no slug.
- **Tools differ in their defaults:** WP-CLI lists every status, `get_posts()` and WP_Query only published content.
- **Taxonomies** group content into **terms**; an upload is an attachment post plus its files and generated sizes.
- **Every update is a revision**, and the newest one equals the current post; trash is a status for 30 days, `--force` deletes for good.
- **Roles bundle capabilities;** code checks the capability (`current_user_can()`, `user_can()`), including meta capabilities like `edit_post` with an ID.
- **Every Settings field is an option** (a string); content is **block markup**: `parse_blocks()` reads it, `do_blocks()` renders it.

WORKED EXAMPLE: FROM A SETTINGS FIELD TO ITS OPTION

Settings > General has a Date Format field. Every Settings field is an option (Lesson 2.1's table); read it, then let WordPress use it the way a theme does:

```
wp option get date_format
```

```
F j, Y
```

```
wp eval 'echo date_i18n( get_option( "date_format" ), strtotime( "2026-10-01" ) ),
```

```
October 1, 2026
```

`F j, Y` is a PHP date format (F = month name, j = day, Y = year). `date_i18n()` formats a time stamp in the site's language and format: the News post dates of the project will print like this. Options can also be searched by name; the image sizes of Settings > Media:

```
wp option list --search='*_size_w' --fields=option_name,option_value
```

```
+---------------------+--------------+
| option_name         | option_value |
+---------------------+--------------+
| thumbnail_size_w    | 150          |
| medium_size_w       | 300          |
| large_size_w        | 1024         |
| medium_large_size_w | 768          |
+---------------------+--------------+
```

Four widths, four rows: this is how you find the option behind a field you cannot name. (The values print without quotes here, but in PHP they are strings: `'300'`.)

◆ GUIDED LAB

- From the project folder: `cp labs/module-02/l2_1-content-model/content_census_starter.sh wordpress/`, then `docker compose run --rm wpcli sh content_census_starter.sh`.
- For each TODO, try the command alone first (`wp post list --post_type=page --format=count`), then put it inside the `$( )` and run the script again. Run it BEFORE you start the project.
- Finished early? `post_report_starter.php`, then `debug_page_report.php` (both only read).

QUESTION	YOUR ANSWER
Which page is the draft, and who made it?	
Why does an editor need so many capabilities?	
Which TODO needed a pipe, and why?	

◆ PREDICT, THEN RUN

Write your prediction on the line first, then run it from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Twenty Twenty-Five shows images. Are they attachments?)

```
wp post list --post_type=attachment --format=count
```

P4

Predict the output, then run it. (An option that is set, and one that does not exist.)

```
wp eval 'var_dump( get_option( "page_on_front" ), get_option( "pagesco_no_such
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Stock the Pages & Co. Site (lesson page, "Project: Stock the Pages & Co. Site"). In class: Steps 1 to 3. At home: Step 4. First, the next-ID check from the demo printed: _____ (it must be 12).

- ☐ Step 1: maria (Editor, user 2) and devon (Author, user 3), with the course passwords
- ☐ Step 2: About Us, Contact, Events published by Maria: IDs 12, 13, 14; the header menu lists them
- ☐ Step 3: the News category (2) and three posts 15, 16, 17 with their dates; /category/news/ lists them newest first
- ☐ Step 4: Maria's edit (revision 18), Devon refused on the Pages screen, `pagesco-content/` deleted, the final list matches the lesson

SELF-CHECK

Q7

Twenty Twenty-Five's header menu lists the site's pages. Which post type stores that menu?

A) `nav_menu_item`

B) `page`

C) `wp_template_part`

D) `wp_navigation`

Q8

On a fresh course site (before the lesson project): what does this print, and why are the two numbers counting different things?

```
wp eval 'echo count( get_posts( array( "post_type" => "page" ) ) ), " ", wp_co
```

Q9

Devon (Author) opens Maria's published post "Book club: Dune" (ID 17). What does `user_can( $devon, 'edit_post', 17 )` answer?

A) false: `edit_post` is a meta capability that maps to `edit_others_posts` here, which authors lack

B) true: authors have `edit_posts`

C) true: the post is published

D) An error: `edit_post` needs a role, not a user

Q10

What does this print?

```
wp eval 'echo parse_blocks( "<!-- wp:separator /-->" )[0]["blockName"], "\n";'
```

Q11

A script creates a post with WP-CLI and then updates it twice. How many revisions does the post have, and what is in the newest one?

A) Three: the newest holds the version before the last update

B) Two: the newest is identical to the current post

C) One: WordPress keeps only the latest

D) None: only the block editor makes revisions

Q12

A deployment script must set the "Site Address (URL)" field of Settings > General. Which option is that?

A) `siteurl`

B) `blogname`

C) `home`

D) `permalink_structure`

JOURNAL

Open pagesco-plan.txt from Lesson 1.1 and sort every item of your site map into the content model: which are pages, which are posts and in which category, which need a custom post type, which are settings, and who (which role) maintains each. Did anything not fit? Write down the one you are least sure about; Lessons 5.2 and 5.4 will help you decide.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Which escaping function for text between tags, for an attribute value, and for an href?

E2

Why does every global name in your theme and plugin start with `pagesco_`?

E3

Posts, pages, uploaded images, revisions and block-theme menus: where are they stored, and what tells them apart?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the Book Card Helper (Steps 2 to 4, PHPCS clean, practice folder deleted, a copy in my notes)

- ☐ Finish Stock the Pages & Co. Site (Step 4 and the final list: users 2 and 3, pages 12 to 14, posts 15 to 17)
- ☐ Lesson 1.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Both "Before the Next Lesson" lists: mu-plugins/ holds only mailpit-smtp.php; no user sam; Settings > Reading on "Your latest posts"
- ☐ Optional: `event_line_starter.php`, `debug_book_badge.php`, `post_report_starter.php`, `debug_page_report.php`
- ☐ Bring to Session 3: the site at the end of Lesson 2.1, book-card.php, my sorted plan, this workbook

GLOSSARY

TERM	MEANING
WPCS / PHPCS	The WordPress Coding Standards / PHP_CodeSniffer, the tool that checks code against them (`phpcbf` fixes the mechanical part)
prefix	`pagesco_` in front of every global name, so it cannot collide with core or another plugin
sanitize / validate	Clean input into the shape you expect / accept it only if it is exactly what is allowed
escape	Make a value safe for the place it is printed into: `esc_html()`, `esc_attr()`, `esc_url()`, `wp_kses_post()`
text domain	The second argument of `__()` and friends: whose translation file to look in (the folder slug)
callback	A value that says "call this later": a function name, a closure, an object or static method
`WP_Error`	The object core returns when something fails; test it with `is_wp_error()`
post type	The label on a row of wp_posts: post, page, attachment, revision, wp_navigation...
post status	Who may see a row: publish, draft, pending, private, future, trash, inherit
taxonomy / term	A way of grouping content (category, post_tag) / one group in it (News)
role / capability	A bundle of permissions (Editor) / one permission (edit_others_posts)

TERM	MEANING
meta capability	A capability about one object (`edit_post` with an ID), mapped to real capabilities by WordPress
option	A row of wp_options: every Settings field is one, and every value is a string
block markup	What the editor stores in post_content: HTML wrapped in `<!-- wp:... -->` comments

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 3: Core Architecture, the Database and Hooks

Your workbook for Session 3: a warm-up, the key ideas of Lessons 2.2 and 2.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 2.2–2.3 | Session 3 · 3 hours | Name: _____

SESSION GOALS

- Read wp-config.php and wp-settings.php as a developer: what each setting does, and what is available at each stage of loading.
- Map WordPress's twelve tables and query them with SQL: joins, the key-value pattern, serialized values, autoloaded options.
- Choose the API over raw SQL for anything that changes data, and say why.
- Write actions and filters with the right priority and arguments, remove someone else's hook, and find the hook you need.
- Start the Pages & Co. store notice: a must-use plugin with hooks of its own, the first file of the course that stays.

WARM-UP: SESSION 2 REVISITED

FIVE QUESTIONS, NO NOTES

Answer from memory, then check with the class.

W1

Which escaping function goes around a URL in an href attribute?

W2

Name the four shapes a WordPress callback can take. Which one is hard for anyone else to unhook?

W3

A page and a post: which table holds each, and which column tells them apart?

W4

Code must decide whether the current user may publish pages. What does it check, and why not the role?

W5

What does `wp_parse_args( $args, $defaults )` return?

Site check, from the project folder: `wp post list --post_type=page,post --orderby=ID --order=asc --fields=ID,post_type,post_name,post_author` and `wp user list --orderby=ID --fields=ID,user_login,roles`. Same IDs as the projector (pages 12 to 14, posts 15 to 17, users 2 and 3)? ☐ yes ☐ no: I finish Lesson 2.1's project in the break.

LESSONS

2.2 Core Architecture & the Database

BY THE END YOU CAN

- Find your way around a WordPress install: what lives in wp-admin/, wp-includes/ and wp-content/, and what a drop-in is
- Read wp-config.php and explain its settings, including the table prefix and the security keys
- Trace how wp-settings.php loads WordPress, and say what is available at each stage
- Map the twelve wp_* tables, their keys and indexes, and join them with SQL the way WordPress's own queries do
- Explain the meta (key-value) pattern, serialized values and options autoloading, with their trade-offs
- Query the Pages & Co. site with wp db cli, wp db query and WP-CLI, and choose the API over raw SQL in your code

KEY IDEAS

- **Core** is wp-admin/, wp-includes/ and the root files; **yours** is wp-content/ (where drop-ins such as object-cache.php can replace parts of core) and wp-config.php.
- **wp-config.php** holds the database connection, eight secret keys and `$table_prefix`, and its last line loads wp-settings.php: nothing defined after it takes effect.
- **wp-settings.php loads in a fixed order:** database, must-use plugins, plugins, pluggable functions, theme, current user, then `init` and `wp_loaded`.
- **Twelve tables** in five groups (content, taxonomies, users, comments, settings), linked by ID columns but with **no foreign keys**: integrity lives in PHP, so change data through WordPress.
- **Meta tables are key-value rows:** flexible, but every value is text, each condition is one more join, and values are not indexed.
- **Arrays are stored serialized**, lengths included: never edit them as text (`wp option patch`, `update_option()`, `wp search-replace`).
- **Autoload:** `on`, `auto`, `auto-on` are loaded with every request; `off`, `auto-off` are read on demand (one query, then cached). In code: `{$wpdb->posts}`, never `wp_posts`.

WORKED EXAMPLE: A SLUG LOOKUP AND ITS INDEX

How does WordPress turn `/about/` into page 12 so fast? Ask MySQL how it would find a page by its slug (EXPLAIN, from MySQL Foundations Lesson 6.1; `\G` prints the plan vertically):

```
wp db query "EXPLAIN SELECT ID FROM wp_posts WHERE post_name = 'about' AND post_ty
```

```
*************************** 1. row ***************************
EXPLAIN: -> Filter: ((wp_posts.post_name = 'about') and (wp_posts.post_type = 'pa
    -> Index lookup on wp_posts using post_name (post_name = 'about')  (cost=0.29
```

"Index lookup on wp_posts using post_name": the prefix index `post_name(191)` exists for exactly this question, so MySQL reads one row instead of the table. WordPress asks the same question through its API:

```
wp post list --post_type=page --name=about --field=ID
```

```
12
```

Every index on wp_posts matches a question WordPress asks all the time: by slug, by parent, by author, and "the latest published posts of this type" (`type_status_date`). There is no index on `wp_postmeta.meta_value` , which is why searching meta by value alone reads the whole table.

GUIDED LAB

- From the project folder: `cp labs/module-03/l2_2-core-database/meta_pivot_starter.sh wordpress/` , then `docker compose run --rm wpcli sh meta_pivot_starter.sh` . Open `wordpress/meta_pivot_starter.sh` in VS Code and `wp db cli` in a second terminal.
- For each TODO, build the query at the prompt first, then put it in the script inside `wp db query "..."` and run the script again.
- Step 3 is the new idea: one join, GROUP BY the user, and one `MAX(CASE WHEN um.meta_key = '...' THEN um.meta_value END)` per column.

- Done when your output matches the "Expected when done" comment at the end of the file. Then `rm wordpress/meta_pivot_starter.sh` .
- Finished early? `wpdb_report_starter.php` , then `debug_meta_report.php` in the same folder (run both with `wp eval-file --use-include`).

QUESTION	YOUR ANSWER
Why does Step 3 need GROUP BY?	
What would one more field in the report cost?	

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (An option that does not exist, twice.)

```
wp eval 'var_dump( get_option( "pagesco_no_such_option" ), get_option( "pagesc
```

P2

Predict the output, then run it. (The list of active plugins, as the database stores it.)

```
wp db query "SELECT option_value FROM wp_options WHERE option_name = 'active_p
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Database Inventory of Pages & Co. (lesson page, "Project: A Database Inventory of Pages & Co."). In class: Steps 1 and 2. At home: Steps 3 and 4. Nothing on the site changes.

- ☐ Step 1: content by type and status; the numbers agree with `wp_count_posts()`
- ☐ Step 2: all eight published posts and pages with author and categories (LEFT JOIN, the taxonomy condition in ON); both queries saved in `pagesco-inventory.sql`
- ☐ Step 3: `pagesco-inventory.php` with `{$wpdb->posts}`, `{$wpdb->users}` and `$wpdb->prepare()`; the totals agree with the API

- ☐ Step 4: my five largest autoloaded options: _____; the script deleted from the WordPress root

SELF-CHECK

Q1

A tutorial says to add `define( 'DISALLOW_FILE_EDIT', true );` to wp-config.php. You paste it as the very last line of the file, after `require_once ABSPATH . 'wp-settings.php';`. What happens?

A) It works: wp-config.php is read on every request, so the position does not matter

B) PHP stops with a parse error

C) It is defined too late: wp-settings.php has already loaded WordPress, so the file editors stay

D) WordPress moves the line above the "stop editing" comment the next time it saves the file

Q2

What does this print on the course site, and what is that value?

```
wp config get DB_HOST
```

Q3

In which order does wp-settings.php load these on every request?

A) The theme's functions.php, active plugins, must-use plugins, pluggable.php

B) Must-use plugins, active plugins, pluggable.php, the theme's functions.php

C) pluggable.php, must-use plugins, the theme's functions.php, active plugins

D) Active plugins, must-use plugins, the theme's functions.php, pluggable.php

Q4

Devon's first name is stored once in wp_usermeta. What does this print?

```
wp eval 'var_dump( get_user_meta( 3, "first_name" ) );'
```

Q5

Someone runs `UPDATE wp_usermeta SET meta_value = REPLACE(meta_value, 'author', 'editor')` on Devon's `a:1:{s:6:"author";b:1;}`. What happens?

A) The value breaks: unserialize() fails and Devon loses every role

B) It works, but only because both words have 6 characters: the s:6 length still matches. Change roles through WordPress anyway

C) MySQL refuses: the column holds serialized PHP

D) WordPress recomputes the lengths when it reads the value

Q6

An option's autoload column says `auto-off`. What does that mean?

A) The code that saved it passed false

B) It is loaded on every request

C) The option is broken and will be deleted

D) Nobody chose, and WordPress decided not to autoload it, usually because the value is large (over 150,000 bytes)

JOURNAL

Picture the bookstore database you built in MySQL Foundations next to WordPress's twelve tables. Where would a book, its author, its price and its genres live in WordPress? Write down one thing your own design did better (types, foreign keys, indexes) and one thing WordPress's design does better (flexibility, one API for everything). Which would you choose for an online shop, and why?

2.3 The Hooks System

BY THE END YOU CAN

- Explain the difference between an action and a filter, and choose the right one for a task
- Attach callbacks with add_action() and add_filter(), using priorities and the accepted-arguments count correctly
- Remove a hook someone else added with remove_action() / remove_filter(), and say why a removal silently fails
- Create your own action and filter hooks with do_action() and apply_filters() so other code can extend yours
- Find the hook you need: in the core source, with has_filter() and $wp_filter, and from the command line with WP-CLI
- Build the Pages & Co. store notice as a must-use plugin that others can customize through its own hooks

KEY IDEAS

- **A hook** is a named point where WordPress calls the functions registered for it: how you change WordPress without editing core.
- **Actions** (`add_action`, `do_action`) run code at a moment; **filters** (`add_filter`, `apply_filters`) receive a value and must return it.
- **Priority** (default 10, lower first, ties in the order added) orders callbacks; **accepted args** (default 1) must match what the callback asks for: `10, 2`.
- **Removing** needs the same hook, callback and priority, after the hook was added; `remove_action()` returns false silently when nothing matches. Closures cannot be removed by anyone else.
- **Your own hooks** make code extensible: prefix them, a filter for values and an action for moments, pass context, document them with a DocBlock.
- **Finding hooks:** the Code Reference, the function you want to change, `grep -rn` in wordpress/, and `has_filter()` / `$wp_filter` on the running site.
- **"Too early"** is the most common hook bug: code at the top of a plugin file runs while plugins load; move it into a callback on init or later.

WORKED EXAMPLE: THE TWO HOOKS OF MAILPIT-SMTP.PHP

Since Lesson 1.2 a must-use plugin has been sending your site's email to Mailpit. It is two hooks. Find them:

```
grep -n "add_filter\|add_action" wp-content/mu-plugins/mailpit-smtp.php
```

```
11:add_filter( 'wp_mail_from', function ( $from ) {
15:add_action( 'phpmailer_init', function ( $phpmailer ) {
```

A filter on `wp_mail_from` (the sender address: a value, so the closure returns it) and an action on `phpmailer_init` (a moment: WordPress hands over its mailer object and the closure configures it for Mailpit). Ask the running site whether the filter is attached, and run the value through it the way wp_mail() does:

```
wp eval 'var_dump( has_filter( "wp_mail_from" ), apply_filters( "wp_mail_from", "w
```

```
bool(true)
string(21) "wordpress@example.com"
```

`has_filter()` without a callback answers "is anything attached?" (with a callback it returns the priority). And `apply_filters()` is all it takes to ask a filter for its answer: `wordpress@localhost`, which the mailer would reject, comes back as `wordpress@example.com`. Both callbacks are closures, so nobody else can remove them: fine for a must-use plugin on your own site.

GUIDED LAB

- From the project folder: `cp labs/module-03/l2_3-hooks/filter_chain_starter.php wordpress/`, then `wp eval-file --use-include filter_chain_starter.php`.
- Do the TODOs in order and run the file after each one. TODO 2 needs both values of the hook: what must `add_filter()` be told?
- Finished early? `hook_finder_starter.sh`, the title-parts folder lab (copy `title_parts_starter/pagesco-title.php` into `wordpress/wp-content/mu-plugins/`, then run `test_title_parts.sh`), then `debug_price_tag.php`.

QUESTION	YOUR ANSWER
What did remove_filter() return without the priority?	
What changes if the capitalizing callback runs at 20?	

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (A filter on one hook, and a hook nobody filters.)

```
wp eval 'add_filter( "pagesco_p", "strrev" ); echo apply_filters( "pagesco_p",
```

P4

Predict the output, then run it. (What an action gives back.)

```
wp eval 'add_action( "pagesco_p", function () { echo "ran "; } ); var_dump( do
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Pages & Co. Store Notice (lesson page, "Project: The Pages & Co. Store Notice"). In class: Steps 1 to 3. At home: Step 4 and the checklist. This file stays: Session 4 starts from it.

- ☐ Step 1: `wordpress/wp-content/mu-plugins/pagesco-site.php` prints the notice right after `<body>`
- ☐ Step 2: the filter `pagesco_store_notice` and the actions `pagesco_before_store_notice` and `pagesco_store_notice_end`, each with a DocBlock
- ☐ Step 3: the Thursday hours (priority 20), the See events link, the generator removal
- ☐ Step 4: pagesco-hello.php deleted; `has_action()` says 10, 20 and false; `grep -c` counts 0 generator tags
- ☐ Returning '' from my own test filter hid the bar; the test is removed again

SELF-CHECK

Q7

What does this print when run with wp eval-file?

```php
<?php
add_filter( 'pagesco_q7', fn( $text ) => $text . 'b', 20 );
add_filter( 'pagesco_q7', fn( $text ) => strtoupper( $text ) );
add_filter( 'pagesco_q7', fn( $text ) => $text . 'a', 5 );

echo apply_filters( 'pagesco_q7', 'x' ), "\n";
```

Q8

A members page must send visitors who are not logged in to the login screen. On which action do you call wp_safe_redirect() and exit?

A) `plugins_loaded`

B) `wp_footer`

C) `template_redirect`

D) `shutdown`

Q9

A theme adds `add_action( 'wp_head', 'pagesco_theme_banner' )` inside its after_setup_theme callback. Your plugin calls `remove_action( 'wp_head', 'pagesco_theme_banner' )` at the top of its main file, and the banner stays. Why?

A) The plugin ran before the theme added the hook, so there was nothing to remove: remove it on init

B) remove_action() only works on core's hooks

C) The priority is wrong: it must be 1

D) Plugins cannot change what a theme does

Q10

What does this print, and what does the value mean?

```
wp eval 'var_dump( has_filter( "the_content", "do_blocks" ) );'
```

Q11

After a new the_title filter, posts have no titles and the only message is `Deprecated: str_replace(): Passing null to parameter #3 ... in .../wp-includes/formatting.php on line 5777`. What is the likely bug?

A) Your callback does not return the title, so the next callback (core's, at priority 11) received null

B) A bug in WordPress 7.1's formatting.php: update core

C) PHP 8.5 deprecated str_replace()

D) The filter needs 2 accepted arguments

Q12

Your plugin prints a gift-card form. Other developers should be able to print a line of their own just BEFORE the form. What do you add right before you print it?

A) `apply_filters( 'pagesco_before_gift_form', '' );`

B) `add_action( 'pagesco_before_gift_form', 'pagesco_print_form' );`

C) `do_action( 'before_gift_form' );`

D) `do_action( 'pagesco_before_gift_form' );` with a DocBlock above it

◆ JOURNAL

Describe a website you use often as if it were built with hooks. Name three actions it would fire during a visit and one filter it might run on something you see. Which of them would you want to hook into if you were a developer working for that site, and what would your callback do?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit before Checkpoint 1.

E1

Why should code change WordPress data through WordPress (functions or WP-CLI) and not with UPDATE or DELETE?

E2

Action or filter, and which hook: a banner at the very top of every page; the length of every excerpt.

E3

`remove_action()` returns false and the hook keeps running. Name two possible reasons.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the inventory (Steps 3 and 4) and the store notice (Step 4, and Step 3 if needed); `mu-plugins/` = mailpit-smtp.php + pagesco-site.php
- ☐ Lesson 2.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 2.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); each exercise file deleted afterwards
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Both "Before the Next Lesson" lists (no leftover files, the orphan query, default-filters.php, mailpit-smtp.php)
- ☐ Checkpoint 1 in Session 4 (Modules 1 and 2): reread the Key Takeaways of Lessons 1.1 to 2.3, redo the self-checks of Sessions 1 to 3
- ☐ Optional: `wpdb_report_starter.php`, `debug_meta_report.php`, `hook_finder_starter.sh`, the title-parts lab, `debug_price_tag.php`
- ☐ Bring to Session 4: the site with the full store notice on every page, the project folder with `labs/`, my journal and this workbook

GLOSSARY

TERM	MEANING
drop-in	A reserved file name in `wp-content/` (object-cache.php, db.php ...) that replaces part of core
`$table_prefix`	The first letters of every table name (`wp_`), set in wp-config.php; in code `$wpdb->prefix`
pluggable function	A core function (wp_mail(), wp_get_current_user()) loaded after plugins, so a plugin may define it first
`$wpdb`	WordPress's database object: `get_var()`, `get_col()`, `get_row()`, `get_results()`, `prepare()`, table names
key-value (EAV) pattern	Extra fields stored as rows of (object, key, value) in the four meta tables
serialized value	An array saved as PHP's `serialize()` text, lengths included (`a:1:{s:6:"editor";b:1;}`)
autoload	Whether an option is loaded with every request (`on`, `auto`, `auto-on`) or on demand (`off`, `auto-off`)
orphan	A meta or term row whose post no longer exists

TERM	MEANING
action / filter	A hook for a moment (your callback does something) / for a value (your callback returns it)
callback	The function a hook calls: a name, a closure, an object or static method
priority	The third argument of add_action/add_filter: lower runs first, default 10
accepted args	The fourth argument: how many of the hook's values the callback receives, default 1
must-use plugin	A PHP file in `wp-content/mu-plugins/`: loaded on every request, before plugins, no activation

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 4: Theme Anatomy and Building a Block Theme

Your workbook for Session 4: the warm-up, Checkpoint 1 and its rubric, the key ideas of Lessons 3.1 and 3.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 3.1–3.2 | Session 4 · 3 hours | Name: _____

SESSION GOALS

- Show what you can do with Modules 1 and 2 in Checkpoint 1: hooks, WordPress functions, the content model, a prepared query, escaping.
- Tell block, classic and hybrid themes apart, and find your way around Twenty Twenty-Five's files.
- Predict which template draws a URL from the template hierarchy, and watch WordPress make the choice.
- Build the first version of pagesco: style.css, a theme.json design system, a header part and the index template, active on your site.
- Know the three quiet failures of a block theme: invalid JSON, a wrong part slug, a Site Editor copy.

WARM-UP

Write your answer, then run the command on your site (the end of Lesson 2.3).

W1

Session 3: priorities: What does this print on the site of Lesson 2.3?

```
wp eval 'echo has_filter( "pagesco_store_notice", "pagesco_notice_late_opening" ),
```

W2

Session 3: must-use plugins: Which two names does this list on your site, and why does neither need activating?

```
wp plugin list --status=must-use --fields=name,status
```

W3

Session 3: removing a hook: Why does this print 0?

```
curl -s http://localhost:8080/about/ | grep -c 'name="generator"'
```

CHECKPOINT 1 (MODULES 1–2)

THE TASK: THREE ADDITIONS TO THE STORE NOTICE AND THE FOOTER

A readiness check, not a grade that follows you. The starter `pagesco-checkpoint.php` is a must-use plugin with the three functions already named; its comments repeat this page.

- **Start:** `cp labs/module-04/checkpoint-1/checkpoint_starter/pagesco-checkpoint.php wordpress/wp-content/mu-plugins/` and `cp labs/module-04/checkpoint-1/test_checkpoint.sh wordpress/`, then `docker compose run --rm wpcli sh test_checkpoint.sh` after each part.
- **(a) Latest news.** `pagesco_checkpoint_latest_news()` on the filter `pagesco_store_notice` (your hook from Lesson 2.3), priority 30: add " Latest news: <title>." with the title of the newest post in the category News (slug `news`), found with `get_posts()` and an arguments array. Return the text.
- **(b) Ask Maria.** `pagesco_checkpoint_ask_link()` on the action `pagesco_store_notice_end`, priority 20: print " · " and a link to the author archive of the user with the login `maria`, with the text "Ask " and her display name. Escape the URL and the text.

- **(c) Page count.** `pagesco_checkpoint_page_count()` on `wp_footer` : `<p class="pagesco-pages">Published pages: N</p>` , N counted with ONE prepared `$wpdb` query on the posts table.
- **Rules:** 23 minutes, alone, open site, closed notes (your own course site, WP-CLI with `wp help` , the starter's comments; no lesson pages, notes, neighbors or AI assistants). Partial work is fine. Done = the test shows the newest post, the link and the count, with no message in the page or in debug.log.

HOW IT IS MARKED (2 / 1 / 0 PER ROW, 10 POINTS)

CRITERION	2 · MEETS	1 · PARTLY	0 · NOT YET
Hooks, names and priorities	`add_filter( 'pagesco_store_notice', ..., 30 )` and `add_action( 'pagesco_store_notice_end', ..., 20 )` with the callbacks named as strings; the filter RETURNS the text; the order in the bar is late opening, latest news, events link, Ask Maria.	Works, but a priority is off (Ask Maria before See events) or the filter is attached with add_action.	The filter echoes instead of returning (the notice text disappears), or a hook name is misspelled.
WordPress functions with arguments	`get_posts()` with an arguments array (`category_name` news, `numberposts` 1); `get_user_by( 'login', 'maria' )` ; `get_author_posts_url()` ; each result checked before use.	Right answer by a detour (all posts fetched and filtered in PHP, the URL typed by hand, the title or name hard-coded).	The newest post or Maria's link is wrong or missing.
Escaping at output	`esc_url()` on the link and `esc_html()` on its text; the filter returns PLAIN text, because the notice escapes it (Lesson 2.3).	One part unescaped, or the filter's text escaped a second time.	User data printed with no escaping at all.
A prepared query	ONE `$wpdb->get_var( $wpdb->prepare( ... ) )` on `{$wpdb->posts}` with `%s` for page and publish; the count printed as a number.	Correct count, but the values are written into the SQL string, or the table name is typed as wp_posts.	No query (a number typed in, or the count of the wrong post type).
Clean, standard code	The test prints the three expected lines and NO message in the page or in debug.log; every function has the pagesco_ prefix and a DocBlock; WordPress spacing and tabs.	One message, or the prefix or DocBlocks missing on some functions.	The site shows a warning or a critical error, or the work is unfinished.

AFTER THE 23 MINUTES

What the test printed, and one line: what would you fix with ten more minutes?

LESSONS

3.1 Theme Anatomy & the Template Hierarchy

BY THE END YOU CAN

- Explain what belongs in a theme and what belongs in a plugin, and where WordPress records which theme is active
- Tell block, classic and hybrid themes apart by their files, and confirm it with wp_is_block_theme() and wp_theme_has_theme_json()
- Find your way around a block theme: style.css, theme.json, templates/, parts/, patterns/, styles/ and functions.php in Twenty Twenty-Five
- Predict which template WordPress uses for a URL, for block themes and classic themes, from the template hierarchy
- Watch WordPress make that choice on a running site with the *_template_hierarchy filters and template_include

KEY IDEAS

- **A theme owns presentation**: content stays in the database, features in plugins; switching themes keeps both. The `template` and `stylesheet` options record the active theme.
- **Block** themes: templates/*.html, parts/*.html, theme.json (true, true). **Classic**: PHP templates (false, false). **Hybrid**: classic plus theme.json (false, true).
- **Twenty Twenty-Five**: style.css (the header), theme.json (settings and styles), templates/, parts/, patterns/ (3.3), styles/ (3.3), functions.php (hooks, 4.3). Read it; never edit it.
- **The template hierarchy**: a candidate list from most specific to index; the first file the theme has wins. Slugs, not titles; posts include the post type (single-post-hello-world).
- **front-page** = the site's front page, whatever it shows; **home** = the list of latest posts.
- **Block templates** are rendered by core's template-canvas.php, which calls wp_head(), wp_body_open() and wp_footer(); a classic theme must call them itself.
- **Tools:** `get_template_hierarchy()`, `get_block_templates()`, the `{$type}_template_hierarchy` and `template_include` filters; Site Editor saves are `wp_template` posts that win over files.

WORKED EXAMPLE: BLOCK OR CLASSIC, WITHOUT SWITCHING

Lesson 3.1 asked the ACTIVE theme what kind it is. You can ask any installed theme without switching to it. First its files (in the WP-CLI container shell):

```
ls -1 wp-content/themes/twentytwentythree
```

```
assets
parts
patterns
readme.txt
screenshot.png
style.css
styles
templates
theme.json
```

`templates/` and `theme.json`, no `index.php`: a block theme, and no functions.php at all (Twenty Twenty-Three needs no PHP). WordPress agrees: `wp_get_themes()` returns a `WP_Theme` object per folder, and `is_block_theme()` checks for templates/index.html:

```
wp eval 'foreach ( wp_get_themes() as $slug => $t ) { printf( "%-18s block: %s\n",
```

```
twentytwentyfive   block: yes
twentytwentyfour   block: yes
twentytwentythree  block: yes
```

On a client site with ten themes installed, this one line tells you which are block themes before you switch to any of them.

GUIDED LAB

- From the project folder: `cp labs/module-04/l3_1-theme-anatomy/which_template_starter.php wordpress/`, then `wp eval-file --use-include which_template_starter.php`.
- Before any code: write your prediction (which Twenty Twenty-Five template answers) for each URL here:

URL	MY PREDICTION	WORDPRESS SAYS
/tag/staff-picks/		
/author/devon/		
/hello-world/		
/?s=dune		
/		

URL	MY PREDICTION	WORDPRESS SAYS
/contact/		

- TODOs 1 to 3 in VS Code (`wordpress/which_template_starter.php`); run again after each. Done when the output matches "Expected when done". Then `rm wordpress/which_template_starter.php` .
- Finished early? `theme_inspector_starter.sh` , then `debug_theme_report.php` in the same folder.

PREDICT, THEN RUN

Write your prediction first, then run it on your site: `wp ...` commands from the project folder, the others in the WP-CLI container shell (`docker compose run --rm wpcli sh`).

P1

Predict the output, then run it. (The front page's candidate list.)

```
wp eval 'echo implode( ", ", get_template_hierarchy( "front-page" ) ), "\n";'
```

P2

Predict the output, then run it. (Twenty Twenty-Five's template parts.)

```
wp eval 'echo count( get_block_templates( array(), "wp_template_part" ) ), "\r
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Map the Templates of Pages & Co. (lesson page, "Project"). In class: Step 1. At home: Steps 2 to 5.

- ☐ Step 1: every URL requested with the logger in place; the TEMPLATE MAP in `pagesco-plan.txt` (URL, the Twenty Twenty-Five template, what pagesco needs)
- ☐ Step 2: the sketch theme (style.css + index.php) answers false, false; the About page used index.php

- ☐ Step 3: page.php and a theme.json: false, true (hybrid); --wp--preset--color--ink in the page
- ☐ Step 4: templates/index.html: true; the post used the block template, /about/ still page.php. Why?
- ☐ Step 5: Twenty Twenty-Five active, pagesco-sketch and pagesco-template-map.php deleted; two must-use plugins

SELF-CHECK

Q1

Twenty Twenty-Five is active. Which candidate list does WordPress build for the About page, `/about/` (page ID 12)?

A) `about, page, index`

B) `page-about, page-12, page, singular, index`

C) `single-page-about, single-page, single, singular, index`

D) `page-12, page-about, page, index`

Q2

What does this print on the course site?

```
wp eval 'echo implode( ", ", get_template_hierarchy( "author-devon" ) ), "\n";
```

Q3

A client's classic theme has `wp_head()` in header.php and `wp_footer()` in footer.php, but no `wp_body_open()`. What happens to the Pages & Co. store notice?

A) A PHP warning names the missing function

B) WordPress calls wp_body_open itself, as it does for block themes

C) The notice moves to the footer

D) It disappears, with no message anywhere

Q4

Every function in Twenty Twenty-Five's functions.php is wrapped in `if ( ! function_exists( ... ) )`. Why?

A) So a child theme can define a function of the same name first and replace it

B) So WordPress can load functions.php twice

C) PHP 8.5 requires it for theme functions

D) So the functions run only in the block editor

Q5

Settings > Reading shows the latest posts on the front page. The theme has front-page.html, home.html and index.html. Which template draws http://localhost:8080/?

A) home.html

B) index.html

C) front-page.html

D) The theme cannot have both front-page.html and home.html

Q6

Name the filter that changes the list of candidate templates for category archives, before WordPress looks for files.

JOURNAL

Look at your template map. If Pages & Co. wanted its News category to look different from every other archive, which file would you add, and what would the hierarchy do for every other category? Then think of a site you visit often: which of its pages look like they share one template, and which look like they have their own?

3.2 Building a Block Theme

BY THE END YOU CAN

- Write a theme header in style.css that WordPress reads and checks on activation
- Define a design system in theme.json: settings (palette, fonts, sizes, spacing, layout) and styles (global, elements, blocks), and read the CSS WordPress generates from it
- Write templates and template parts in block markup, combining static blocks with dynamic ones such as the Query Loop, Post Title and Post Content
- Activate and verify a block theme with WP-CLI and the Site Editor, with site features such as the store notice still working
- Diagnose an invalid theme.json, a missing template part and a Site Editor copy that overrides your file

KEY IDEAS

- **Two files make a block theme**: style.css (the header WordPress reads and checks on activation: Requires at least, Requires PHP) and templates/index.html.
- **theme.json settings = what is available** (palette, font families and sizes, spacing, layout widths, controls); **styles = what is applied** (the page, elements, single block types).
- **Every preset becomes a custom property and classes**: oxblood → `--wp--preset--color--oxblood`, `.has-oxblood-color`; `var:preset|color|oxblood` in theme.json. Use presets, never hex values.
- **Static blocks** carry their exact HTML in the file; **dynamic blocks** (Site Title, Navigation, Query Loop, Post Title, Post Content) are self-closing and rendered by PHP.
- **Template parts** live in parts/ and are included by slug; `tagName` gives the header, main and footer landmarks.
- **Quiet failures**: a JSON syntax error drops the whole theme.json (a Notice only with WP_DEBUG); a wrong part slug renders nothing for visitors; a Site Editor copy wins over your file.

WORKED EXAMPLE: FROM A PRESET TO A CLASS TO THE CSS

On your site after Lesson 3.2: follow the footer's dark band from the template part to the page. The part asks for presets by slug:

```
grep -o '"backgroundColor":"ink","textColor":"paper"' wp-content/themes/pagesco/pa
```

```
"backgroundColor":"ink","textColor":"paper"
```

WordPress turns the attributes into classes on the Group's `<div>`:

```
curl -s http://localhost:8080/ | grep -o '<div class="wp-block-group alignfull has
```

```
<div class="wp-block-group alignfull has-paper-color has-ink-background-color has
```

and prints one rule per palette color, pointing at the custom property:

```
curl -s http://localhost:8080/ | grep -o '\.has-ink-background-color{[^}]*}'
```

```
.has-ink-background-color{background-color: var(--wp--preset--color--ink) !import
```

Three steps, one name: `ink`. Change the color of `ink` in theme.json (or let a style variation do it in Lesson 3.3) and the footer follows, because no file contains #1f2a2e except theme.json. The `!important` makes a color chosen for one block beat every other rule.

GUIDED LAB

- Note your active theme: `wp theme list --status=active --field=name`.
- `cp -r labs/module-04/l3_2-block-theme/book_fair_starter wordpress/wp-content/themes/pagesco-fair`, `wp theme activate pagesco-fair`, `cp labs/module-04/l3_2-block-theme/test_book_fair.sh wordpress/`, then `docker compose run --rm wpcli sh test_book_fair.sh`.
- The four TODOs are in the theme's style.css. Run the test after each one and note which line changed.
- Done when the test prints all three colors, the page colors, `posts listed: 4` and `footer parts: 1`. Then switch back to your theme and `wp theme delete pagesco-fair`; `rm wordpress/test_book_fair.sh`.

QUESTION	YOUR ANSWER
TODO 1 also changed the header. Why?	
Why are links night, not lantern?	
Which test line proves styles uses a preset?	

PREDICT, THEN RUN

Write your prediction first, then run it on your site: `wp ...` commands from the project folder, the others in the WP-CLI container shell (`docker compose run --rm wpcli sh`).

P3

Predict the output, then run it. (On your site after Lesson 3.2: a page that does not exist.)

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/nothing-here/
```

P4

Predict the output, then run it. (On your site after Lesson 3.2: the class rule of a palette color.)

```
curl -s http://localhost:8080/ | grep -o '\.has-oxblood-color{[^}]*}'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Finish the pagesco Theme (lesson page, "Project"). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ Before: style.css, the first index.html, theme.json, parts/header.html and the finished index.html from the lesson; pagesco active
- ☐ Step 1: the full theme.json; the link rule and the .pagesco-notice CSS in the page
- ☐ Step 2: parts/footer.html; the footer text on the home page
- ☐ Step 3: single, page, archive, search and 404 templates; get_block_templates() lists six, all source theme
- ☐ Step 4: the per-URL loop: every page 200 (404 for /nothing-here/) with notice:1 header:1 footer:1 and its h1

SELF-CHECK

Q7

On the course site after Lesson 3.2 (pagesco active), what does this print?

```
wp eval 'echo wp_get_global_settings( array( "layout", "wideSize" ) ), "\n";'
```

Q8

In theme.json, `"fontSize": "var:preset|font-size|large"` under styles. What does WordPress print in the page's CSS?

A) `font-size: large`

B) `font-size: 1.75rem`

C) `font-size: var(--wp--preset--font-size--large)`

D) `font-size: var(--large)`

Q9

Which of these blocks is written SELF-CLOSING in a template file, with no HTML of its own?

A) Group

B) Heading

C) Paragraph

D) Post Title

Q10

A live site (WP_DEBUG off) gets a theme.json with a trailing comma. What do visitors see?

- A) The site in core's default design: the theme's colors, fonts and sizes are gone, with no message
- B) A critical error page
- C) The notice wp_json_file_decode(): Syntax error at the top of every page
- D) Nothing changes: WordPress skips the bad line

Q11

A template contains `<!-- wp:template-part {"slug":"footer","tagName":"footer"} /-->`. Which file of the theme does it include, and what does the tagName do?

Q12

What does `"defaultPalette": false` in settings.color do?

- A) Deletes the theme's own palette
- B) Hides WordPress's own palette from the editors, so they choose from the theme's colors
- C) Turns off all color controls
- D) Removes --wp--preset--color--black and --white from the page

◆ JOURNAL

You wrote a style guide (theme.json) before the floor plans (templates). Imagine the shop rebrands next year with new colors and a new typeface. List exactly which files of pagesco you would have to touch, and which you would not. What does that tell you about where design decisions should live?

EXIT TICKET

Three questions, no notes.

E1

A block theme has index.html, page.html and archive.html. Which draws /author/maria/, and why?

E2

In theme.json, where does the shop's palette go, and where the page's background color?

E3

Your header vanished and there is no message. Name two quiet causes.

NOTES

HOMEWORK CHECKLIST

- ☐ 3.1 project Steps 2 to 5 (sketch theme, then everything deleted again; the logger too)
- ☐ 3.2 project Steps 3 and 4 (five templates, the per-URL check); pagesco active, no saved copies
- ☐ Lesson 3.1 Exercises 1 to 3; Lesson 3.2 Exercises 1 to 3, each undone (Exercise 4 of each is a stretch)
- ☐ Read 3.2 Common Mistake 3 (the Site Editor copy) and the FAQ

- ☐ Both Quick Quizzes and both journal entries
- ☐ Checkpoint 1 finished at home if I did not finish in class
- ☐ Optional: `theme_inspector_starter.sh`, `debug_theme_report.php`, `theme_json_reader_starter.php`, the café handout
- ☐ Bring to Session 5: pagesco active, my plan with the template map, my journal, this workbook

GLOSSARY

TERM	MEANING
block theme / classic theme / hybrid	templates/*.html + theme.json / PHP templates / a classic theme with a theme.json
template hierarchy	The candidate list of template names WordPress builds for a request, most specific first, ending in index
front-page / home	The site's front page, whatever it shows / the list of latest posts
`template-canvas.php`	Core's file that renders a block template and prints the page around it (wp_head, wp_body_open, wp_footer)
`theme.json`	A theme's design system: settings (what is available) and styles (what is applied)
preset	A named design value (color, font size, spacing) that becomes `--wp--preset--<type>--<slug>` and classes
template part	A file in parts/ (header, footer) included with the Template Part block by slug
static / dynamic block	Saves its HTML in the content / self-closing, rendered by PHP on every request
Query Loop (`inherit`)	Repeats its Post Template for each post; inherit = whatever the main query found
`wp_template`, `wp_template_part`	Site Editor copies saved as posts; they win over the theme's files

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 5: Patterns, Style Variations and Child Themes

Your workbook for Session 5: the warm-up, the key ideas of Lessons 3.3 and 3.4, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 3.3–3.4 | Session 5 · 3 hours | Name: _____

SESSION GOALS

- Write pattern files with the right header, translatable text and theme URLs, and use them in a template by slug.
- Choose between a theme pattern, a synced pattern and a template part, and explain why a theme never ships a synced pattern.
- Tell a section style from a style variation, and follow a Site Editor change from the database back into the right file.
- Build a child theme: the Template header, theme.json merged (preset lists replaced), the stylesheet enqueued, functions.php's load order.
- Decide whether a change belongs in the Site Editor, a child theme, your own theme or a plugin.

START HERE: WARM-UP AND READINESS

WARM-UP (SESSION 4)

Answer W1 to W4 from memory, then run W5's two commands on your site. Your instructor goes through the answers.

W1

In theme.json, what is the difference between settings and styles?

W2

The front page shows the latest posts. In a block theme, which templates does WordPress look for, in order?

W3

The manager changed the Single Posts template in the Site Editor. Where is that change, and which version does a visitor see?

W4

A template part is included as {"slug":"heder"}. What do visitors see? And a logged-in administrator?

W5

Run the two commands on the slide. What should a site ready for today print?

```
wp eval 'var_dump( wp_is_block_theme(), wp_theme_has_theme_json() );'
wp post list --post_type=wp_template,wp_template_part,wp_global_styles --format=cou
```

If W5's second line is not `0`, you have Site Editor copies: list them with `wp post list --post_type=wp_template,wp_template_part,wp_global_styles --fields=ID,post_type,post_name` and delete each with `wp post delete <ID> --force` before the demo.

LESSONS

3.3 Patterns, Style Variations & the Site Editor

BY THE END YOU CAN

- Write PHP pattern files with the right header fields, translatable text and escaped theme URLs, and place them in templates with the Pattern block
- Explain the difference between theme patterns, synced patterns and template parts, and create a synced pattern that updates every page it is used on
- Define section styles and a style variation as JSON files in styles/ and read what the Site Editor saves when someone chooses one
- Bundle a web font with fontFace in theme.json, checking that its license allows redistribution
- Bring Site Editor changes back into the theme's files with Export and WP-CLI, then reset the database copies

KEY IDEAS

- **Three tools:** a theme pattern (a PHP file in patterns/, copied when inserted), a synced pattern (a `wp_block` post, one original shown everywhere), a template part (parts/, included by slug).
- **A pattern file** = a header (Title and Slug required; Categories, Keywords, Description, Post Types, Template Types, Block Types, Inserter) + block markup. PHP makes it translatable (`esc_html_e()`), portable (`get_theme_file_uri()` , `home_url()`) and escaped.
- **The pattern-file list is cached** for 30 minutes, keyed on the theme's Version. `WP_DEVELOPMENT_MODE` = `theme` switches the theme caches off: local sites only.
- **Templates use patterns by slug**: `<!-- wp:pattern {"slug":"pagesco/hero"} /-->` . `front-page.html` is above home and index for the front page.
- **Synced patterns are content**, referenced by ID (`<!-- wp:block {"ref":20} /-->`): never in theme files.
- **styles/*.json** with `blockTypes` = a section style (`is-style-<slug>`); without it = a style variation for the whole site. Choosing one in the Site Editor saves user global styles (a `wp_global_styles` post).
- **fontFace** bundles a web font (`file:./assets/fonts/...`): only fonts whose license allows redistribution (OFL 1.1).
- **The Site Editor expands patterns when it saves.** List copies with `wp post list --post_type=wp_template,wp_template_part,wp_global_styles` , move each change into the right file, delete the copy.

WORKED EXAMPLE: WHERE THE FRONT PAGE'S DESIGN LIVES

At the end of Lesson 3.3 the design of the front page comes from three different places. Ask WordPress about each. First the theme's patterns, with their categories and the post types they are offered in:

```
wp eval 'foreach ( WP_Block_Patterns_Registry::get_instance()->get_all_registered(
```

```
pagesco/call-to-action | call-to-action | (any)
pagesco/featured-books | featured | (any)
pagesco/hero | banner, featured | wp_template
```

Only the hero says `wp_template`: it is offered only in the Site Editor's templates, because inserted into a page its `<h1>` would compete with the page title. Then the synced pattern, which is not in the theme at all:

```
wp post list --post_type=wp_block --fields=ID,post_title,post_name
```

```
+----+---------------+---------------+
| ID | post_title    | post_name     |
+----+---------------+---------------+
| 20 | Opening hours | opening-hours |
+----+---------------+---------------+
```

A post of type `wp_block`, in the database. Is it synced? An unsynced user pattern carries the meta key `wp_pattern_sync_status`; count its meta:

```
wp post meta list 20 --format=count
```

```
0
```

No meta, so synced: every page that references it by ID shows the one original. Files for the theme's design, posts for the owner's content.

GUIDED LAB

- From the project folder: `cp -r labs/module-05/l3_3-patterns-style-variations/hours_band_starter wordpress/wp-content/themes/pagesco-lab-hours`, then `wp theme activate pagesco-lab-hours` (a one-file child theme of pagesco: your own theme stays untouched).
- `cp labs/module-05/l3_3-patterns-style-variations/test_hours_band.php wordpress/`, then `wp eval-file --use-include test_hours_band.php`. Open `wordpress/wp-content/themes/pagesco-lab-hours/patterns/hours-band.php` in VS Code.

- Do TODOs 1 to 5, running the test after each. The test translates the child's strings into capitals and moves the site to pagesandco.example: lower case means not translatable, localhost means broken on the live site.

QUESTION	YOUR ANSWER
Which text domain made the heading change case, and why that one?	
What would the hard-coded link have done on the live site?	
Which header field decides where the inserter files the pattern?	

- Done when the heading prints `VISIT THE SHOP` and the button links to `https://pagesandco.example/contact/`. Then `wp theme activate pagesco` and remove the folder and the test.
- Finished early? `debug_pattern_files.sh` in the same folder (`docker compose run --rm wpcli sh debug_pattern_files.sh`). At home, after the Evening section: `variation_contrast_starter.php`.

PREDICT, THEN RUN

Write your prediction first, then run it on your site (`wp ...` from the project folder, `curl` in the WP-CLI container shell). The site is the one at the end of the lesson. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (The hero pattern's Post Types header, as WordPress stored it.)

```
wp eval 'var_dump( WP_Block_Patterns_Registry::get_instance()->get_registered(
```

P2

Predict the output, then run it. (A page that uses the synced pattern: what reaches the browser?)

```
curl -s http://localhost:8080/about/ | grep -c 'wp:block'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Pages & Co. Front Page (lesson page, "Project: The Pages & Co. Front Page"). In class: Step 1 and the start of Step 2. At home, FIRST the lesson's Evening and web-font sections (Step 4's check expects the font), then the rest.

- ☐ Step 1: `patterns/featured-books.php` registered; the loop prints Dune, The Hobbit, Fahrenheit 451
- ☐ Step 2: `styles/sections/section-ink.json` and `patterns/call-to-action.php`
- ☐ Step 3: the final `templates/front-page.html` (hero, staff picks, call to action, news)
- ☐ Step 4: `screenshot.png` (1200 × 900), the registered patterns, the file list, no Site Editor copies, the per-URL loop

SELF-CHECK

Q1

The live site has no `WP_DEVELOPMENT_MODE`. You deploy pagesco with a new `patterns/gift-cards.php`, and the inserter does not show it for half an hour. What makes WordPress read the patterns folder again at once, as part of every release?

A) Raise the theme's Version in style.css

B) Call `register_block_pattern()` for the file in functions.php

C) Run `wp cache flush`

D) Rename `patterns/` and back

Q2

On the site at the end of Lesson 3.3, what does this print, and what are the two numbers?

```
wp eval 'echo count( WP_Theme_JSON_Resolver::get_style_variations() ), " ", co
```

Q3

Which of these lines should never be written into a template file in a theme?

A) `<!-- wp:pattern {"slug":"pagesco/hero"} /-->`

B) `<!-- wp:template-part {"slug":"footer"} /-->`

C) `<!-- wp:block {"ref":20} /-->`

D) `<!-- wp:query {"query":{"inherit":true}} -->`

Q4

After the shop manager saved the front page in the Site Editor, the `wp_template` copy contains `"patternName":"pagesco/hero"` and `src="http://localhost:8080/..."`. What does that tell you?

A) The copy still refers to hero.php, so changes to the file will show up

B) The Site Editor could not read the pattern file and left a placeholder

C) Nothing: patternName is only used for the inserter's preview

D) The pattern was expanded before saving: its PHP already ran, so the URL is hard-coded and the strings are frozen in English

Q5

You move `section-ink.json` from `styles/sections/` up into `styles/`. What changes?

A) It becomes a style variation for the whole site

B) Nothing: it is still a section style, because it has `blockTypes`

C) WordPress ignores it: section styles must be in styles/sections/

D) It replaces `evening.json`, because both are in styles/ now

Q6

The page source announces Vollkorn in a @font-face rule, but every browser shows the fallback serif, and debug.log is empty. Where do you look first, and for what?

JOURNAL

Today the design of Pages & Co. lives in four places: theme files, synced patterns, user global styles and Site Editor template copies. For each, write who should be allowed to change it (you, the shop manager, any editor) and how you would notice if it changed. Where would you draw the line between "the developer's design" and "the owner's content" for this shop?

3.4 Child Themes & Customizing Existing Themes

BY THE END YOU CAN

- Create and activate a child theme with the Template header, and explain the template and stylesheet options it sets
- Predict how a child's theme.json, templates, template parts, patterns and style variations combine with the parent's
- Use a child functions.php to enqueue the child's stylesheet, replace a pluggable parent function and remove a parent hook at the right time
- Build a classic child theme and explain how get_stylesheet_directory(), get_template_directory() and locate_template() choose files
- Choose between the Site Editor, a child theme, your own theme and a plugin for a requested change

KEY IDEAS

- **A child theme** is a theme folder whose style.css says `Template: <parent folder>`. WordPress looks in the child first and falls back to the parent; the parent stays untouched and can be updated.
- **Two options:** `template` = the parent, `stylesheet` = the child. `get_template_directory()` vs `get_stylesheet_directory()`; `get_theme_file_path()` / `_uri()` = the child's copy if it exists, else the parent's.
- **theme.json is merged** (core, parent, child, user), single values key by key, but **preset lists are replaced whole**: repeat every slug, change only values.
- **Same name replaces:** templates, parts, classic PHP templates, styles/ files (by file name only); **same slug replaces** patterns.
- **functions.php:** both run, the child's FIRST. A block child's style.css loads only when enqueued (depend on the parent's real handle). Pluggable parent functions (`function_exists()`) can be replaced; parent hooks are removed on `after_setup_theme`.
- **Classic child themes** work the same with PHP templates (`locate_template()`, `get_header()`); never `@import` the parent's CSS. One level only: no grandchildren.
- **Which tool?** Site Editor (owner, database) · child theme (lasting design changes to someone else's theme) · your own theme · a plugin (features that must survive a theme switch).

WORKED EXAMPLE: WHAT A CHILD THEME OVERRIDES, AND WHAT IT INHERITS

The child theme of Lesson 3.4 has five files. Everything else comes from the parent:

```
find wp-content/themes/pagesco-tt5-child -type f | sort
```

```
wp-content/themes/pagesco-tt5-child/functions.php
wp-content/themes/pagesco-tt5-child/patterns/footer.php
wp-content/themes/pagesco-tt5-child/style.css
wp-content/themes/pagesco-tt5-child/templates/single.html
wp-content/themes/pagesco-tt5-child/theme.json
```

Which theme serves each template? `get_block_templates()` names them all after the active theme, so ask core's internal `_get_block_template_file()` (fine for checking, not for theme code), with the child switched on for a moment:

```
wp theme activate pagesco-tt5-child
wp eval 'foreach ( array( "single", "page", "404" ) as $slug ) { echo $slug, ": ",
wp theme activate pagesco
```

```
Success: Switched to 'Pages &amp;amp; Co. TT5 Child' theme.
single: pagesco-tt5-child
page: twentytwentyfive
404: twentytwentyfive
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

`single` is the child's copy (the one without comments); `page` and `404` are the parent's, and keep receiving Twenty Twenty-Five's updates. Override as little as you can.

GUIDED LAB

- `cp -r labs/module-05/l3_4-child-themes/tt4_child_starter wordpress/wp-content/themes/pagesco-lab-tt4-child`, then `wp theme activate pagesco-lab-tt4-child`.
- `cp labs/module-05/l3_4-child-themes/test_tt4_child.sh wordpress/`, then `docker compose run --rm wpcli sh test_tt4_child.sh`.
- Before TODO 1, open the PARENT: `wp-content/themes/twentytwentyfour/theme.json` (the palette), `functions.php` (handles, pluggable functions) and `parts/footer.html` (which pattern it shows).

INHERITANCE RULE	WHAT YOU CHANGED IN THE CHILD	THE TEST LINE THAT PROVES IT
theme.json palette		
style.css loaded		
pattern by slug		
pluggable function		

- Done when the test shows `#7a2e2e`, `child CSS: 1`, the shop's footer and only the checkmark list style. Then `wp theme activate pagesco` and remove the folder and the test.
- Finished early? `merge_layers_starter.php`, then `debug_child_theme.sh`.

PREDICT, THEN RUN

Write your prediction first, then run it on your site (`wp ...` from the project folder, `curl` in the WP-CLI container shell). The site is the one at the end of the lesson. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (The child has no assets/ folder. Which image URL?)

```
wp theme activate pagesco-tt5-child
wp eval 'echo get_theme_file_uri( "assets/images/book-image.webp" ), "\n";'
wp theme activate pagesco
```

P4

Predict the output, then run it. (Whose post-format function ran?)

```
wp theme activate pagesco-tt5-child
wp eval 'var_dump( function_exists( "twentytwentyfive_post_format_setup" ), ge
wp theme activate pagesco
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Finish pagesco-tt5-child (lesson page, "Project: Finish pagesco-tt5-child"). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ Step 1: `patterns/footer.php` with the slug `twentytwentyfive/footer`; the home page shows the shop's footer
- ☐ Step 2: the complete functions.php; the post formats are aside and quote, the checkmark style is gone

- ☐ Step 3: the check, one command per inheritance rule
- ☐ Step 4: `wp theme activate pagesco`: pagesco active, the child installed (needed for Session 6)

SELF-CHECK

Q7

A client uploads only your child theme (pagesco-tt5-child.zip) to a site where Twenty Twenty-Five has been deleted, and clicks Activate. What happens?

A) WordPress uses its default theme as the parent instead

B) It refuses: the parent theme is missing, and the theme is listed under Broken Themes

C) The child activates, unstyled, until the parent is installed

D) WordPress downloads Twenty Twenty-Five automatically

Q8

On the site at the end of Lesson 3.4: what do the two middle commands print?

```
wp theme activate pagesco-tt5-child
wp option get stylesheet
wp option get template
wp theme activate pagesco
```

Q9

A colleague's child theme.json lists only the two colors they wanted to change, base and contrast. What happens to Twenty Twenty-Five's accent colors?

A) They disappear: the child's palette replaces the parent's whole palette, and styles that use accent variables silently lose their colors

B) They stay: theme.json is merged key by key

C) WordPress prints a notice that the palette is incomplete

D) The child's two colors are added to the end of the parent's list

Q10

Why can a child of Twenty Twenty-Five replace `twentytwentyfive_post_format_setup()` by defining a function with that name in its own functions.php?

A) Child functions always override parent functions of the same name

B) WordPress renames the parent's function when a child theme is active

C) The parent's function is registered on a hook, and hooks prefer the child

D) The child's functions.php loads first, and the parent defines that function only `if ( ! function_exists( ... ) )`

Q11

Twenty Twenty-Four enqueues no stylesheet of its own. In a child of it you write wp_enqueue_style('my-child-style', get_stylesheet_uri(), $deps, $version). What should $deps be?

A) `array( 'twentytwentyfour-style' )`

B) `array( 'wp-block-library' )`

C) `array()`

D) `get_template_directory_uri() . '/style.css'`

Q12

The shop manager wants to try a darker link color for the holidays and switch it back in January, without calling you. Site Editor, child theme, or plugin? Why?

JOURNAL

Imagine a client on Twenty Twenty-Five whose previous developer edited the theme's files directly. The next update is waiting. Write the steps you would take, in order, to move their changes into a child theme without losing anything. Which of today's commands (diff, _get_block_template_file(), wp theme list) would you use, and where?

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Theme pattern or synced pattern: the shop's opening hours on four pages? And a hero section for new landing pages?

E2

Name two places where the Site Editor stores changes, and the command that lists them.

E3

In a child theme, which runs first, the child's functions.php or the parent's, and what does that mean for removing a parent hook?

NOTES

HOMEWORK CHECKLIST

- ☐ No Site Editor copies: `wp post list --post_type=wp_template,wp_template_part,wp_global_styles` lists nothing
- ☐ Lesson 3.3: the Evening variation, the web font, Export, Common Mistakes; the project Steps 2 to 4
- ☐ Lesson 3.4: the classic child theme section (needs the internet), Common Mistake 1; the project Steps 3 and 4
- ☐ pagesco is the active theme again; pagesco-tt5-child is installed but inactive
- ☐ Lesson 3.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 3.4 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Labs not reached: `variation_contrast_starter.php`, `merge_layers_starter.php`, the two debug handouts
- ☐ Bring to Session 6: the site at the end of Lesson 3.4, the project folder with `labs/`, `pagesco-plan.txt`, my journal, this workbook

GLOSSARY

TERM	MEANING
theme pattern	A PHP file in the theme's `patterns/` folder; inserted as a copy of its blocks
synced pattern	A `wp_block` post shown wherever `<!-- wp:block {"ref":ID} /-->` refers to it; one original
Pattern block	`<!-- wp:pattern {"slug":"..."} /-->`: a theme pattern used in a template by slug
`WP_DEVELOPMENT_MODE`	`theme` (also core, plugin, all): switches off the caches that get in a developer's way; never on production
section style	A JSON file in `styles/` with `blockTypes`: a named look for one Group or Columns block (`is-style-<slug>`)
style variation	A JSON file in `styles/` without `blockTypes`: an alternative look for the whole site
user global styles	The `wp_global_styles` post the Site Editor saves; the top layer over core and theme
`fontFace`	The theme.json key that describes bundled font files (`file:./assets/fonts/...`)
child theme	A theme with `Template: <parent folder>` that holds only changes; the rest comes from the parent
`template` / `stylesheet`	The two options that name the parent and the child (the same folder without a child theme)
pluggable function	A parent function wrapped in `if ( ! function_exists( ... ) )`, which a child may define first
`after_setup_theme`	The action that fires after both themes' functions.php files have loaded

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 6: The Loop and Custom Queries

Your workbook for Session 6: the warm-up, the key ideas of Lessons 4.1 and 4.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 4.1–4.2 | Session 6 · 3 hours | Name: _____

SESSION GOALS

- Explain what the Loop does with the main query, and write it with have_posts() and the_post().
- Use template tags in both forms, printed and returned, and escape what you print yourself.
- Split a classic theme into header, footer and template parts, and vary it with conditional tags asked at the right time.
- Ask for your own lists with WP_Query, read post_count, found_posts and max_num_pages, and reset the post afterwards.
- Change the main query with a guarded pre_get_posts callback, and paginate without 404s.

START HERE: WARM-UP AND READINESS

WARM-UP (SESSION 5)

Answer W1 to W4 from memory, then run W5's two commands on your site.

W1

Theme pattern or synced pattern: which one is a post in the database, and why does a theme never ship one?

W2

Two JSON files in styles/: what makes one a section style and the other a style variation?

W3

A child theme is active. What do the template and stylesheet options hold?

W4

Why did the child's style.css not load until functions.php enqueued it, and why must a parent hook be removed on after_setup_theme?

W5

Run the two commands on the slide. What should a site ready for today print?

```
wp theme list --fields=name,status
wp post list --post_type=wp_template,wp_template_part,wp_global_styles --format=cou
```

Still on the child theme? `wp theme activate pagesco` before the demo.

LESSONS

4.1 The Loop & Template Tags

BY THE END YOU CAN

- Explain what the Loop does with the main query, step by step, and write it correctly with have_posts() and the_post()
- Use template tags and their the_ / get_ pairs, and choose between printing and returning (and escaping) a value
- Split a classic theme into header.php, footer.php and template parts with get_template_part(), including passing arguments
- Use conditional tags such as is_home(), is_singular() and is_search(), and say when they work and when they are called too early
- Build pagesco-classic with index.php, singular.php and 404.php, and turn it into a hybrid theme with theme.json

KEY IDEAS

- **The main query runs before any template** and lives in `$wp_query`; a classic template only walks its result: `if ( have_posts() ) { while ( have_posts() ) { the_post(); ... } }`, even on single posts.
- **the_post()** moves to the next post and sets the global `$post` every template tag reads; without it the Loop never ends.
- **Template tags come in pairs:** `the_title()` prints HTML-ready text, `get_the_title()` returns it for you to escape (`esc_attr()` , `esc_url()` , `esc_html()`); `the_title_attribute()` for attributes. `get_the_content()` is raw; `the_content()` renders blocks. `the_date()` prints once per day.
- **Parts:** `get_header()` , `get_footer()` , `get_template_part( $slug, $name, $args )` through locate_template() (child first); no `.php` in the slug; false = not found.
- **Conditional tags** (is_home(), is_singular('post'), is_page('about'), is_archive(), is_search(), is_404() ...) read the main query: from template_redirect on, never at a plugin's top level.
- **A classic theme with a theme.json is a hybrid:** presets and global styles, but no styles.css custom CSS.

WORKED EXAMPLE: WHAT THE MAIN QUERY FOUND, AS BODY_CLASS() PRINTS IT

Every classic template starts from the same question: what did the main query find? The classes `body_class()` prints answer it in the page itself. With pagesco-classic active (the end of Lesson 4.1), from the WP-CLI container shell:

```
for url in / /about/ /category/news/ "/?s=dune" /nothing-here/; do
  printf '%-16s %s\n' "$url" "$(curl -s "http://localhost:8080$url" | grep -o '<bo
done
```

```
/                home blog
/about/          wp-singular page-template-default page page-id-12
/category/news/  archive category category-news category-2
/?s=dune         search search-results
/nothing-here/   error404
```

home and blog for the latest posts, page plus the page's ID, archive plus the category, search, error404: the same answers the conditional tags give (is_home(), is_page(), is_category(), is_search(), is_404()), printed for the stylesheet. index.php drew three of these five pages, singular.php and 404.php the others.

GUIDED LAB

- `cp -r labs/module-06/l4_1-loop-template-tags/shelf_theme_starter wordpress/wp-content/themes/pagesco-lab-shelf`, then `wp theme activate pagesco-lab-shelf`.
- `cp labs/module-06/l4_1-loop-template-tags/test_shelf_theme.sh wordpress/`, then `docker compose run --rm wpcli sh test_shelf_theme.sh`. Open the theme's index.php in VS Code.
- TODOs 1 to 5, the test after each.

QUESTION	YOUR ANSWER
Which line decides which posts each page lists?	
Why esc_html() around get_the_date() but not around the_title()?	
Why does /author/devon/ say Archive?	

- Done when the test shows `Shelf: News`, three dated items and `Nothing on this shelf.` for zzz. Then switch back (`wp theme activate pagesco-classic`, or pagesco) and remove the folder and the test.
- Finished early? `template_tags_starter.php`, then `debug_loop_report.php`.

PREDICT, THEN RUN

Write your prediction first, then run it on your site. The hint says which site state it was run on. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (Devon's author archive in pagesco-classic: which heading?)

```
curl -s http://localhost:8080/author/devon/ | grep -o '<h1>.*</h1>' | sed 's/<
```

P2

Predict the output, then run it. (WP-CLI and the main query.)

```
wp eval 'var_dump( did_action( "wp" ), have_posts() );'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Finish pagesco-classic (lesson page). In class: Step 1. At home: Steps 2 to 4.

- ☐ Step 1: singular.php; /book-club-dune/ shows meta, Filed under and the post navigation; /about/ only title and content
- ☐ Step 2: 404.php; /nothing-here/ answers 404 with a search form
- ☐ Step 3: theme.json; `wp_theme_has_theme_json()` is true (hybrid)
- ☐ Step 4: the per-URL loop; the pagesco-plan.txt line

SELF-CHECK

Q1

A classic theme has only index.php, singular.php and 404.php. Which file draws /author/maria/?

A) `singular.php`

B) `index.php`, with a Loop over Maria's posts

C) `author.php`, which WordPress generates

D) `404.php`, because there is no author template

Q2

On the course site, what does this print, and why?

```
wp eval 'var_dump( is_home(), get_the_title() );'
```

Q3

Inside the Loop you need the post's title in a link's `title` attribute and as the link text inside an h2. Which pair is right?

A) `title="<?php the_title(); ?>"` and `echo '<h2>' . the_title() . '</h2>';`

B) `title="<?php echo get_the_title(); ?>"` and `the_title( '<h2>', '</h2>' );`

C) `title="<?php echo esc_html( the_title() ); ?>"` and `the_title();`

D) `title="<?php the_title_attribute(); ?>"` and `the_title( '<h2>', '</h2>' );`

Q4

`get_template_part( 'template-parts/content.php' );` prints nothing, and there is no message. Why?

A) It looked for template-parts/content.php.php, found nothing and returned false

B) Template parts only work inside the Loop

C) Template parts must be in the theme's root folder

D) It found the file but needs a third argument with the post

Q5

A plugin must do something only on single posts and pages. Which is the EARLIEST of these hooks on which `is_singular()` gives the right answer?

A) `plugins_loaded`

B) `init`

C) `template_redirect`

D) `wp_footer`

Q6

singular.php shows only one page. Why does it still use while (have_posts()) { the_post(); ... }?

JOURNAL

You have now built the same site twice, as pagesco (blocks) and pagesco-classic (PHP). Pick one page, the search results for example, and describe how each theme produces it: which file, which "loop", where the heading comes from. Which version would you rather hand to a shop owner, and which would you rather debug at midnight? Why?

4.2 Custom Queries

BY THE END YOU CAN

- Tell the main query from secondary queries, and pick WP_Query, get_posts() or pre_get_posts for a task
- Write WP_Query arguments for post types, categories, authors, dates, searches, ordering and limits, and read found_posts and max_num_pages
- Run a secondary Loop on the same page as the main one and restore the current post with wp_reset_postdata()
- Change the main query with a pre_get_posts callback guarded by is_main_query() and is_admin()
- Paginate lists with the_posts_pagination(), and explain why paging a custom query in a template leads to a 404

KEY IDEAS

- **One main query per request; every other list is a secondary query** you create: `new WP_Query( $args )` runs at once.
- **Loop over your object** (`$q->have_posts()` , `$q->the_post()`) and finish with `wp_reset_postdata()` . `get_posts()` returns an array and needs no reset.
- **post_count** (this page), **found_posts** (total), **max_num_pages**; `no_found_rows` skips the count for lists you never paginate.
- **pre_get_posts** changes the main query before it runs. Guard it: `is_admin()` , `$query->is_main_query()` , the query's own conditionals. It is an action: `$query->set()` .
- **the_posts_pagination()** paginates the main query; paging a custom query in a template ends in 404s.
- **Argument pairs** (cat / category_name, author / author_name, p / name) fail silently with the wrong kind of value. query_posts() is for old tutorials only.

WORKED EXAMPLE: THREE QUERIES ON ONE FRONT PAGE

At the end of Lesson 4.2 the front page runs two queries, and the template-parts box a third on single posts. List the headings and links of page 1 and page 2:

```
curl -s http://localhost:8080/ | grep -o '<h[123][^>]*>.*</h[123]>\|<li><a [^>]*>[
echo ---
curl -s http://localhost:8080/page/2/ | grep -o '<h[123][^>]*>.*</h[123]>' | sed '
```

```
Good books, good company
Visit us
About Us
Events
Contact
News from the shop
Hello world!
Book club: Dune
Staff picks for the season
Posts pagination
---
News from the shop
Welcome to the new Pages &#038; Co. website
Posts pagination
```

Visit us comes from the secondary WP_Query (three pages by slug, in that order); the news from the MAIN query, three per page because of pre_get_posts, which is also why /page/2/ exists and shows only the news. Each list knows nothing about the other.

GUIDED LAB

- `cp labs/module-06/l4_2-custom-queries/query_shelf_starter.php wordpress/`, then `wp eval-file --use-include query_shelf_starter.php`.
- Fill in one argument array at a time (the lesson's argument table), run after each.

QUESTION	YOUR ANSWER
Which argument takes a login and which an ID?	
Why found_posts 3 on the last line, although one post was loaded?	
Which line would no_found_rows change, and how?	

- Done when all five lines match "Expected when done". Then `rm wordpress/query_shelf_starter.php`.
- Finished early? The archive-rules must-use plugin with `test_archive_rules.sh`, then `debug_related_box.php`.

PREDICT, THEN RUN

Write your prediction first, then run it on your site. The hint says which site state it was run on. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (Two per page, three News posts.)

```
wp eval '$q = new WP_Query( array( "category_name" => "news", "posts_per_page"
```

P4

Predict the output, then run it. (After the project: page 2 of the home page and of the News archive.)

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/page/2/
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/category/news/p
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Front Page with Several Loops (lesson page). In class: Step 1. At home: Steps 2 and 3.

- ☐ Step 1: front-page.php; / shows Visit us and three news items, /page/2/ only the news
- ☐ Step 2: the corrected related-posts.php (wp_reset_postdata()); the post navigation is right again
- ☐ Step 3: the check; the pagesco-plan.txt line

SELF-CHECK

Q7

On the site at the end of Lesson 4.1: what does this print, and why the second number?

```
wp eval '$q = new WP_Query( array( "post_type" => "page", "posts_per_page" =>
```

Q8

A pre_get_posts callback reads `if ( is_home() ) { $query->set( 'posts_per_page', 3 ); }` and nothing else. What goes wrong?

A) Nothing: is_home() makes sure only the blog home changes

B) Every other WP_Query that runs on the blog home (a box, a widget, a block) is also cut to three posts

C) It is a filter, so it must return $query

D) The admin's Posts screen breaks, but the front end is fine

Q9

A query with `'posts_per_page' => 3` matches 7 posts. On page 3, what are post_count, found_posts and max_num_pages?

A) 1, 7, 3

B) 3, 7, 3

C) 1, 3, 7

D) 3, 3, 3

Q10

In which case does your code need NO `wp_reset_postdata()`?

A) After a WP_Query Loop at the end of a template part

B) After a secondary Loop inside the main Loop of singular.php

C) After a secondary Loop in front-page.php, before the main Loop

D) After `get_posts()`, when each WP_Post is passed to the functions you call (`get_the_title( $post )`)

Q11

An old template starts with `query_posts( 'cat=2&posts_per_page=5' );`. How do you translate it?

A) Keep it, and call wp_reset_query() at the end

B) Replace it with get_posts() and the same string

C) If it changes the page's main list: pre_get_posts on the main query (category_name, posts_per_page); if it adds a list: new WP_Query plus wp_reset_postdata()

D) Move it into functions.php so it runs earlier

Q12

Module 5 brings a Book post type. The shop wants (a) a "three newest books" box under every post and (b) 12 books per page on /books/. Which tool for each?

JOURNAL

Think of the Pages & Co. shop once the Book post type exists (Module 5). Write down three lists the website will need, for example "new arrivals" or "all fantasy books under $15". For each, decide: main query changed with pre_get_posts, a secondary WP_Query, or get_posts()? Which arguments would you pass?

EXIT TICKET

Three questions, no notes.

E1

In one sentence: what does the Loop do, and what does the_post() do inside it?

E2

Name a template tag pair and say when you use each half.

E3

The blog home should show three posts per page. Where does that change go, and which two guards must it have?

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 4.1: Common Mistakes 2 to 4; the project Steps 2 to 4
- ☐ Lesson 4.2: query-args.php, Common Mistakes 2 and 4; the project Steps 2 and 3
- ☐ Lesson 4.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 4.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); sticky_posts back to []
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Labs not reached: `template_tags_starter.php`, the archive-rules lab, the two debug handouts
- ☐ Bring to Session 7: the site at the end of Lesson 4.2 (pagesco-classic active), the project folder with `labs/`, pagesco-plan.txt, my journal, this workbook

GLOSSARY

TERM	MEANING
main query	The one WP_Query WordPress runs from the URL, in `$wp_query`; templates and conditional tags work from it
the Loop	`while ( have_posts() ) { the_post(); ... }`: walks the main query's posts
template tag	A function about the current post: `the_` prints, `get_` returns
template part	A piece of a classic theme loaded with `get_template_part()`, `get_header()` or `get_footer()`
conditional tag	A yes/no function about the request (`is_home()`, `is_singular()` ...), reliable after the main query
hybrid theme	A classic theme with a theme.json
secondary query	A `new WP_Query( $args )` your code creates
`wp_reset_postdata()`	Sets the global `$post` back to the main query's current post
`pre_get_posts`	The action that lets you change a query before it runs
`found_posts` / `max_num_pages`	The total number of matches / pages of the current size

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 7: functions.php: Supports, Assets, Menus & Widgets

Your workbook for Session 7: the warm-up, the key ideas of Lesson 4.3, a worked example, the guided lab, predict-then-run practice, the Part 1 project's first milestones and your template map, the self-check and your exit ticket.

Lesson 4.3 + Part 1 project | Session 7 · 3 hours | Name: _____

SESSION GOALS

- Explain when functions.php runs and put each job in a prefixed callback on the right hook.
- Declare theme supports, and say which ones block themes get automatically.
- Enqueue a stylesheet and a deferred script with dependencies and a version that busts caches.
- Register a menu location and a widget area, print them, and fill them from WP-CLI; add the accessibility and translation basics.
- Start the Part 1 project, a block theme for the Fernhill Community Garden: Milestone 1 done, Milestone 2 started, Milestone 3 planned.

START HERE: WARM-UP AND READINESS

WARM-UP (SESSION 6)

Answer W1 to W4 from memory, then run W5's two commands.

W1

What does the Loop do, and what does the_post() do inside it?

W2

the_title() or get_the_title() inside echo '<h2>' . … . '</h2>'?

W3

posts_per_page 2, three matches: post_count, found_posts, max_num_pages on page 1?

W4

Name the three guards of a pre_get_posts callback that changes the blog home.

W5

Run the two commands on the slide. What should a site ready for today print?

```
wp theme list --status=active --field=name
curl -s http://localhost:8080/ | grep -c 'class="entry-title"'
```

LESSON

4.3 functions.php: Supports, Assets, Menus & Widgets

BY THE END YOU CAN

- Explain when functions.php runs and choose the right hook for theme setup, widgets and assets
- Declare theme supports such as title-tag and html5 with add_theme_support(), and say which ones block themes get automatically
- Enqueue a stylesheet and a deferred script with wp_enqueue_style() and wp_enqueue_script(), with dependencies and a version that busts browser caches
- Register a menu location and a widget area, print them with wp_nav_menu() and dynamic_sidebar(), and fill them from WP-CLI
- Add the accessibility and translation basics every theme needs: a skip link, screen-reader text, labeled navigation and strings WordPress can translate

KEY IDEAS

- **functions.php runs on every request** while the theme is active (admin, REST and WP-CLI included): code goes in prefixed callbacks on the right hook.
- **Supports** on `after_setup_theme`: `title-tag`, `automatic-feed-links`, `post-thumbnails`, `responsive-embeds`, `html5` with an array. Block themes get most of them, and their title, automatically.
- **Enqueue** on `wp_enqueue_scripts`: a prefixed handle, `get_parent_theme_file_uri()`, dependencies, the theme's Version; scripts with `'strategy' => 'defer', 'in_footer' => true`.
- **Menus:** `register_nav_menus()` (locations), the owner assigns, `wp_nav_menu()` behind `has_nav_menu()`. **Widget areas:** `register_sidebar()` with a fixed id on `widgets_init`, `is_active_sidebar()` + `dynamic_sidebar()`; widgets are blocks since 5.8.
- **Accessibility:** a skip link, `.screen-reader-text` CSS, labeled navigation, real buttons, visible focus. **Translation:** a literal text domain; 6.8+ loads a theme's languages/ itself.

WORKED EXAMPLE: WHAT A CLASSIC THEME LEAVES IN THE DATABASE

pagesco-classic is inactive at the end of Lesson 4.3, but what you set up for it is still in the database, in the theme's own option:

```
wp option get theme_mods_pagesco-classic --format=json
```

```
{"nav_menu_locations":{"primary":4},"custom_css_post_id":-1,"sidebars_widgets":{"
```

The menu location (`primary` → menu 4) and the widget area's contents (`footer` → block-7) are theme mods of pagesco-classic. The menu itself is a term:

```
wp menu list --fields=term_id,name,slug,count
```

```
+---------+-----------+-----------+-------+
| term_id | name      | slug      | count |
+---------+-----------+-----------+-------+
| 4       | Main menu | main-menu | 3     |
+---------+-----------+-----------+-------+
```

Switch to pagesco-classic again and both come straight back; pagesco's header uses its Navigation block instead, and a block theme has no widget areas.

GUIDED LAB

- `cp -r labs/module-07/l4_3-functions-php/corner_theme_starter wordpress/wp-content/themes/pagesco-lab-corner`, then `wp theme activate pagesco-lab-corner`.
- `cp labs/module-07/l4_3-functions-php/test_corner_theme.sh wordpress/`, then `docker compose run --rm wpcli sh test_corner_theme.sh`.
- Four TODOs in functions.php, one callback each, the test after each.

JOB	HOOK	FUNCTION YOU CALLED
title and html5		
style and script		
menu location		
widget area		

- Done when the test shows one `<title>`, html5, `style.css?ver=2.0.0`, `corner.js?ver=2.0.0, defer`, the location and the area. Then `wp theme activate pagesco-classic` and remove the folder and the test.

PREDICT, THEN RUN

Write your prediction first, then run it.

P1

Predict the output, then run it. (pagesco-classic at the end of Lesson 4.2, before today's functions.php.)

```
wp eval 'var_dump( current_theme_supports( "title-tag" ), current_theme_suppor
```

P2

Predict the output, then run it. (pagesco at the end of Lesson 4.3: a block theme that declares no supports.)

```
wp eval 'var_dump( current_theme_supports( "title-tag" ), current_theme_suppor
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Finish the Classic Theme, Then Back to pagesco (lesson page). In class: Steps 1 and 3. At home: Step 2.

- ☐ Step 1: style.css at 1.1.0; the stylesheet URL ends in ?ver=1.1.0
- ☐ Step 2: the per-URL check (one title, the CSS, the current menu item, the widget)
- ☐ Step 3: pagesco/functions.php with the pattern category; the three pattern headers; **pagesco active**

SELF-CHECK

Q1

Which of these actions fires FIRST in a front-end request?

A) `wp_enqueue_scripts`

B) `widgets_init`

C) `after_setup_theme`

D) `wp_head`

Q2

At the end of Lesson 4.3, what does this print, and why does the version column matter for the theme's CSS?

```
wp theme list --name=pagesco-classic --fields=name,status,version
```

Q3

pagesco-classic enqueued its stylesheet with `get_stylesheet_uri()` instead of `get_parent_theme_file_uri( 'style.css' )`. A client builds a child theme of it. What happens?

A) The page loads the CHILD's style.css under the parent's handle, and the parent's CSS is gone

B) Nothing changes: both functions return the parent's file

C) WordPress loads both files automatically

D) A notice: get_stylesheet_uri() cannot be used in a parent theme

Q4

Why does header.php wrap `wp_nav_menu()` in `has_nav_menu( 'primary' )`?

A) It makes the menu load faster

B) So a location without an assigned menu prints nothing instead of wp_page_menu()'s list of every page

C) wp_nav_menu() refuses to run without it

D) It registers the location

Q5

pagesco-classic ships languages/es_ES.l10n.php and says Requires at least: 6.6. Does it still need load_theme_textdomain()?

A) Yes, every theme needs it on every version

B) No, never: translations always load themselves

C) Only plugins need it

D) Not on WordPress 6.8 and later, which load the active theme's languages/ folder itself; it stays for 6.6 and 6.7

Q6

A must-use plugin calls wp_enqueue_style() at the top level of its file. What does WordPress say, and what is the real damage?

JOURNAL

Make a two-column table, pagesco and pagesco-classic, with one row per job from this lesson: the page title, the CSS, the menu, the footer content, the skip link, screen-reader text, translations. For each cell, write who does the job (core, theme.json, a template, functions.php, the site owner in an admin screen). Which theme puts more of the work on WordPress, and which gives a developer more control?

PART 1 PROJECT: THE FERNHILL COMMUNITY GARDEN THEME

Your brief is `labs/module-08/p1-project/BRIEF.md` (requirements, milestones, rules, hand-in, rubric). The client, Odette Marsh, wrote:

“Our website runs on a free theme from 2019 that nobody dares to touch. We would like our own theme, made for us: green and calm, our news on the front page three at a time, a big welcome at the top, a call for Saturday volunteers, and an autumn look we can switch on in October. Our volunteers write news and edit pages in the editor; none of us writes code. And please: nothing that breaks when WordPress updates itself.”

REQ	WHAT	FROM
R1	Design system	Lesson 3.2
R2	Templates and parts	Lessons 3.1, 3.2
R3	Patterns and the front page	Lessons 3.3, 1.3
R4	Style variation	Lesson 3.3
R5	functions.php	Lessons 4.3, 2.3
R6	The main query	Lesson 4.2
R7	404 page	Lesson 3.1
R8	Quality	Lessons 1.3, 3.2

MILESTONE 1 (IN CLASS)

```
cp -r labs/module-08/p1-project/fernhill_starter wordpress/wp-content/themes/fernhill
wp theme activate fernhill
cp labs/module-08/p1-project/test_fernhill.php wordpress/
wp eval-file --use-include test_fernhill.php
```

- ☐ `Checks passed: 1 of 12`
- ☐ I read every starter file and its TODOs
- ☐ Last thing today: `wp theme activate pagesco`

MILESTONE 2 (START IN CLASS, FINISH AT HOME)

- ☐ style.css header: Version 1.0.0, Requires at least 7.1, Requires PHP, Tested up to 7.1
- ☐ theme.json: the five role colors in order, defaultPalette off, 680px / 1100px
- ☐ fonts (a serif for text, a sans-serif for headings) and element styles (links, buttons in primary)
- ☐ my self-check after M2: Checks passed: ____ of 12

PLAN MILESTONE 3: MY TEMPLATE MAP

TEMPLATE / PART	WHICH URLS IT DRAWS	WHAT IT CONTAINS (LANDMARKS, BLOCKS)
index.html		
front-page.html		
single.html		
page.html		
404.html		
parts/header.html		
parts/footer.html		

SELF-CHECK: PROJECT SKILLS

Q7

Fernhill is a block theme. What is its style.css for?

- A) It names the theme: the header fields WordPress reads (Theme Name, Text Domain, Version, Requires at least ...)
- B) It is the theme's stylesheet, loaded on every page
- C) It is a fallback when theme.json has an error
- D) It registers the patterns

Q8

The brief wants links and buttons in the primary color. Where does that go?

- A) A hex color typed into each template's markup
- B) assets/css/fernhill.css
- C) `theme.json`: `styles.elements.link` and `styles.elements.button` with `var:preset|color|primary`
- D) A pattern that wraps every link

Q9

R6: the blog home lists three posts per page with working pagination. Which way does the brief want?

A) A Query Loop with `perPage` 3 that does not inherit

B) A `pre_get_posts` callback in functions.php with the three guards: not the admin, the main query only, the blog home only

C) Settings > Reading set to 3

D) A second WP_Query in the front-page pattern

Q10

You tried a header change in the Site Editor and clicked Save. Now your edits to parts/header.html no longer show. What happened, and what do you do?

A) The theme cache needs 30 minutes

B) parts/ files are only read on activation: switch themes twice

C) theme.json overrides parts

D) A wp_template_part copy in the database wins over the file: move the change into header.html, then delete the copy

Q11

Odette wants the autumn look she can switch on in October (R4). Which file, and what may it change?

Q12

You stop working on Fernhill for the day. Which command do you run, and why?

EXIT TICKET

E1

Name the hook for each: a theme support, a stylesheet, a widget area.

E2

Why does the enqueue use the theme's Version?

E3

Which Fernhill requirement will take you longest, and which lesson will you reread for it?

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 4.3: Common Mistakes 2 to 4, the project's Step 2, Exercises 1 to 3, the Quick Quiz, the journal
- ☐ Part 1 project: Milestones 2 and 3 (about 1 hour 45 minutes), the self-check after each work session
- ☐ After each work session: `wp theme activate pagesco`; no Site Editor copies left

- ☐ Labs not reached: `menus_widgets_starter.sh`, `debug_theme_setup.php`
- ☐ Bring to Session 8: pagesco active, fernhill with M2 and M3, my last self-check output, my template map, this workbook

GLOSSARY

TERM	MEANING
`after_setup_theme`	The action for theme supports and menu locations, right after functions.php is loaded
theme support	A feature a classic theme switches on with `add_theme_support()` (title-tag, html5 ...)
enqueue	Putting a stylesheet or script on WordPress's list with a handle, dependencies and a version
handle	The unique, prefixed name of an enqueued file (`pagesco-classic-style`; the element id adds -css or -js)
menu location	A named place in a template where the owner's menu appears (`register_nav_menus()`)
widget area	A slot for widgets (`register_sidebar()` with an id), printed with `dynamic_sidebar()`
skip link	"Skip to content": the first focusable link, jumping past the header to `<main>`
`.screen-reader-text`	The class for text meant to be heard, not seen; classic themes must style it
Fernhill	The Part 1 project: a block theme for the Fernhill Community Garden (a fictional client)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 8: Part 1 Project and Assessment

Your workbook for Session 8: the Part 1 map, the review, your project studio checklist, how the Part 1 assessment runs, a self-check on all of Part 1, the retrospective journal and the exit ticket.

Modules 1–4 | Session 8 · 3 hours | Name: _____

SESSION GOALS

- See Part 1 as one map: what each module taught and where it shows up in your project and the assessment.
- Move your Fernhill theme forward by at least one milestone, and know exactly what is left.
- Finish a client's classic theme in one hour, alone: the Part 1 assessment.
- Name what you would trust yourself with on a client's site, and what you want to review before Part 2.

START HERE: PART 1 ON ONE PAGE

MODULE	WHAT YOU CAN DO NOW	PROJECT (R) · ASSESSMENT (T)	LESSONS
1	WordPress as a PHP application; the local site; PHP the WordPress way (escaping, i18n, prefixes, standards)	R8, every task; T4	1.1 to 1.3
2	The content model, the database, hooks (actions, filters, priority)	R5, R6; T1 to T3	2.1 to 2.3
3	Block themes: anatomy and the hierarchy, theme.json, templates and parts, patterns, variations, child themes	R1 to R4, R7	3.1 to 3.4
4	Classic themes: the Loop and template tags, custom queries and pre_get_posts, functions.php	R5, R6; T1 to T5	4.1 to 4.3

Before the session: run your project's self-check and write its last line here: _____

WARM-UP

W1

Name the five extension points of WordPress and the module that covers each.

W2

Block theme or classic theme: where does each keep its templates, and in which language?

W3

Which three guards does a pre_get_posts callback for the blog home need?

W4

Name four escaping functions and where each goes.

W5

Your project's last self-check line?

PART 1 REVIEW

WHAT WORDPRESS THINKS EACH ADDRESS IS

On the projector: `labs/module-08/p1-review/request_map_solution.sh`. Predict the request type of each address before it runs. Your turn, at home or early: `request_map_starter.sh` in the same folder (two TODOs).

ADDRESS	MY PREDICTION (BODY CLASSES)	TEMPLATE IN FERNHILL	TEMPLATE IN PAGESCO-CLASSIC
`/`			
`/book-club-dune/`			
`/about/`			
`/category/news/`			
`/no-such-path/`			

PREDICT, THEN RUN

Write your prediction first, then run it on your site (pagesco active).

P1

Predict the output, then run it. (A child theme's parent, from its header.)

```
wp eval 'echo wp_get_theme( "pagesco-tt5-child" )->get( "Template" ), "\n";'
```

P2

Predict the output, then run it. (Page 2 of the blog home, on pagesco (ten posts per page).)

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/page/2/
```

P3

Predict the output, then run it. (A WP_Query of pages, by title.)

```
wp eval '$q = new WP_Query( array( "post_type" => "page", "orderby" => "title", "or
echo implode( ", ", wp_list_pluck( $q->posts, "post_title" ) ), "\n";'
```

P4

Predict the output, then run it. (Escaping a string you wrote yourself.)

```
wp eval 'echo esc_html( "<b>Fernhill</b> & friends" ), "\n";'
```

PART 1 PROJECT STUDIO: FERNHILL

WHERE I AM

	MILESTONE	DONE?	SELF-CHECK AFTER
M1	Set up	☐	
M2	The design system	☐	
M3	Templates and parts	☐	
M4	Patterns and the front page	☐	
M5	functions.php	☐	
M6	The Autumn variation, NOTES.md, the per-URL check and the self-check (`Checks passed`	☐	

My next step (from the conference): _____

THE PER-URL CHECK (LESSON 3.2), FOR FERNHILL

From the project folder, in the WP-CLI container shell (`docker compose run --rm wpcli sh`), with fernhill active. Every line should show one header, one main and one footer; `/no-such-path/` answers 404.

```
for url in / /page/2/ /hello-world/ /about/ /category/news/ "/?s=garden" /no-such-path/
    page=$(curl -s -w '\n%{http_code}' "http://localhost:8080$url")
    echo "$url  $(echo "$page" | tail -n 1)  header:$(echo "$page" | grep -c '<header c
done
```

BEFORE YOU HAND IN (ONE WEEK FROM TODAY)

- ☐ `Checks passed: 12 of 12` (or a line in NOTES.md about what is not)
- ☐ the per-URL check: every page has its landmarks, the 404 answers 404
- ☐ `debug.log` quiet while I click through every page
- ☐ no Site Editor copies: `wp post list --post_type=wp_template,wp_template_part,wp_global_styles` shows none for fernhill
- ☐ NOTES.md: what is unfinished, one decision and why, the self-check's last lines
- ☐ `fernhill-<myname>.zip` of the theme folder; then `wp theme activate pagesco`

PART 1 ASSESSMENT: HOW IT RUNS

- **60 minutes, alone, open book** (lesson pages, notes, this workbook, your own course code). No other people, no AI assistants, no code from the web.
- **A new client's classic theme to finish:** five tasks, three of them from the client's bug reports. Your instructor hands out the files at 1:25; the BRIEF lists the tasks and the rubric.
- **A self-check** like the project's: run it as often as you like. It is a help, not your score.
- **Hand-in at 2:35:** a zip of the theme folder with the self-check's output in it. Unfinished? One honest line at the top of functions.php.

Notes during the assessment (what I would do with more time):

SELF-CHECK: PART 1

Q1

The site shows the latest posts on its front page. The active block theme has `templates/front-page.html`, `templates/home.html` and `templates/index.html`. Which template draws `/`?

A) `home.html`, because the front page lists posts

B) `index.html`, the fallback for everything

C) `front-page.html`

D) `page.html`, because the front page is a page

Q2

What does this print?

```
wp eval 'add_filter( "the_title", function ( $t ) { return "A" . $t; }, 20 );
add_filter( "the_title", function ( $t ) { return "B" . $t; } );
echo apply_filters( "the_title", "x" ), "\n";'
```

Q3

Which file must a block theme have, besides `style.css`, for WordPress to accept it?

A) `functions.php`

B) `templates/index.html`

C) `theme.json`

D) `index.php`

Q4

pagesco-tt5-child is a child theme of Twenty Twenty-Five. What does the middle command print?

```
wp theme activate pagesco-tt5-child
wp eval 'echo get_template(), " ", get_stylesheet(), "\n";'
wp theme activate pagesco
```

Q5

A learner changed Fernhill's footer in the Site Editor yesterday. Today every edit to parts/footer.html is ignored. Why, and what fixes it?

A) The database copy of the part (a wp_template_part post) wins over the file: copy what you want into the file, then reset or delete the copy

B) OPcache: restart the containers

C) The footer area in theme.json is wrong: change it to "footer-1"

D) Template parts are cached for 30 minutes: wait

Q6

This callback forgets the main-query guard. What does the last line print on the course site (four published posts)?

```
wp eval 'add_action( "pre_get_posts", function ( $q ) {
    if ( is_admin() ) { return; }
    if ( $q->is_home() ) { $q->set( "posts_per_page", 3 ); }
} );
$q = new WP_Query( array( "posts_per_page" => 5 ) );
echo $q->post_count, "\n";'
```

A) 4

B) 5

C) 0

D) 3

Q7

What does this print? (Three posts are in News.)

```
wp eval '$q = new WP_Query( array( "category_name" => "news", "posts_per_page" => 2
echo $q->post_count, " ", $q->found_posts, "\n";'
```

A) 2 2

B) 2 3

C) 3 3

D) 3 2

Q8

A pattern prints a link to an address that comes from `home_url()`. Which function goes around it inside `href="..."`?

A) `esc_html()`

B) `esc_attr()`

C) `esc_url()`

D) `wp_kses_post()`

Q9

What does this print? (Post 17 is "Book club: Dune".)

```
wp eval 'global $post; $post = get_post( 17 ); setup_postdata( $post );
echo "<h2>" . the_title() . "</h2>\n";'
```

Q10

Fernhill must load `assets/css/fernhill.css` on the front end. On which hook does `wp_enqueue_style()` belong?

A) `init`

B) `after_setup_theme`

C) `wp_head`

D) `wp_enqueue_scripts`

Q11

A theme has `styles/autumn.json` with a title and a palette, and no `blockTypes`. What is it?

A) A style variation the site owner can switch on for the whole site

B) A section style for Group blocks

C) A child theme of the theme

D) A pattern

Q12

pagesco's theme.json has the palette color oxblood, #7a2e2e. What does this print?

```
wp eval 'echo wp_get_global_stylesheet( array( "variables" ) ), "\n";' \
    | grep -o -- '--wp--preset--color--oxblood: [^;]*'
```

JOURNAL: PART 1 RETROSPECTIVE

Look back at your first journal entry (Lesson 1.1: why you want to learn WordPress development, what you hope to build). Which part of that can you build now, with what you learned in Part 1? Then: one skill you would trust yourself with on a client's site tomorrow, one you would look up first, and one question you want Part 2 to answer.

EXIT TICKET

Three questions, no notes.

E1

A client wants "news three at a time" on a block theme's front page. Where does the code go, and why not a second query in the template?

E2

Why does every string in a pattern go through `esc_html_e()` even though you wrote it?

E3

One skill from Part 1 you will use first, and one you want to review.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the Fernhill theme; hand in `fernhill-<myname>.zip` with NOTES.md, one week from today
- ☐ Journal: the Part 1 retrospective
- ☐ Self-check Q1 to Q12

- ☐ Part 2 (if it follows): `wp theme activate pagesco`; Node 24 or `docker pull node:24`; `docker pull composer:2`
- ☐ Optional: `request_map_starter.sh` and `debug_query_review.php` in `labs/module-08/p1-review/`

Part 2: Plugins to Launch

Sessions 9 to 18. Modules 5 to 9: the pagesco-books plugin (custom post types, security, settings, custom tables), custom blocks with the Interactivity API, the REST API and custom endpoints, external APIs and cron, debugging, testing, performance, internationalization and deployment, ending with the final project: the Pages & Co. website, launched.

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 9: Your First Plugin and Custom Post Types

Your workbook for Session 9, the first session of Part 2: the Part 2 check, the starting note and a Part 1 refresher, the key ideas of Lessons 5.1 and 5.2, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 5.1–5.2 | Session 9 · 3 hours | Name: _____

SESSION GOALS

- Check that your computer and your site are ready for Part 2: `check_environment.sh --part2` ends with READY.
- Build a plugin that stays tidy at fifteen features: a header WordPress checks, a namespace, Composer's autoloader, one class per feature, a main object that starts them on plugins_loaded.
- Use activation, deactivation and uninstall for the right jobs, and recognize the first-plugin failures.
- Teach WordPress about books: a post type with an archive and REST, a genre taxonomy, typed public meta, and the rewrite rules that make the URLs work.
- Seed the Pages & Co. catalog, eleven books, with a WP-CLI script.

START HERE: PART 2

PART 2 ENVIRONMENT CHECK

- ☐ In the project folder, with the site running: `bash labs/module-01/l1_0-orientation/check_environment.sh --part2`
- ☐ The last line says `READY for PHP and WordPress Development` (or `READY WITH NOTES`). My Node line says: _____
- ☐ The site line says OK: `pagesco` is active, `pagesco-classic` is installed, `mu-plugins/pagesco-site.php` exists (the site as Lesson 4.3 left it)
- ☐ If something said FIX, my plan (and who helps me at the break):

No Node 24 on this computer is a NOTE, not a problem, until Session 11: the node:24 Docker image runs the same commands (Lesson 6.1).

STARTING WITH PART 2? READ THIS FIRST

Part 2 continues the Pages & Co. site that Part 1 built. If you join here, three things must be true before Lesson 5.1 makes sense:

1. **The course project runs on your computer** (Lesson 1.2: Docker, `~/pagesco-dev`, the `wp` shortcut, debug.log, Mailpit). The kit's setup guide walks you through it; the environment check above proves it.
2. **Your site is the Part 2 starting site**, the site as Lesson 4.3 leaves it: the `pagesco` block theme active, `mu-plugins/pagesco-site.php` with the store notice and its three hooks (Lesson 2.3), the users maria (editor) and devon (author) and the pages About Us, Contact and Events (Lesson 2.1). The course download `pagesco-part2-start.zip` installs it in about a minute: its README.txt has the three steps (copy the folder into `wordpress/`, run `docker compose run --rm wpcli sh pagesco-part2-start/install.sh`, log in as admin / password).
3. **You can read WordPress PHP**: prefixed functions, arrays of arguments, escaping on output (Lesson 1.3), and hooks: `add_action()`, `add_filter()`, priorities and custom hooks (Lesson 2.3). Skim Lessons 1.3, 2.1, 2.3 and 4.2 this week: the Part 1 refresher below is what Part 2 uses most.

Learners coming from Part 1: check your site against point 2 and help a neighbor who is new.

PART 1 REFRESHER: FIVE QUESTIONS

One minute alone, then compare with a neighbor. Nobody collects this.

R1

The store notice of Lesson 2.3 lives in which file, and which of its three hooks is a filter?

R2

Load, query, template: in which of the three do plugins load, and which action tells them WordPress is ready to register things?

R3

What makes `pagesco` a block theme, as `wp_is_block_theme()` sees it?

R4

Why does a `pre_get_posts` callback check `is_admin()` and `$query->is_main_query()` first?

R5

A feature should survive the next redesign. Theme or plugin?

LESSONS

5.1 Your First Plugin

BY THE END YOU CAN

- Write a plugin header WordPress understands, including the version requirements it checks on activation
- Organize a plugin as a short main file plus namespaced classes in src/, loaded by Composer's PSR-4 autoloader
- Start features from one main object on plugins_loaded, and add a new feature by writing one class and listing it
- Use activation and deactivation hooks and uninstall.php for the right jobs, and say what each must never do
- Recognize the typical first-plugin failures: direct file access, a missing autoloader, a class the autoloader cannot find, a hook registered too late

KEY IDEAS

- **A plugin** is a folder in `wp-content/plugins/` with a main file of the same name whose header has at least `Plugin Name:`. WordPress reads the header without running the file; `Requires at least` and `Requires PHP` are checked on activation.
- **Guard** every file that can run on its own: `defined( 'ABSPATH' ) || exit;` (uninstall.php: `WP_UNINSTALL_PLUGIN`).
- **Namespace + PSR-4:** classes live in `PagesCo\Books`; Composer's autoloader loads `PagesCo\Books\X\Y` from `src/X/Y.php`, letter for letter. `composer dump-autoload` only when composer.json's autoload changes.
- **The main file stays short:** header, guard, three constants, the autoloader, the life-cycle hooks, and `boot()` on `plugins_loaded`.
- **A feature** is a class implementing `Feature`; its `register()` adds its hooks. New feature = one class + one line in `Plugin::feature_classes()`.
- **Life cycle:** activation sets up once (roles, defaults); deactivation pauses and keeps data; uninstall.php removes the plugin's data. Register both hooks at the top level of the main file.

WORKED EXAMPLE: WHAT WORDPRESS KNOWS ABOUT YOUR PLUGIN

After Lesson 5.1, ask WordPress what it knows about your plugin, three ways. The list of plugins (the must-use plugins too):

```
wp plugin list --fields=name,status,version
```

```
+---------------+----------+---------+
| name          | status   | version |
+---------------+----------+---------+
| akismet       | inactive | 5.7.2   |
| hello         | inactive | 1.7.2   |
| pagesco-books | active   | 1.0.0   |
| mailpit-smtp  | must-use |         |
| pagesco-site  | must-use | 1.1.0   |
+---------------+----------+---------+
```

Your constants, defined by the main file on every request:

```
wp eval 'echo PAGESCO_BOOKS_VERSION, "\n", PAGESCO_BOOKS_DIR, "\n";'
```

```
1.0.0
/var/www/html/wp-content/plugins/pagesco-books/
```

And the option that makes a plugin "active" (Lesson 2.2): the main file's path relative to wp-content/plugins/. That pair, folder and main file of the same name, is how WordPress identifies pagesco-books:

```
wp option get active_plugins
```

```
array (
  0 => 'pagesco-books/pagesco-books.php',
)
```

Deactivating removes the line from this option; nothing else changes until uninstall.

GUIDED LAB

- From the project folder: `cp -r labs/module-09/l5_1-first-plugin/gift_wrap_starter wordpress/wp-content/plugins/pagesco-gift-wrap`, then `wp plugin activate pagesco-gift-wrap`.
- `cp labs/module-09/l5_1-first-plugin/test_gift_wrap.sh wordpress/`, then `docker compose run --rm wpcli sh test_gift_wrap.sh`. Read every line: what is TODO, why is the direct request a 500?
- Do TODOs 1 to 5 in `wordpress/wp-content/plugins/pagesco-gift-wrap/`. After each change: deactivate and activate the plugin (so the activation hook runs), then the test.

QUESTION	YOUR ANSWER
Why is the direct request a 500 without the guard, and 200 with it?	
What would happen to the price if the activation hook were registered inside the filter callback?	
After the test's last command, where did the option go, and why not on deactivation?	

- Done when the test prints `Gift wrapping: $2.50.` in the notice and `after uninstall: (not stored)`. Then `rm -r wordpress/wp-content/plugins/pagesco-gift-wrap wordpress/test_gift_wrap.sh`.
- Finished early? `psr4_paths_starter.php`, then `debug_plugin_audit.php` in the same folder (`wp eval-file --use-include`).

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site: `wp ...` commands from the project folder, the others in the WP-CLI container shell (`docker compose run --rm wpcli sh`). A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (The plugin off for a moment, on the site after Lesson 5.1.)

```
wp plugin deactivate pagesco-books
curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>
wp plugin activate pagesco-books
```

P2

Predict the output, then run it. (Has the autoloader loaded StoreNotice yet?)

```
wp eval 'var_dump( class_exists( "PagesCo\Books\StoreNotice", false ), class_e
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The First Feature Moves In (lesson page, "Project: The First Feature Moves In"). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ The skeleton from the demo: header, guard, constants, composer.json + vendor/, Feature, Plugin, Activator, uninstall.php; `wp plugin list` says active
- ☐ Step 1: src/StoreNotice.php, listed in feature_classes(); the notice shows everything twice
- ☐ Step 2: pagesco-site.php 1.1.0; the notice once; with the plugin off, only the free-shipping text
- ☐ Step 3: the missing class, on purpose, read and fixed (200 again)
- ☐ Step 4: the final check; the rule "new feature = one class + one line" in pagesco-plan.txt

SELF-CHECK

Q1

A client's server runs PHP 8.1. pagesco-books says `Requires PHP: 8.2`. What happens when the administrator clicks Activate?

A) The plugin activates, and the first page that uses PHP 8.2 syntax dies with a fatal error

B) WordPress refuses to activate it and says which PHP version the plugin requires

C) The plugin activates with the features that need PHP 8.2 switched off

D) Composer's platform_check.php deletes the vendor folder

Q2

On the course site after Lesson 5.1, what does this print?

```
wp eval 'echo PAGESCO_BOOKS_DIR, "\n";'
```

Q3

A learner creates `src/Admin/BookColumns.php` but writes `namespace PagesCo\Books;` at the top of it, and lists `Admin\BookColumns::class` in feature_classes(). What happens on the next request?

A) Nothing: PSR-4 only looks at the file name

B) Composer must be run again before the class is found

C) A fatal error: Class `PagesCo\Books\Admin\BookColumns` not found

D) WordPress shows a "called incorrectly" notice and skips the feature

Q4

A site owner deactivates pagesco-books for five minutes to find a conflict. Which of these must the deactivation callback NOT do?

A) Delete the Bookseller role and the plugin's options

B) Clear the plugin's scheduled events

C) Flush the rewrite rules

D) Nothing at all (an empty callback)

Q5

On the course site after Lesson 5.1, what does the second command print, and why?

```
wp plugin uninstall pagesco-books --deactivate --skip-delete > /dev/null
wp eval 'var_dump( get_role( "pagesco_bookseller" ) );'
```

Q6

Why must register_activation_hook() be called at the top level of the main file, not inside a plugins_loaded callback?

A) Because hooks registered on plugins_loaded run twice

B) Because the activation hook needs the autoloader, which loads later

C) Because WordPress only reads the main file's header at the top level

D) Because on the activation request plugins_loaded has already fired before WordPress includes the new plugin's file

JOURNAL

Open pagesco-plan.txt from Lesson 1.1 and read the features you marked content, theme or plugin before you knew any of this. Which would you mark differently now, and why? Then write the next three features you expect pagesco-books to get, each as a class name in PagesCo\Books with one sentence about the hooks it will use.

5.2 Custom Post Types, Taxonomies & Meta

BY THE END YOU CAN

- Register a custom post type with register_post_type() on init, with labels, an archive and REST support
- Register a custom taxonomy with register_taxonomy() and attach it to the post type
- Register typed post meta with register_post_meta(), and explain why the keys are public and show_in_rest
- Add a meta box and extra admin columns so editors can see and change a book's details
- Flush rewrite rules at the right moment, and seed content safely with a WP-CLI script

KEY IDEAS

- **A custom post type** is a new value of `wp_posts.post_type`: `register_post_type( 'book', ... )` on init, with labels, `has_archive`, a rewrite slug and `show_in_rest`.
- **A taxonomy** groups content: `register_taxonomy( 'genre', array( 'book' ), ... )`; the second argument is the type it attaches to.
- **Meta** is a fact about one item: `register_post_meta()` gives a key a type, a label and REST visibility. Shop keys have no leading underscore so they stay public and bindable.
- **Rewrite rules** live in the database: flush them in the activator (after registering the type and taxonomy, because init has not run on that request), never on init.
- **Values are stored as strings:** `get_post_meta()` returns `'412'` and `''` for false; REST and block bindings cast them to the registered type.
- **Seed with a script** that is safe to run twice, and set terms with `wp_set_object_terms()`, not `tax_input`, when no user is logged in.

WORKED EXAMPLE: TWO WAYS TO ASK FOR THE FANTASY SHELF

After the seeding, the same question asked two ways. WP-CLI accepts a taxonomy as a filter (`--genre=<slug>`) on the book type:

```
wp post list --post_type=book --genre=fantasy --fields=ID,post_title --orderby=ID
```

```
+----+-------------------+
| ID | post_title        |
+----+-------------------+
| 35 | The Hobbit        |
| 38 | Lord of the Rings |
+----+-------------------+
```

The REST API filters by term ID (Fantasy is 8) and returns typed meta (a number, not a string):

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?genres=8&_fields=id,title.rende
```

```
[{"id":38,"title":{"rendered":"Lord of the Rings"},"meta":{"pagesco_price":15.99}
```

Same two books. The REST list is ordered newest first by default (the seeding script gave every book a date, one minute apart), the WP-CLI list by ID because you asked. Module 7 builds the shop's own endpoints on top of this.

GUIDED LAB

- From the project folder: `cp -r labs/module-09/l5_2-custom-post-types/book_quotes_starter wordpress/wp-content/plugins/pagesco-quotes`, `wp plugin activate pagesco-quotes`, copy `test_book_quotes.sh` into wordpress/ and run it with `docker compose run --rm wpcli sh test_book_quotes.sh`.
- TODOs 1 to 4 in `pagesco-quotes.php`. After each change: deactivate and activate the plugin, then the test.
- Finished early? `debug_meeting_type.php`. After the project (it needs the books): `shelf_report_starter.php`.

- Done when the archive answers 200 and REST returns the quote with `"pagesco_quote_source":"Pride and Prejudice"`. Then `wp plugin deactivate pagesco-quotes` and `rm -r wordpress/wp-content/plugins/pagesco-quotes wordpress/test_book_quotes.sh`.

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site: `wp ...` commands from the project folder, the others in the WP-CLI container shell (`docker compose run --rm wpcli sh`). A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (A genre archive, after the seeding.)

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/genre/fantasy/
```

P4

Predict the output, then run it. (A registered integer, read with get_post_meta().)

```
wp eval 'var_dump( get_post_meta( 30, "pagesco_pages", true ),
    get_post_meta( 30, "pagesco_pages" ) );'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Fill the Shelves (lesson page, "Project: Fill the Shelves"). In class: Steps 1, 2 and the start of 3. At home: the admin section, the rest of Step 3 and Step 4.

- ☐ BookPostType, GenreTaxonomy, BookMeta and the new Activator from the demo; /books/ answers 200
- ☐ Step 1: `wordpress/pagesco-seed/` with seed-books.php and books.json
- ☐ Step 2: `Success: 11 books in the catalog.`
- ☐ At home: BookMetaBox and BookColumns ("Seeing Books in the Admin"); six features listed
- ☐ Step 3: genres, the column check, Dune in REST; Step 4: the final check, then `rm -r pagesco-seed`

SELF-CHECK

Q7

BookPostType registers `'has_archive' => 'books'` and `'rewrite' => array( 'slug' => 'books' )`. Which address lists all books?

A) `/book/`

B) `/books/`

C) `/?post_type=books`

D) `/wp-json/wp/v2/book`

Q8

The Great Gatsby (book 33) is out of stock; pagesco_in_stock is registered as a boolean. What does this print on the finished Lesson 5.2 site?

```
wp eval 'var_export( get_post_meta( 33, "pagesco_in_stock", true ) ); echo "\r
```

Q9

The shop wants a page /publisher/penguin/ listing every Penguin book, and filters by publisher on the Books screen. How should the publisher be stored?

A) As a taxonomy on the book type

B) As a meta key, pagesco_publisher

C) As an option

D) In the post content of each book

Q10

"To be safe", a developer calls `flush_rewrite_rules()` on init. What is wrong with that?

A) Nothing: it is the recommended place

B) The rules are flushed before the post type is registered, so books 404

C) WordPress refuses to flush on init and logs a notice

D) It rewrites the whole rules option on every page load: a needless, slow database write

Q11

Genre 8 is Fantasy. What does this print on the finished Lesson 5.2 site?

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?genres=8&_fields=title.rend
```

Q12

A WP-CLI seeding script creates a book with wp_insert_post() and must give it two genres. Which reliably works?

A) `'tax_input' => array( 'genre' => $genres )` in the post array

B) A 'genres' key in the post array

C) `wp_set_object_terms( $id, $genres, 'genre' )` after the insert

D) `update_post_meta( $id, 'genre', $genres )`

JOURNAL

You now have three ways to store information about a book: the post itself (title, content), a taxonomy (genre) and meta (author, price). Pick one more thing a bookshop might track (a publisher, a star rating, a signed copy) and decide for each whether it is the post, a taxonomy or meta, with one sentence of reasoning. Then update your pagesco-plan.txt catalog notes with what the book type actually stores.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Name the three files every new pagesco-books feature touches, and what each gets.

E2

What does each moment do to the Bookseller role: activation, deactivation, uninstall?

E3

/books/ is a 404 right after you registered the book type. Why, and where does the permanent fix go?

NOTES

HOMEWORK CHECKLIST

- ☐ 5.1 project Steps 3 and 4; the rule in pagesco-plan.txt

- ☐ 5.2 "Seeing Books in the Admin" (meta box and columns) and project Steps 3 and 4: six features, eleven books, pagesco-seed/ removed
- ☐ Lesson 5.1 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 5.2 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Labs I did not reach: `psr4_paths_starter.php`, `debug_plugin_audit.php`, `debug_meeting_type.php`, `shelf_report_starter.php`
- ☐ Bring to Session 10: the site at the end of Lesson 5.2, two browsers or a private window, `labs/`, the plan, the journal, this workbook

GLOSSARY

TERM	MEANING
plugin header	The comment block at the top of the main file; `Plugin Name:` is required, `Requires at least` / `Requires PHP` are checked on activation
ABSPATH guard	`defined( 'ABSPATH' ) || exit;`: stops a file from running when requested directly
namespace	The prefix for classes: `PagesCo\Books\Plugin`
PSR-4 / autoloader	The rule (and Composer's code) that loads `PagesCo\Books\X` from `src/X.php` on first use
feature	A class implementing `Feature`; `register()` adds its hooks; listed in `Plugin::feature_classes()`
activation / deactivation hook	Callbacks run once when the plugin is switched on / off (`register_activation_hook( __FILE__, ... )`)
uninstall.php	Runs when the plugin is deleted; removes the plugin's own data
custom post type	A new kind of content in wp_posts (`book`)
taxonomy / term	A way of grouping content (`genre`) / one group in it (Fantasy, term 8)
post meta	Key/value facts about one post in wp_postmeta (`pagesco_price`)
rewrite rules	The stored table that turns pretty URLs into queries; rebuilt by a flush
seeding	Filling a site with known data from a script (`wp eval-file pagesco-seed/seed-books.php`)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 10: Security and the Settings API

Your workbook for Session 10: the warm-up, the key ideas of Lessons 5.3 and 5.4, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 5.3–5.4 | Session 10 · 3 hours | Name: _____

SESSION GOALS

- Apply WordPress's four security habits everywhere data crosses a boundary: sanitize what comes in, escape what goes out, check the nonce, check the capability; and $wpdb->prepare() for SQL by hand.
- Name the common vulnerabilities (XSS, CSRF, SQL injection, broken access control) and the defense against each.
- Harden the book meta box save and add a safe "Mark sold out" action.
- Store a plugin's settings as one option with defaults, and build its settings page with the Settings API.
- Trace what happens between Save Changes and the database, and let the store notice read the settings.

START HERE: WARM-UP

FIVE QUESTIONS ON SESSION 9

Two minutes alone, then compare with a neighbor.

W1

A new feature for pagesco-books: which three places does it touch?

W2

Why does `Activator::activate()` register the book type itself before it flushes?

W3

`get_post_meta( 33, 'pagesco_in_stock', true )` on The Great Gatsby returns what?

W4

Where did Lesson 5.2's meta box already use a nonce?

W5

Deactivating pagesco-books: what happens to the eleven books?

HOMEWORK CHECK

- ☐ `wp post list --post_type=book --format=count` prints 11
- ☐ `grep -c '::class,' wordpress/wp-content/plugins/pagesco-books/src/Plugin.php` prints 6
- ☐ The Book details box shows when I open a book in wp-admin (if not: copy BookMetaBox.php and BookColumns.php from Lesson 5.2 at the break)

LESSONS

5.3 Security in WordPress

BY THE END YOU CAN

- Sanitize input on the way in and escape output on the way out, choosing the right function for each context
- Protect every state change with a nonce and a current_user_can() capability check
- Write safe database queries with $wpdb->prepare() and explain how SQL injection works
- Name the common WordPress vulnerabilities (XSS, CSRF, SQL injection, broken access control) and the defense against each
- Harden a real save handler and an admin action in the pagesco-books plugin

KEY IDEAS

- **One data flow:** request → genuine? (**nonce**) → allowed? (**capability**) → **sanitize** → store → **escape** → page.
- **Escape late, for the context:** `esc_html()` text, `esc_attr()` attributes, `esc_url()` links, `wp_kses_post()` HTML that may keep safe tags; even for data you sanitized.
- **Sanitize and validate on the way in:** `sanitize_text_field()`, `absint()`, a cast; a value with a shape (a date) is validated and rejected if wrong.
- **Nonces** (`wp_nonce_field()`, `wp_nonce_url()`, `check_admin_referer()`) stop forged requests (CSRF); **capabilities** (`current_user_can( 'edit_post', $id )`) stop the wrong user. You need both.
- `$wpdb->prepare()` with `%s`, `%d`, `%f`, `%i`: no variable ever goes directly into SQL.
- **Database functions trust their caller:** `wp_insert_post()` and `update_post_meta()` check nothing; guard every write yourself. save_post handlers skip autosaves and revisions.

WORKED EXAMPLE: ONE VALUE, THREE DESTINATIONS

One value with tags, an ampersand and quotes, escaped for three destinations:

```
wp eval '$v = "Dune: <b>signed</b> & \"rare\"";
echo esc_html( $v ), "\n", esc_attr( $v ), "\n", wp_kses_post( $v ), "\n";'
```

```
Dune: &lt;b&gt;signed&lt;/b&gt; &amp; &quot;rare&quot;
Dune: &lt;b&gt;signed&lt;/b&gt; &amp; &quot;rare&quot;
Dune: <b>signed</b> &amp; "rare"
```

esc_html() and esc_attr() both turn the value into text (the quotes too, so it cannot end an attribute); wp_kses_post() keeps the tags a post may contain and escapes the rest. The function is chosen by the destination: text, attribute, or HTML that may keep safe tags. An unknown value printed into a page is never echoed raw.

GUIDED LAB

- From the project folder: `cp labs/module-10/l5_3-security/staff_note_starter.php wordpress/`, then `wp eval-file --use-include staff_note_starter.php`.
- Read the four lines it prints. Which request should a real site refuse, and why?
- TODOs 1 to 4 in `wordpress/staff_note_starter.php`, one at a time; run again after each.

REQUEST	WHICH BOX OF THE FLOW STOPS IT?
no token	
forged token	
real token, as Devon	
real token, as Maria, with a script tag	

- Done when the output matches "Expected when done". Then `rm wordpress/staff_note_starter.php`.
- Finished early? `prepared_search_starter.php`, then `debug_book_search.php` in the same folder.

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (A link that is not a link.)

```
wp eval 'echo esc_url( "javascript:alert(1)" ) === "" ? "(empty)" : "kept", "\
```

P2

Predict the output, then run it. (Tags, an ampersand and whitespace.)

```
wp eval 'echo sanitize_text_field( "  Pages <b>&</b> Co.\n\tBooks  " ), "|\n";
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Harden the Plugin (lesson page, "Project: Harden the Plugin"). In class: Steps 1 and 2. At home: Step 3.

- ☐ Step 1: the hardened BookMetaBox.php; `php -l` clean; the box still registered
- ☐ Step 2: StockAction.php listed; the editor sees the link, the author does not
- ☐ Step 3: the forged request answers 403 "The link you followed has expired."; Dune still in stock; a real click works

SELF-CHECK

Q1

A book title goes into a link's tooltip: `<a href="/books/dune/" title="...">`. Which function escapes it there?

A) `esc_html()`

B) `esc_url()`

C) `esc_attr()`

D) `sanitize_text_field()`

Q2

What does this print?

```
wp eval 'echo esc_html( "Pride & Prejudice <2nd ed.>" ), "\n";'
```

Q3

A form handler calls check_admin_referer() and it passes. What does that prove?

A) The user is allowed to make the change

B) The request came from a page this site served to this user, for this action, recently

C) The submitted values are safe to store

D) The request was not sent by a script

Q4

What does this print?

```
wp eval 'global $wpdb; echo $wpdb->prepare( "SELECT ID FROM {$wpdb->posts} WHE
```

Q5

This handler is reached from a link on the Books screen. Which defense is missing?

```
$book_id = absint( $_GET['book'] );
check_admin_referer( 'pagesco_books_feature_' . $book_id );
update_post_meta( $book_id, 'pagesco_featured', 1 );
```

A) `current_user_can( 'edit_post', $book_id )` before the update

B) A nonce check

C) `esc_html()` on the book ID

D) `$wpdb->prepare()` around the update

Q6

The meta box's "First published" field receives `soon`. What keeps it out of the database?

A) `sanitize_text_field()`: it removes words that are not dates

B) `esc_attr()` when the box prints the value

C) `absint()`, which turns it into 0

D) Validating it: DateTime::createFromFormat('Y-m-d', $v) must give back exactly $v, else store ''

JOURNAL

Pick one handler you have written in this course (the meta box save, the stock action, or the store notice from Lesson 2.3) and trace a single untrusted value through it: where does it enter, where is it checked (nonce, capability), where is it sanitized, and where is it escaped on the way out? Note any gap. Then write the one security habit you are most likely to forget, and how you will remember it.

5.4 Admin Pages, Settings API & Options

BY THE END YOU CAN

- Store a plugin's settings as one array option with defaults, and read it safely with get_option() and wp_parse_args()
- Add an admin page with add_options_page() (or a top-level menu) protected by the right capability
- Build a settings form with the Settings API: register_setting(), sections, fields, settings_fields() and do_settings_sections()
- Validate submitted values in a sanitize callback and report problems with add_settings_error()
- Trace what happens between Save Changes and the database: options.php, the nonce, the allowed options list
- Decide what uninstall.php removes, and remove the plugin's own settings

KEY IDEAS

- **One array option per plugin** (`pagesco_books_settings`), read through one class that knows the name, the defaults (`wp_parse_args()`) and the cleaning.
- `register_setting( $group, $option, $args )` on init: type, `sanitize_callback` , `default` (handed out while the row is missing).
- **Admin pages** hang off `admin_menu` : `add_options_page()` under Settings; the capability (`manage_options`) guards the menu item and the URL.
- **The Settings API** describes the form as data (`add_settings_section()` , `add_settings_field()` with `label_for`) and prints it (`settings_fields( $group )` , `do_settings_sections( $page )` , `submit_button()`).
- **options.php** checks the nonce, the capability and the allowed options list, then saves through your callback. The callback returns the COMPLETE value, treats a missing checkbox as off, and refuses bad values with `add_settings_error()` (keeping the old one).
- **uninstall.php** deletes the plugin's own data (the option, the role) and keeps the shop's content (the books).

WORKED EXAMPLE: DEFAULTS WITHOUT A ROW

The setting is registered with a default, and nothing has been saved since the lesson's uninstall test. Is `get_option()` handing out the defaults?

```
wp eval 'var_dump( get_option( "pagesco_books_settings" ) === PagesCo\Books\Settir
```

```
bool(true)
```

And is there a row?

```
wp db query "SELECT COUNT(*) AS rows_named FROM wp_options WHERE option_name = 'pa
```

```
rows_named
0
```

Exactly the defaults, and no row: register_setting()'s default applies while the row is missing. That is why the notice looks the same after a reactivation, and why Settings::get() still fills keys from defaults() for sites that saved an older array.

GUIDED LAB

- From the project folder: `cp labs/module-10/l5_4-settings-api/newsletter_settings_starter/pagesco-lab-newsletter.php wordpress/wp-content/mu-plugins/` and `cp labs/module-10/l5_4-settings-api/test_newsletter_settings.sh wordpress/`.
- `docker compose run --rm wpcli sh test_newsletter_settings.sh`, and open Settings > Newsletter in your browser.
- TODOs 1 to 3 in `wordpress/wp-content/mu-plugins/pagesco-lab-newsletter.php`; run the test after each.

- Done when the test prints `rows: 3`, `Frequency must be weekly or monthly.` and the cleaned option. Then `rm wordpress/test_newsletter_settings.sh wordpress/wp-content/mu-plugins/pagesco-lab-newsletter.php`.
- Finished early? `debug_newsletter_sanitize.php`; after the project, `settings_probe_starter.php`.

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (An option that does not exist, with a default.)

```
wp eval 'var_dump( get_option( "pagesco_no_such_option", array() ) );'
```

P4

Predict the output, then run it. (The sanitize callback, fed a string.)

```
wp eval 'echo wp_json_encode( sanitize_option( "pagesco_books_settings", "not
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Settings the Shop Can Change (lesson page, "Project: Settings the Shop Can Change"). In class: Steps 1 and 2. At home: Step 3.

- ☐ Settings and Admin\SettingsPage listed (nine features); the page has two sections and four fields; Maria gets a 403
- ☐ Step 1: StoreNotice reads the settings; the notice looks exactly as before
- ☐ Step 2: the notice changed from the admin (tags stripped, link off), then changed back
- ☐ Step 3: uninstall.php deletes the option; the uninstall test; `rm pagesco-save.sh`

SELF-CHECK

Q7

An administrator presses Save Changes on Settings > Pages & Co. Books. Which code checks the nonce and the capability?

A) Core's own handler, wp-admin/options.php, before it calls your sanitize callback

B) Your SettingsPage::render() method

C) Your Settings::sanitize() method

D) Nothing: the Settings API has no nonce

Q8

On the finished Lesson 5.4 site (the uninstall test deleted the row; nothing was saved since), what does this print?

```
wp option get pagesco_books_settings --format=json
```

Q9

The Events link checkbox is unticked when the form is saved. What does options.php receive for it?

A) `show_events_link=0`

B) `show_events_link=false`

C) Nothing: an unticked checkbox is not sent at all

D) `show_events_link=off`

Q10

On the finished Lesson 5.4 site, what does this print, and why is the notice sentence empty?

```
wp eval 'echo wp_json_encode( sanitize_option( "pagesco_books_settings",
    array( "books_per_page" => "0", "events_limit" => "7" ) ) ), "\n";'
```

Q11

Maria, an editor, opens options-general.php?page=pagesco-books and gets a 403. Which line decided that?

A) The current_user_can() check inside render()

B) The nonce in settings_fields()

C) register_setting()'s type 'object'

D) The manage_options capability passed to add_options_page()

Q12

Which value deserves its own option instead of a key in the pagesco_books_settings array?

A) The notice sentence

B) A counter of page views, updated on every request

C) Books per list

D) Whether the notice shows the events link

JOURNAL

List everything Pages & Co. now stores: content (posts, pages, books), terms, meta, options, a role. For each, write who owns it (the shop or the plugin) and what should happen to it when the plugin is deactivated and when it is deleted. Add the list to pagesco-plan.txt; you add the events table to it in Session 11. Then note one setting you would add for a real client, and which section of the page it would go in.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

Name the four checks a handler that changes data makes, in order.

E2

What does options.php do between Save Changes and the database?

E3

Your sanitize callback refuses a value. What should it store, and how does the user find out?

NOTES

HOMEWORK CHECKLIST

- ☐ 5.3 project Step 3 (the forged request, the checkpoint) and a real "Mark sold out" click
- ☐ 5.4 project Step 3 (uninstall.php, the uninstall test, `rm pagesco-save.sh`): nine features

- ☐ Lesson 5.3 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Lesson 5.4 Hands-on Exercises 1 to 3 (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Labs I did not reach: `prepared_search_starter.php`, `debug_book_search.php`, `debug_newsletter_sanitize.php`, `settings_probe_starter.php`
- ☐ For Session 11: Node 24 (`node --version` prints v24) or `docker pull node:24`

GLOSSARY

TERM	MEANING
XSS (cross-site scripting)	Someone else's HTML or script printed into your page; stopped by escaping on output
CSRF (cross-site request forgery)	A forged request riding a logged-in user's session; stopped by a nonce
SQL injection	A crafted value that rewrites your query; stopped by `$wpdb->prepare()`
broken access control	A user doing what their role should forbid; stopped by `current_user_can()`
nonce	A token tied to a user, an action and a few hours; proves a request is genuine
capability	A permission such as `edit_post` or `manage_options`, granted through roles
sanitize / validate	Clean a value / accept or reject it by its shape
escape	Make a value safe for where it is printed: text, attribute, URL, allowed HTML
option	A named row in wp_options; `pagesco_books_settings` holds the shop's settings as one array
Settings API	register_setting(), sections and fields, settings_fields(); saved by options.php
sanitize callback	Your function that cleans a setting before it is stored and returns the complete value
admin-post.php	Core's endpoint for your own form handlers and admin actions (`admin_post_<action>`)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 11: Custom Tables, Shortcodes and Block Setup

Your workbook for Session 11: the warm-up, the key ideas of Lessons 5.5 and 6.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 5.5–6.1 | Session 11 · 3 hours | Name: _____

SESSION GOALS

- Write shortcodes that return escaped HTML, with attributes handled by shortcode_atts().
- Decide when a custom table is the right storage, create and migrate one with dbDelta() and a schema version, and read and write it with $wpdb and placeholders.
- Set up the block toolchain: Node 24, @wordpress/create-block in the existing plugin, @wordpress/scripts.
- Build the Book Card, register it from the manifest with a Blocks feature, and say what each built file is for.

START HERE: WARM-UP

FIVE QUESTIONS ON SESSION 10

W1

A handler that changes data: the four checks, in order?

W2

`settings_fields()` takes the group or the page slug?

W3

What does an unticked checkbox send?

W4

`$wpdb->prepare()`: which placeholder for a table name?

W5

What does uninstall.php remove, and what does it keep?

HOMEWORK CHECK AND NODE

- ☐ `grep -c '::class,' wordpress/wp-content/plugins/pagesco-books/src/Plugin.php` prints 9
- ☐ `node --version` prints v24 (or I use the node:24 image)
- ☐ wordpress/wp-content/plugins/pagesco-books/package.json copied from Lesson 6.1, and `npm install` running in a second terminal in that folder

LESSONS

5.5 Shortcodes, $wpdb & Custom Tables

BY THE END YOU CAN

- Register shortcodes with add_shortcode() whose callbacks return escaped HTML, with attributes handled by shortcode_atts()
- Decide when a custom table is the right storage, and when post types, meta, taxonomies or options are
- Create and upgrade a table with dbDelta() and a schema version stored in an option
- Read and write rows with $wpdb: insert(), update(), delete(), get_row(), get_var(), get_results(), always through prepare() or format arrays
- Remove the table and its version option in uninstall.php

KEY IDEAS

- **A shortcode** is `add_shortcode( $tag, $callback )` on init; the callback RETURNS escaped HTML and never echoes.
- `shortcode_atts( $defaults, $atts, $tag )` fills defaults, drops undeclared attributes (all values are strings) and creates the `shortcode_atts_{$tag}` filter.
- **A custom table** is for many uniform rows queried by their fields; post types, taxonomies and options come with far more for free.
- `dbDelta()` creates a table or adds missing columns and indexes from a carefully formatted CREATE TABLE; it never renames or drops.
- **A schema version** in an option, compared on every request: raise it with every schema change, because updates do not rerun activation.
- **Every** `$wpdb` **value** through placeholders (`%s` , `%d` , `%f` , `%i`) or the format arrays of insert(), update(), delete().

WORKED EXAMPLE: ONE TABLE, TWO WAYS IN

After the project: the Café's events through the shortcode, then the same table asked in SQL:

```
wp eval 'echo do_shortcode( "[pagesco_events location=\"Café\"]" ), "\n";'
```

```
<ul class="pagesco-events"><li class="pagesco-event"><strong>An evening with Ana
```

```
wp db query "SELECT location, COUNT(*) AS events, SUM(capacity) AS seats FROM wp_p
```

```
location    events  seats
Café    2   70
Reading corner  2   40
Children's corner   1   15
Front of the shop   1   0
```

The shortcode reads through EventsTable::find() (a %s placeholder for the location, the setting's limit); the GROUP BY and SUM on real columns is where a table pays off.

GUIDED LAB

- `cp labs/module-11/l5_5-custom-tables/wishlist_table_starter.php wordpress/`, then `wp eval-file --use-include wishlist_table_starter.php`.
- TODOs 1 to 5 in order; run after each. Stuck half-way? `wp db query "DROP TABLE IF EXISTS wp_pagesco_lab_wishlist"`.

- Done when the output ends with `tables left: 0`. Finished early? The open_hours shortcode lab, then `debug_events_report.php`.

PREDICT, THEN RUN

Write your prediction first, then run it on your site.

P1

Predict the output, then run it. (Page 14 is the Events page, not a book.)

```
wp eval 'echo do_shortcode( "[pagesco_book id=\"14\"]" ) === "" ? "(empty)" :
```

P2

Predict the output, then run it. (The events table after the seeding.)

```
wp eval 'global $wpdb; echo $wpdb->get_var( "SELECT COUNT(*) FROM " . PagesCo\
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Events Calendar (lesson page). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ BookShortcode and EventsTable (version 1) listed; the table exists
- ☐ Step 1: version 2, capacity added on the next request
- ☐ Step 2: uninstall.php drops the table and both options
- ☐ Steps 3 and 4: EventsShortcode, six events, the Events page lists five

SELF-CHECK

Q1

What does this print?

```
wp eval 'echo wp_json_encode( shortcode_atts( array( "limit" => 5, "location"
    array( "limit" => "3", "color" => "red" ), "pagesco_events" ) ), "\n";'
```

Q2

Which of these do you get for free with a post type, but must build yourself for a custom table?

A) A REST route, admin screens, revisions and export

B) Fast queries on an indexed column

C) A schema you control column by column

D) JOINs with wp_posts

Q3

On the finished Lesson 5.5 site, what does this print?

```
wp eval 'global $wpdb; echo $wpdb->prepare( "SELECT title FROM %i WHERE locati
    PagesCo\Books\EventsTable::name(), "Café" ), "\n";'
```

Q4

EventsTable::add() calls $wpdb->insert(). Where does it find the new row's ID?

A) In the return value of insert()

B) `$wpdb->insert_id`

C) `$wpdb->last_query`

D) Only with a second query, SELECT LAST_INSERT_ID()

Q5

Why does EventsTable check the schema version on every request (maybe_upgrade() on init), and not only on activation?

A) Because dbDelta() must run on every request

B) Because the activation hook cannot create tables

C) Because a plugin update replaces the files but does not run the activation hook again

D) Because the version option is not autoloaded

Q6

On the finished Lesson 5.5 site, what does this print?

```
wp eval 'echo do_shortcode( "[pagesco_events location=\"Reading corner\" limit
```

JOURNAL

Add the events table to the storage list in pagesco-plan.txt, with its owner and what deactivation and deletion do to it. Then name, for a client site you know, one kind of data for each row of the lesson's "store it as" table, and write the CREATE TABLE columns for the custom-table one.

6.1 Block Development Setup

BY THE END YOU CAN

- Describe a block type as WordPress sees it: block.json metadata, an editor script with edit and save, styles, and (later) PHP rendering
- Scaffold a block into an existing plugin with npx @wordpress/create-block and --no-plugin, and read every generated file
- Build blocks with @wordpress/scripts, and say what each file in build/ is for, including index.asset.php and the blocks manifest
- Register all built blocks from a plugin feature with wp_register_block_types_from_metadata_collection()
- Recognize the setup mistakes that cost beginners an afternoon: a second plugin by accident, a missing build, and stale build files

KEY IDEAS

- **A block type** is block.json metadata plus an editor script with **edit** (the editing view) and **save** (the stored HTML), plus styles; a static block's front end needs no JavaScript.
- `npx @wordpress/create-block@latest <slug> --no-plugin --target-dir=src/blocks/<slug> --namespace=pagesco --textdomain=pagesco-books` adds a block to the existing plugin.
- `npm run build` (@wordpress/scripts) turns src/blocks into build/blocks: compiled scripts and styles, `index.asset.php` (dependencies + version) and, with --blocks-manifest, `blocks-manifest.php`.
- `wp_register_block_types_from_metadata_collection( $dir, $manifest )` on init registers every built block and its asset handles: the Blocks feature.
- **WordPress reads only build/**: rebuild after every change (or keep `npm start` running), and build wherever the plugin is installed from source.

WORKED EXAMPLE: THE SHELF, NOT THE KIT

After the project: is the build's block.json the same file as the source's?

```
cd wp-content/plugins/pagesco-books && diff src/blocks/book-card/block.json build/
```

> `differ`

Different: the build rewrites it (formatting, and file: paths relative to build/). And the saved text survives minification word for word:

```
grep -o '"Book Card: a staff pick is coming soon."' wp-content/plugins/pagesco-boo
```

> "Book Card: a staff pick is coming soon."

GUIDED LAB

- `cp -r labs/module-11/l6_1-block-setup/shelf_label_starter wordpress/wp-content/plugins/pagesco-shelf-label`, `wp plugin activate pagesco-shelf-label`, copy `test_shelf_label.sh` into wordpress/ and run it.
- TODO 1 in `pagesco-shelf-label.php`; with Node, TODO 2 (keywords, then `npm install` and `npm run build` in the lab plugin's folder).

QUESTION	YOUR ANSWER
Why does the page show the label before TODO 1?	
Why does it have no stylesheet then?	

- Done when the test prints `front end: 1 stylesheet(s)`. Then `wp plugin deactivate pagesco-shelf-label` and remove the folder and the test.

PREDICT, THEN RUN

Write your prediction first, then run it on your site.

P3

Predict the output, then run it. (The manifest, required as PHP.)

```
wp eval 'echo implode( ", ", array_keys( require WP_PLUGIN_DIR . "/pagesco-boo
```

P4

Predict the output, then run it. (The block's stylesheet on the home page.)

```
curl -s http://localhost:8080/ | grep -c 'pagesco-book-card-style'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Book Card's First Draft (lesson page). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ package.json, npm install, npm run build; the Blocks feature (thirteen features)
- ☐ Step 1: the shop's block.json, edit, save, styles; no view.js
- ☐ Step 2: the Book Card in the editor
- ☐ Steps 3 and 4: the front end and the final check

SELF-CHECK

Q7

Which file in build/blocks/book-card/ has no source file of its own in src/?

A) `block.json`

B) `index.js`

C) `style-index.css`

D) `index.asset.php`, the dependency list and version hash

Q8

On the finished Lesson 6.1 site, what does this print?

```
wp eval 'echo implode( ", ", WP_Block_Type_Registry::get_instance()
    ->get_registered( "pagesco/book-card" )->style_handles ), "\n";'
```

Q9

You worked all afternoon with npm start running. What do you run before you commit or deploy?

A) `npm run build`: the minified production files, without .map files

B) `npm install` again

C) `npx @wordpress/create-block` again

D) Nothing: npm start already wrote build/

Q10

Posts already contain the Book Card. A developer renames the block from pagesco/book-card to pagesco/staff-pick. What happens to those posts?

A) WordPress renames the block in every post automatically

B) Nothing visible: names are cosmetic

C) Their stored comments still say wp:pagesco/book-card, a block that no longer exists: the editor cannot handle them

D) The posts are deleted

Q11

On the finished Lesson 6.1 site, what does this print, and why not 4?

```
grep -c "'wp-" wp-content/plugins/pagesco-books/build/blocks/book-card/index.a
```

Q12

What does `npx @wordpress/create-block@latest book-card --no-plugin --target-dir=src/blocks/book-card` NOT create?

A) A block.json

B) A plugin PHP file, a package.json and node_modules/

C) An edit.js and a save.js

D) A view.js

◆ JOURNAL

Draw the path of the Book Card from src/blocks/book-card/ to a visitor's screen in your own boxes and arrows. Then write what you would check first if (a) the block is missing from the inserter, (b) the inserter shows an old version, (c) the front end looks unstyled.

EXIT TICKET

E1

Why must a shortcode callback return its HTML instead of echoing it?

E2

You add a column to schema(). What else must change, and why?

E3

Name the path of the Book Card from your files to the editor.

NOTES

HOMEWORK CHECKLIST

- ☐ 5.5 project Steps 3 and 4
- ☐ 6.1 project Steps 3 and 4
- ☐ Exercises 1 to 3 of both lessons (Exercise 4 is a stretch)
- ☐ Both Quick Quizzes and both journal entries
- ☐ Labs not reached: open_hours, debug_events_report, core_blocks_tour, debug_block_registration

GLOSSARY

TERM	MEANING
shortcode	A [tag] in content that a callback replaces with HTML on display
custom table	A table of your own, `{$wpdb->prefix}pagesco_events`, for uniform rows

TERM	MEANING
dbDelta()	Creates or extends a table from a CREATE TABLE statement; only adds
schema version	An option (`pagesco_books_db_version`) compared with the code's DB_VERSION
block.json	A block type's metadata: name, title, attributes, supports, files
edit / save	The editing view (React) / the function that returns the stored HTML
@wordpress/scripts	WordPress's build tool (webpack, Babel, Sass, dependency extraction)
index.asset.php	The built script's dependency handles and version hash
blocks manifest	`build/blocks/blocks-manifest.php` : every block.json as one PHP array

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 12: Static and Dynamic Blocks

Your workbook for Session 12: a warm-up, the key ideas of Lessons 6.2 and 6.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 6.2–6.3 | Session 12 · 3 hours | Name: _____

SESSION GOALS

- Say where a static block keeps each attribute (the saved HTML or the comment), and read both with PHP.
- Add editable text, sidebar controls and color, spacing and typography supports to a block, and know which changes to save() need a deprecation.
- Render a dynamic block on the server with render.php: attributes as input, a query, the wrapper attributes, escaping.
- Bind core blocks to a book's data with core/post-meta and with the shop's own source, pagesco/book.
- Start the staff pick on the Staff picks post and the book pages with live details.

WARM-UP: LESSONS 5.5 AND 6.1

Answer from memory first; the class goes through them together.

W1

A shortcode callback ends with `echo $html;` instead of `return $html;`. What goes wrong?

W2

Why does EventsTable::find() build its SQL with `$wpdb->prepare()` even though only the plugin calls it?

W3

Where does `npm run build` write, and which file does WordPress read to register the plugin's blocks?

W4

You add a second block folder in src/blocks/ and build. What PHP do you add to register it?

W5

You edited src/blocks/book-card/edit.js, reloaded the editor, and nothing changed. First question?

LESSONS

6.2 Static Blocks: Attributes, Controls & Supports

BY THE END YOU CAN

- Declare attributes in block.json and choose where each one is stored: in the saved HTML (source and selector) or in the block's comment
- Make text editable on the canvas with RichText and save it with RichText.Content
- Add settings to the block sidebar with InspectorControls and components from @wordpress/components
- Opt into color, spacing and typography block supports and read the classes and styles they save
- Explain block validation, and change save safely with a deprecated entry and migrate

KEY IDEAS

- **Attributes with a** `source` **and** `selector` **live in the saved HTML** (title, author, a link's href); attributes without one live in the **block comment**, and only when they differ from their default. `parse_blocks()` sees only the comment.
- **RichText** edits text on the canvas (`allowedFormats`), **RichText.Content** saves it; `setAttributes()` is the only way to change attributes.
- **InspectorControls** puts `@wordpress/components` controls into the sidebar's Settings tab; **BlockControls** into the toolbar. Both live in edit.
- **Block supports** (color, spacing, typography) add attributes, controls, classes and styles through `useBlockProps()` / `useBlockProps.save()` , with the theme's presets saved as slugs.
- **Validation:** the editor re-runs save() and compares it with the stored HTML. save() must be pure: no translations, dates or random values.
- **Changing save() for existing content needs a deprecated entry:** the old attributes, supports and save, newest first, with an optional `migrate()` .
- In PHP, a static block is **finished HTML**: read it with the HTML API, or store the value in the comment, or make the block dynamic.

WORKED EXAMPLE: THE SUPPORTS, AND THE STYLE ENGINE

Supports add attributes, controls, classes and styles without code of yours. Ask the registry which supports the finished card declares (the site after Lesson 6.2):

```
wp eval 'echo wp_json_encode( WP_Block_Type_Registry::get_instance()->get_register
```

```
{"html":false,"color":{"background":true,"text":true,"link":true},"spacing":{"mar
```

A padding chosen in the Styles tab is saved in the comment as `var:preset|spacing|medium` and printed as an inline style. The style engine does that translation; ask it directly:

```
wp eval 'echo wp_style_engine_get_styles( array( "spacing" => array( "padding" =>
```

```
padding-top:var(--wp--preset--spacing--medium);padding-bottom:var(--wp--preset--s
```

The value points at the theme's spacing preset (a CSS custom property), never at a number of pixels: change the preset in theme.json and every block that uses it follows, exactly like the color slugs.

GUIDED LAB

- From the project folder: `cp labs/module-12/l6_2-static-blocks/attribute_sources_starter.php wordpress/`, then `wp eval-file --use-include attribute_sources_starter.php`.
- TODO 1: the registered block type. TODOs 2 and 3: describe each attribute: in the HTML (with its source and selector) or in the comment (with its default). TODO 4: what parse_blocks() returns for the stored heading and image.
- Before TODO 4, write your prediction: is the image's src in the attrs? The heading's text?

QUESTION	YOUR ANSWER
Which source does core use that Lesson 6.2's table does not list?	
Why is the button's tagName in the comment?	
Who reads the image's src back, and how?	

- Done when your output matches "Expected when done". Then `rm wordpress/attribute_sources_starter.php`.
- Finished early? The folder lab `card_price_starter/` (README: install it as a must-use plugin, run `test_card_price.sh`), then `debug_card_report.php`.

PREDICT, THEN RUN

Write your prediction on the line first, then run it in the WP-CLI container shell (`docker compose run --rm wpcli sh`) or, for `wp` commands, from the project folder. P1 and P2 need the site after Lesson 6.2; P3

and P4 after Lesson 6.3.

P1

Predict the output, then run it. (A heading stored at its default level.)

```
wp eval 'echo wp_json_encode( parse_blocks( "<!-- wp:heading --><h2 class=\"wp
```

P2

Predict the output, then run it. (How core's paragraph declares its text.)

```
wp eval 'echo wp_json_encode( WP_Block_Type_Registry::get_instance()->get_regi
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Staff Pick on the Staff Picks Post (lesson page, "Project: A Staff Pick on the Staff Picks Post"). In class: Step 1, if your card builds with the badge and the deprecation. At home: Steps 2 and 3.

- ☐ Step 1: the Hobbit card after the list: title, author, recommendation ("everyone" in italics), link, H3, badge on, Sage
- ☐ The Code editor shows the card's comment with only `{"backgroundColor":"sage"}`; I can say why
- ☐ Step 2: the front end shows the card between the list and the quote; the title links to the Hobbit's page
- ☐ Step 3: comparison with Lesson 1.3's helper; Card test deleted; parse_blocks lists four blocks; the plan line

SELF-CHECK

Q1

A block attribute is declared `"url": { "type": "string", "source": "attribute", "selector": "a", "attribute": "href" }`. Where is the URL stored once the post is saved?

A) In the block comment, as JSON

B) In post meta, under the key url

C) In the saved HTML: the `href` of the first `<a>` inside the block

D) Nowhere: the editor asks for it again

Q2

On the course site after Lesson 6.2: what does this print?

```
wp eval 'echo wp_json_encode( parse_blocks( "<!-- wp:pagesco/book-card {\"show
```

Q3

Cards are already on the site. Which change to the Book Card needs a new deprecated entry?

A) A new rule in style.scss that rounds the corners more

B) A new sidebar control in edit.js only

C) Wrapping the author in a `<cite>` in save()

D) A new attribute whose default makes save() print nothing new

Q4

A designer wants the card's heading level in the toolbar that floats above the selected block, like core's Heading block. Where does the control go?

A) Inside `<BlockControls>` in edit

B) Inside `<InspectorControls>` in save

C) Inside `<InspectorControls>` in edit

D) In block.json under supports

Q5

An editor picks the theme's Sage background for a card. What does the post store?

A) The hex color in an inline style

B) The class `has-sage-background-color` in the HTML and `"backgroundColor":"sage"` in the comment

C) A row in wp_options

D) Nothing: the theme decides at view time

Q6

Why does the course write the badge text `Staff pick` as a plain string in save() instead of `__( 'Staff pick', 'pagesco-books' )`?

JOURNAL

List every piece of data in the Book Card and, for each, write where it is stored (HTML or comment), who can change it (the editor on the canvas, the sidebar, the theme) and what happens to it when the plugin is updated. Then pick one content element a client of yours would want as a block (a team member, a testimonial, an opening-hours box) and sketch its block.json attributes with a source for each, and the supports you would turn on.

6.3 Dynamic Blocks & Block Bindings

BY THE END YOU CAN

- Explain when a block should be dynamic, and what a dynamic block stores in post_content
- Render a block on the server with render.php: attributes, a WP_Query through get_posts(), get_block_wrapper_attributes(), and escaping
- Preview a dynamic block in the editor with ServerSideRender, and fill its controls from the REST API through @wordpress/core-data
- Offer ready-made versions of a block as variations, generated in PHP
- Bind core blocks to post meta with core/post-meta, and write a custom binding source with register_block_bindings_source()

KEY IDEAS

- **A dynamic block stores only its comment.** `"render": "file:./render.php"` makes WordPress run the PHP on every view, with `$attributes`, `$content` and `$block`; the build copies render.php into build/.
- **In render.php treat attributes as input:** check optional ones with `isset()` (WordPress fills in defaults only for attributes that have one), cast, clamp, sanitize; query with `get_posts()`; start the wrapper with `get_block_wrapper_attributes()`; escape what you print.
- **An attribute without a default** can mean "use the site's setting": the Book List's count follows Books per list.
- **ServerSideRender** previews render.php through the block-renderer REST route; `@wordpress/core-data` fills controls with the site's data.
- **Variations** are presets of a block; `"variations": "file:./variations.php"` builds them in PHP, only when something asks.
- **Block bindings** connect core blocks' attributes to sources: `core/post-meta` returns raw values of registered, public meta; `register_block_bindings_source()` with `uses_context` gives formatted ones. They fail silently to the fallback.

WORKED EXAMPLE: A DYNAMIC BLOCK RENDERED FROM PHP

A dynamic block can be rendered anywhere PHP runs, with any attributes: do_blocks() finds the comment and runs render.php, as a page view does (the site after Lesson 6.3):

```
wp eval 'echo do_blocks( "<!-- wp:pagesco/book-list {\"genre\":\"dystopian\",\"cou
```

```
<ul class="wp-block-pagesco-book-list">
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco-boo
            <li class="wp-block-pagesco-book-list__item"><span class="pagesco-boo
    </ul>
```

Two of the three dystopian books, in title order, each printed by BookShortcode::render(). The variations come from variations.php, built from the genre terms:

```
wp eval 'foreach ( WP_Block_Type_Registry::get_instance()->get_registered( "pagesc
```

```
Classic Books Dystopian Books Fantasy Books Fiction Books Science Fiction Books
```

One variation per genre, generated in PHP when something asks for them. Add a genre on Books > Genres and the list grows without a build.

GUIDED LAB

- From the project folder: `cp -r labs/module-12/l6_3-dynamic-blocks/next_events_starter wordpress/wp-content/plugins/pagesco-next-events`, then `wp plugin activate pagesco-next-events`.
- `cp labs/module-12/l6_3-dynamic-blocks/test_next_events.php wordpress/`, then `wp eval-file --use-include test_next_events.php`: four blocks, four TODO paragraphs.
- Edit wordpress/wp-content/plugins/pagesco-next-events/block/render.php: TODO 1 the count (no default!), TODO 2 the events, TODO 3 the empty case, TODO 4 the list. Run the test after each one.

- Done when the four blocks print five events, two Café events, the empty sentence and `count 99: 6 events`.
- Then `wp plugin deactivate pagesco-next-events` and `rm -r wordpress/wp-content/plugins/pagesco-next-events wordpress/test_next_events.php`.
- Finished early? `core_sources_starter.php`, then `debug_genre_counts.php` (same folder).

PREDICT, THEN RUN

Write your prediction on the line first, then run it in the WP-CLI container shell (`docker compose run --rm wpcli sh`) or, for `wp` commands, from the project folder. P1 and P2 need the site after Lesson 6.2; P3

and P4 after Lesson 6.3.

P3

Predict the output, then run it. (A Book List that asks for 30 books.)

```
wp eval 'echo substr_count( do_blocks( "<!-- wp:pagesco/book-list {\"count\":3
```

P4

Predict the output, then run it. (What get_post_meta() returns for a sold-out book's stock.)

```
wp eval 'echo get_post_meta( 37, "pagesco_in_stock", true ) === "" ? "empty st
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Book Pages with Live Details (lesson page, "Project: Book Pages with Live Details"). In class: Steps 1 and 2. At home: Step 3, the editor section and the variations section.

- ☐ Step 1: templates/single-book.html with core/post-meta (author) and pagesco/book (price, stock, genres, pages, year) and New Arrivals
- ☐ Step 2: Dune shows $14.99, In stock, two genre links, 412 pages, 1965; Gatsby shows Sold out
- ☐ Step 3: Dune on sale shows $12.99 on its page and in the List test page, then $14.99 again; List test deleted; 14 features
- ☐ The List settings panel and the variations (New Arrivals, one per genre) in my editor

SELF-CHECK

Q7

The Book List already sits on twenty pages. You change render.php so every item gets a second line. What else must you do?

A) Add a deprecated entry to the block

B) Open and update each of the twenty pages

C) Run `npm run build` (render.php is copied into build/), then nothing: every page follows on its next view

D) Bump the block's version so caches notice

Q8

On the course site after Lesson 6.3 (six Classic books): what does this print?

```
wp eval 'echo substr_count( do_blocks( "<!-- wp:pagesco/book-list {\"genre\":\
```

Q9

In render.php the wrapper is `<ul <?php echo get_block_wrapper_attributes(); ?>>`, with no escaping function around it. Why is that safe?

A) It is not: wrap it in esc_html()

B) Block attributes are always numbers

C) Only administrators can add blocks

D) get_block_wrapper_attributes() returns attributes that are already escaped (class and style from the supports)

Q10

A list item is bound to `core/post-meta` with the key `pagesco_in_stock`. What does it show on The Great Gatsby's page (sold out)?

A) Sold out

B) The fallback text of the list item

C) false

D) Nothing: an empty list item

Q11

You register a binding source without `uses_context`. What does a bound paragraph in single-book.html show?

A) Its fallback text, with no error

B) A PHP warning about the missing context

C) The value for the first book in the catalog

D) The value for the post being shown, as usual

Q12

On the course site after Lesson 6.3: what does this print (book 40 is Animal Farm, published 1945-08-17)?

```php
<?php
$parsed = parse_blocks( '<!-- wp:paragraph {"metadata":{"bindings":{"content":
$block  = new WP_Block( $parsed, array( 'postId' => 40, 'postType' => 'book' )

echo wp_strip_all_tags( $block->render() ), "\n";
```

JOURNAL

Go through the Pages & Co. site page by page and sort every piece of book data you see into three columns: typed into content (Book Card, the hard-coded staff picks pattern), rendered by PHP (shortcodes, Book List), or bound (book pages). For each item in the first column, decide whether it should move, and to which of the other two. Then write down one place on a client's site where a binding would replace a custom block.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit before the checkpoint.

E1

A static block's title and heading level: where is each stored?

E2

Name two things a dynamic block's render.php must do with its attributes.

E3

A bound paragraph shows its fallback text on a book page. Name two causes.

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the staff pick (Steps 2 and 3) and the book pages (Step 3, the editor section, the variations section and build)
- ☐ Lesson 6.2 Hands-on Exercises 1 and 2 (Exercise 3 is a stretch)
- ☐ Lesson 6.3 Hands-on Exercises 1 and 2 (Exercise 3 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Before the Next Lesson: the card valid, Card test and List test gone, a final `npm run build`; JavaScript getters and preventDefault() refreshed
- ☐ Checkpoint 2 next session (Modules 5 and 6): reread Lessons 5.2 to 5.5, 6.2 and 6.3
- ☐ Optional: `card_price_starter/`, `core_sources_starter.php`, both debug handouts
- ☐ Bring to Session 13: a site at the end of Lesson 6.3, a working build, this workbook

GLOSSARY

TERM	MEANING
attribute	A piece of a block's data, declared in block.json with a type (and a source, a default)
source / selector	Where an attribute is read back from the saved HTML (`html`, `rich-text`, `text`, `attribute`) and which element
block comment	`<!-- wp:name {...} -->`: the block's name and its source-less, non-default attributes as JSON
RichText	The editable text component; `RichText.Content` prints its value in save
InspectorControls / BlockControls	Render controls into the sidebar's Settings tab / the block toolbar
block supports	Core features a block opts into in block.json (color, spacing, typography, align...)
validation	The editor's check that save() still produces the stored HTML

TERM	MEANING
deprecation	An entry in `deprecated` describing an older attributes + supports + save, with optional `migrate`
dynamic block	A block whose HTML render.php (or a render_callback) makes on every view; it stores only its comment
`get_block_wrapper_attributes()`	The wrapper's escaped class and style attributes in PHP (supports included)
ServerSideRender	An editor component that previews a block with the server's render.php
variation	A named preset of a block's attributes, listed in the inserter
block binding / source	A block attribute replaced on every view by a source's value (`core/post-meta`, `pagesco/book`)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 13: The Interactivity API and Using the REST API

Your workbook for Session 13: a warm-up, the key ideas of Lessons 6.4 and 7.1, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, Checkpoint 2 and your exit ticket.

Lessons 6.4-7.1 | Session 13 · 3 hours | Name: _____

SESSION GOALS

- Explain the Interactivity API's split: the server renders HTML with directives (and evaluates them), a store in a script module takes over.
- Design an interactive feature as state, derived state and context, and write the PHP half with wp_interactivity_state() and wp_interactivity_data_wp_context().
- Read the catalog through the REST API: routes, _fields, filters, pages and their headers.
- Authenticate the right way (cookie plus nonce, application passwords) and read 400, 401, 403 and 404.
- Show what you can do in Modules 5 and 6 in Checkpoint 2.

WARM-UP: LESSONS 6.2 AND 6.3

Answer from memory first; the class goes through them together.

W1

The Hobbit card on the Staff picks post: which of its attributes does PHP's parse_blocks() see?

W2

You changed the card's save(). When do you need a deprecated entry, and when not?

W3

The Book List stores only `<!-- wp:pagesco/book-list /-->`. Who decides how many books it shows?

W4

Which line makes WordPress pass the book's ID to a paragraph bound to pagesco/book?

W5

What does the Hobbit's page show as its price, and where does the value come from?

LESSONS

6.4 The Interactivity API

BY THE END YOU CAN

- Explain how the Interactivity API splits the work: server-rendered HTML with directives, then hydration by a store in a script module
- Scaffold an interactive block into the plugin, and build its viewScriptModule with --experimental-modules
- Set state and per-element context in PHP with wp_interactivity_state() and wp_interactivity_data_wp_context(), including derived state the server can compute
- Write a store with store(), state getters, getContext() and actions, and connect it with data-wp-on, bind, class and text
- Build interactive features that stay accessible and keep working without JavaScript

KEY IDEAS

- **Server first:** render.php prints the finished HTML with `data-wp-*` directives; WordPress **evaluates bind, class, style, text, context and each on the server**, so the page is right before any JavaScript.
- `"interactivity": true` and a **viewScriptModule**, built with `--experimental-modules` (silently skipped without it); the module loads only where the block is.
- **State** is shared by the store (`wp_interactivity_state()`), **context** belongs to one element (`wp_interactivity_data_wp_context()`), **derived state** is computed: PHP closures for the server, getters in view.js for the browser, with the same answers.
- **The store:** `store( namespace, { state, actions } )`, `getContext()`; actions change state and every directive follows. Actions that use the event synchronously need `withSyncEvent()`.
- **Progressive enhancement:** real links, state from the address, validated input: the feature works without JavaScript and for assistive technology (aria-current, aria-live, hidden).

WORKED EXAMPLE: WHAT THE SERVER ALREADY DECIDED

The server evaluates the Book Filter's directives for any address, so curl (no JavaScript at all) can check every filter. How many books does the server hide for Classic and for Fantasy (the site after Lesson 6.4)?

```
for genre in classic fantasy; do curl -s "http://localhost:8080/find-a-book/?genre
```

```
5
9
```

Eleven books in the HTML each time: five hidden for Classic (six classics), nine for Fantasy (two fantasy books). And the links: which one is current?

```
curl -s 'http://localhost:8080/find-a-book/?genre=fantasy' | grep -o 'aria-current
```

```
      5 aria-current="false"
      1 aria-current="page"
      1 aria-current="true"
```

One link says aria-current="true" (Fantasy), the other five "false": aria- attributes get the words, so a screen reader hears which filter is active before any JavaScript runs.

GUIDED LAB

- From the project folder: `cp labs/module-13/l6_4-interactivity-api/open_today_starter.php wordpress/`, then `wp eval-file --use-include open_today_starter.php`.
- TODOs 1 and 2: the two derived-state closures (isToday, isHidden). TODOs 3 and 4: the directives on the paragraph and on each day.
- Before each run, predict: which <li> gets hidden? aria-current? the class?

QUESTION	YOUR ANSWER
Which directive in the HTML did the server leave untouched, and why?	
Why does Friday say aria-current="false" instead of nothing?	
What would view.js need for the button to work?	

- Done when your output matches "Expected when done". Then `rm wordpress/open_today_starter.php`.
- Finished early? `event_list_starter.php` (data-wp-each), then `debug_stock_badges.php`.

PREDICT, THEN RUN

Write your prediction on the line first, then run it in the WP-CLI container shell (`docker compose run --rm wpcli sh`) on the site after Lesson 6.4.

P1

Predict the output, then run it. (A genre the shop does not have.)

```
curl -s 'http://localhost:8080/find-a-book/?genre=romance' | grep -o 'visibleC
```

P2

Predict the output, then run it. (A directive whose state does not exist.)

```
wp eval 'echo wp_interactivity_process_directives( "<div data-wp-interactive=\
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Find a Book Page (lesson page, "Project: A Find a Book Page"). In class: Step 1. At home: Steps 2 and 3.

- ☐ Step 1: the page "Find a Book" with the paragraph and the Book Filter; `?genre=dystopian` shows "Showing 3 of 11 books"
- ☐ Step 2: every genre without a reload and with an empty Console; a reload on ?genre=classic; the keyboard; JavaScript off
- ☐ Step 3: Filter test deleted; the menu shows Find a Book; package.json has --experimental-modules twice; 14 features

SELF-CHECK

Q1

A visitor opens /find-a-book/?genre=fantasy. Where does the browser's store get state.genre's starting value from?

A) From the address bar, read by view.js

B) From the state JSON that render.php's wp_interactivity_state() put into the page

C) From localStorage

D) From a REST request the store sends on load

Q2

Which of these directives does the SERVER not evaluate when it renders the block?

A) `data-wp-bind--hidden="!state.isVisible"`

B) `data-wp-text="state.visibleCount"`

C) `data-wp-class--is-selected="state.isSelected"`

D) `data-wp-on--click="actions.selectGenre"`

Q3

On the course site after Lesson 6.4 (three Dystopian books of eleven): what does this print?

```
curl -s 'http://localhost:8080/find-a-book/?genre=dystopian' | grep -c '<li hi
```

Q4

Each genre link must know its own slug. Where does that slug belong?

A) In the element's context, `wp_interactivity_data_wp_context( array( 'slug' => $slug ) )`

B) In the state, as one key per genre

C) In a global JavaScript variable

D) In the link's class name

Q5

An action calls event.preventDefault() to stop a link. What does WordPress 6.8 and later require for it?

A) Nothing: actions always run synchronously

B) data-wp-on-async--click instead of data-wp-on--click

C) Wrapping the action in withSyncEvent()

D) Returning false from the action

Q6

A visitor's browser blocks the Book Filter's script module, and she clicks Fantasy. What happens, and why?

JOURNAL

For the Book Filter, write three short lists: the state, the derived state, and the context, and say for each item why it belongs in that list. Then pick an interactive feature you have seen on another site (an accordion, a "load more" button, a size picker) and design it the same way: what is the one piece of state, what is derived, what does each element need to know about itself, and how would it work without JavaScript?

7.1 Using the REST API

BY THE END YOU CAN

- Explain routes, endpoints, methods, JSON responses and status codes, and list the routes your site offers
- Read the catalog from core endpoints and shape the answer with _fields, _embed, context, filters and the pagination headers
- Authenticate the right way: cookie plus nonce for logged-in JavaScript, application passwords for other programs
- Call the REST API from JavaScript with apiFetch, and handle its errors
- Read REST errors (400, 401, 403, 404) and tell "who are you?" from "you may not"

KEY IDEAS

- **The REST API** serves the site's data as JSON at `/wp-json/` (or `?rest_route=`): namespaces (`wp/v2`), routes, endpoints (route + method). Core builds routes for every type registered with show_in_rest.
- **Shape the answer:** `_fields` (keep `_links` and `_embedded` when embedding), `_embed`, `context`, filters such as `genres` (IDs) and `search`; page with `per_page` / `page` and the headers **X-WP-Total**, **X-WP-TotalPages**, Link.
- **Cookie + nonce** for the site's own JavaScript: without the X-WP-Nonce the request is anonymous, with a wrong one it is refused (CSRF protection).
- **Application passwords** for programs: one per program, Basic authentication over HTTPS (or locally), the smallest role that works, revocable; never the account password.
- **apiFetch** adds the root URL, the nonce and the locale, sends data as JSON and rejects with the server's error object.
- **Errors** are `{ code, message, data.status }`: **400** the request, **401** who are you?, **403** you may not, **404** no such route or item.

WORKED EXAMPLE: TWO QUESTIONS A CLIENT ASKS FIRST

A client that needs to know how big the catalog is never downloads it: it asks for the headers only. With pages of five:

```
curl -s -I 'http://localhost:8080/wp-json/wp/v2/books?per_page=5' | grep -i '^X-WP
```

```
X-WP-Total: 11
X-WP-TotalPages: 3
```

Eleven books, three pages of five. And a client that knows a book's slug (from a ?genre= link, a URL) asks for exactly the fields it needs:

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?slug=the-hobbit&_fields=id,link
```

```
[{"id":35,"link":"http:\/\/localhost:8080\/books\/the-hobbit\/"}]
```

The REST API escapes slashes in JSON strings (`\/`); any JSON parser turns them back. A slug filter returns a LIST, even for one book.

GUIDED LAB

- From the project folder: `cp labs/module-13/l7_1-rest-api/catalog_queries_starter.sh wordpress/` , then `docker compose run --rm wpcli sh catalog_queries_starter.sh` .
- For each TODO, try the request alone in a container shell first, then put it in the script. Quote every URL that contains `&` .
- TODO 2 needs the ID TODO 1 found: the genres filter does not take slugs.

- Done when your output matches "Expected when done". Then `rm wordpress/catalog_queries_starter.sh` .
- Finished early? `who_may_starter.sh` (predict the whole table first), then `debug_stock_list.php` .

PREDICT, THEN RUN

Write your prediction on the line first, then run it in the WP-CLI container shell (`docker compose run --rm wpcli sh`) on the site after Lesson 6.4.

P3

Predict the output, then run it. (Books whose text contains "the", one per page.)

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?search=the&per_page=1&_fiel
```

P4

Predict the output, then run it. (One genre, two fields.)

```
curl -s 'http://localhost:8080/wp-json/wp/v2/genres/8?_fields=name,count'; ech
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Stock at a Glance (lesson page, "Project: Stock at a Glance"). In class: Step 1. At home: Steps 2 to 4.

- ☐ Step 1: the inventory script marks Fahrenheit 451 sold out and back in stock with Maria's "Inventory script" password; the book page follows
- ☐ Step 2: `src/Admin/StockWidget.php`, the fifteenth feature
- ☐ Step 3: `assets/js/stock-widget.js` (wp.apiFetch, textContent, aria-live)
- ☐ Step 4: Maria's Dashboard shows all eleven books; each button switches the stock without a page load; Gatsby and Catcher sold out again

SELF-CHECK

Q7

Anonymously, `GET /wp-json/wp/v2/books/30?context=edit`. What comes back?

A) 200 with the raw title and the content as block markup

B) 404 rest_no_route

C) 401 rest_forbidden_context

D) 403 rest_cannot_edit

Q8

On the course site (six Classic books, genre ID 7): what does this print?

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?genres=7&per_page=4&_fields
```

Q9

A Dashboard page sends Maria's cookie and an X-WP-Nonce header with an old, made-up value. What does the REST API answer?

A) 403 rest_cookie_invalid_nonce: the page must fetch a fresh nonce

B) 200: the cookie is enough

C) 401 rest_not_logged_in, as if no cookie had been sent

D) 500: the nonce check crashed

Q10

`/wp/v2/books/30?_embed=wp:term&_fields=id,title` **answers without any genres. Why?**

A) The book has no genres

B) _embed works only for authenticated requests

C) wp:term is not an embeddable relation

D) _fields removed _links, so there was nothing to follow, and _embedded is cut off too

Q11

Why can you create application passwords on the course site although it runs on plain http://localhost:8080?

A) Application passwords never need HTTPS

B) WP_ENVIRONMENT_TYPE is local; elsewhere WordPress offers them only on HTTPS

C) Docker encrypts the traffic

D) Because the administrator created them

Q12

Name two things wp.apiFetch() adds to a request that a plain fetch() on the Dashboard does not.

JOURNAL

List every program that might want to talk to the Pages & Co. site (the inventory system, a phone app, a newsletter tool, a partner bookstore...). For each one write: what it reads, what it changes, how it should authenticate, and which WordPress user and role it should run as. Which of them need no authentication at all?

CHECKPOINT 2: MODULES 5 AND 6

25 minutes, on your own, open notes (the lesson pages, your own plugin, pagesco-plan.txt). Your instructor hands out `labs/module-13/checkpoint-2/checkpoint_starter.php` at 2:20. Copy it into wordpress/ as `checkpoint_YOURNAME.php`, write the bodies of the four functions, and run it with `wp eval-file --use-include checkpoint_YOURNAME.php` until it prints four lines without a message. Do not change the main section; the script must change nothing on your site.

PARTS A TO C: THE WEEKLY PICKS REPORT (IN THE FILE)

- **Part A (Lessons 5.2, 5.3):** `pagesco_cp_cheap_books( float $max )`: the books in stock that cost at most $max, cheapest first, as "Title ($8.49)": one get_posts() with a meta_query (price as a number, stock '1') and orderby meta_value_num.
- **Part B (Lesson 5.5):** `pagesco_cp_events_at( string $location )`: how many events take place at a location: one COUNT query on the events table through `$wpdb->prepare()` (the main section tries a hostile location).
- **Part C (Lessons 6.2, 6.3):** `pagesco_cp_list_titles( string $genre )`: render a Book List block for the genre, cheapest first, with do_blocks() and return its titles; `pagesco_cp_cards_in( string $slug )`: how many Book Cards a post stores and the first one's comment attributes.

PART D: THREE WRITTEN ANSWERS

D1

A bookseller writes a review of one book for the newsletter archive; the front page shows "the three cheapest fantasy books right now". Static block or dynamic block for each, and why?

D2

A settings form in the admin checks `current_user_can( 'manage_options' )` before saving. Lesson 5.3 says that is not enough. What is missing, and which attack does it stop?

D3

The Book Filter still filters when its script module fails to load. Name the two things in render.php that make that work.

EXIT TICKET

Three questions, no notes, right after the checkpoint.

E1

In the Book Filter, name the state and one piece of derived state.

E2

401 or 403: which one means "who are you?"

E3

Which credential does a partner's nightly price script use?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the Find a Book page (Steps 2 and 3) and Stock at a Glance (Steps 2 to 4)
- ☐ Lesson 6.4 Hands-on Exercises 1 and 2 (Exercise 3 is a stretch)
- ☐ Lesson 7.1 Hands-on Exercises 1 and 2 (Exercises 3 and 4 are stretches)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Before the Next Lesson: the widget works for Maria; "Inventory script" kept safe (not in wordpress/); Devon's "Test" password created; Lessons 5.3 and 5.5 reread
- ☐ Optional: `event_list_starter.php`, `who_may_starter.sh`, both debug handouts
- ☐ Bring to Session 14: a site at the end of Lesson 7.1, both application passwords, this workbook

GLOSSARY

TERM	MEANING
directive	A `data-wp-*` attribute that ties an element to the store (`data-wp-text`, `data-wp-bind--hidden`, `data-wp-on--click`)
state / derived state	The store's data, shared by all its elements / values computed from it (PHP closures, JS getters)
context	Data attached to one element and the directives inside it (`wp_interactivity_data_wp_context()`, `getContext()`)
store	`store( namespace, { state, actions } )` in view.js: the browser half
script module	A native JavaScript module (`<script type="module">`) loaded with an import map; built with `--experimental-modules`
hydration	The browser's store taking over the HTML the server rendered
route / endpoint	A path pattern under /wp-json/ / a route plus a method
`_fields`, `_embed`	Which fields to return / include linked resources in `_embedded`

TERM	MEANING
X-WP-Total, X-WP-TotalPages	Headers with the number of matching items and of pages
nonce (`wp_rest`)	The token that proves a cookie request comes from the site's own page (X-WP-Nonce)
application password	A per-program password for REST requests (Basic authentication), revocable, never for wp-login.php
apiFetch	WordPress's fetch wrapper: root URL, nonce, JSON, errors (`wp.apiFetch` , dependency wp-api-fetch)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 14: Custom Endpoints, External APIs and Cron

Your workbook for Session 14: a warm-up, the key ideas of Lessons 7.2 and 7.3, worked examples, the guided labs, predict-then-run practice, a self-check for each lesson, and your exit ticket.

Lessons 7.2–7.3 | Session 14 · 3 hours | Name: _____

SESSION GOALS

- Register routes with arguments, a schema, proper errors and a real permission_callback.
- Say in which order WordPress validates, sanitizes, checks permission and calls your callback.
- Call another server with wp_remote_get() and handle each way it can fail.
- Cache answers in transients, failures included, and schedule work with WP-Cron exactly once.
- Start the Events API and the Supplier Sync.

WARM-UP: LESSONS 6.4 AND 7.1

Answer from memory first; the class goes through them together.

W1

curl asks /find-a-book/?genre=fantasy. How many books does the server send with the hidden attribute?

W2

Why does selectGenre need `withSyncEvent()`?

W3

A client gets 403 rest_cannot_edit. Fix the request, the credentials or the role?

W4

How does a client learn how many pages the catalog has at per_page=5, without reading the books?

W5

Why can the Stock at a glance widget POST to /wp/v2/books with plain apiFetch, and an outside script cannot?

LESSONS

7.2 Custom REST Endpoints

BY THE END YOU CAN

- Register routes with register_rest_route() on rest_api_init, as a feature class of the plugin
- Declare arguments with types, defaults, limits and enums, and add your own validate_callback and sanitize_callback without losing the built-in checks
- Describe responses with a JSON Schema, and return WP_REST_Response objects with headers and status codes
- Return errors as WP_Error with the right status, and know what WordPress does with one that has none
- Write a real permission_callback for every route, and explain why __return_true is only for public reads

KEY IDEAS

- `register_rest_route( 'pagesco/v1', '/books', ... )` on **rest_api_init**; a route holds endpoints (methods, callback, permission_callback, args) and a schema. The namespace is vendor/version.
- **Arguments with a type** are validated and sanitized from their schema (minimum, maximum, enum, required, default). A sanitize_callback of your own replaces that: keep `'validate_callback' => 'rest_validate_request_arg'` next to it.
- **Order of work:** validate, sanitize, permission, callback. Your callback receives clean, typed values.
- **Return** data, a WP_REST_Response (status, headers such as X-WP-Total) or a **WP_Error with a prefixed code and an explicit status**; without one it is a 500. Do not leak what exists.
- **Every route has a permission_callback:** `__return_true` only for public reads; writes check a capability; `rest_authorization_required_code()` gives 401 or 403.
- **OPTIONS** shows the arguments and the schema; **rest_do_request()** calls routes from PHP without HTTP.

WORKED EXAMPLE: A ROUTE FROM PHP, AND ITS MENU

Code on the same site can call a route without HTTP: the same validation, permission and callback, at the speed of a function call (the site after Lesson 7.2):

```
wp eval '$r = rest_do_request( new WP_REST_Request( "GET", "/pagesco/v1/books/30"
```

```
200 Dune ["fiction","science-fiction"]
```

And the events route's menu: which methods each endpoint answers, as OPTIONS reports them:

```
curl -s -X OPTIONS http://localhost:8080/wp-json/pagesco/v1/events | grep -o '"met
```

```
"methods":["GET","POST"]
"methods":["GET"]
"methods":["POST"]
```

The route as a whole (GET and POST), then each endpoint: READABLE is GET, CREATABLE is POST. A client learns what it may send before it sends anything.

GUIDED LAB

- From the project folder: `cp -r labs/module-14/l7_2-custom-endpoints/opening_hours_starter wordpress/wp-content/plugins/pagesco-opening-hours`, then `wp plugin activate pagesco-opening-hours`.
- `cp labs/module-14/l7_2-custom-endpoints/test_opening_hours.sh wordpress/`, then `docker compose run --rm wpcli sh test_opening_hours.sh`.
- TODO 1: the day argument (type, enum, no default). TODOs 2 and 3: the helper and the two answers. Run the test after each.

QUESTION	YOUR ANSWER
Which line of yours produced the 400 for funday?	
What does $request['day'] hold when no day is sent?	
Why is __return_true right here?	

- Done when the test prints Saturday's hours, Sunday closed, a 400, seven days and the argument's description. Then deactivate and `rm -r` the plugin and the test.
- Finished early? `schema_rules_starter.php`, then `debug_staff_picks_route.php`.

PREDICT, THEN RUN

Write your prediction on the line first, then run it (`wp ...` from the project folder, curl in the WP-CLI container shell). P1 and P2 need the site after Lesson 7.2; P3 and P4 after Lesson 7.3.

P1

Predict the output, then run it. (A page size of zero.)

```
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?per_page=0'; echo
```

P2

Predict the output, then run it. (A POST with no arguments and no user.)

```
wp eval '$r = rest_do_request( new WP_REST_Request( "POST", "/pagesco/v1/event
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Events API (lesson page, "Project: The Events API"). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ Step 1: `src/Rest/EventsEndpoint.php`, the seventeenth feature
- ☐ Step 2: GET /pagesco/v1/events shows five events; `?location=Café` two
- ☐ Step 3: POST anonymously 401, as Devon 403, Maria's bad date 400 (only starts_at reported)
- ☐ Step 4: event 7 created (201, the tag stripped); the row in the table; four routes

SELF-CHECK

Q1

A route registers `'methods' => WP_REST_Server::CREATABLE`. Which HTTP method does that endpoint answer?

A) GET

B) POST

C) POST, PUT and PATCH

D) DELETE

Q2

On the course site after Lesson 7.2 (Books per list = 6, eleven books): what does this print?

```
wp eval '$r = rest_do_request( new WP_REST_Request( "GET", "/pagesco/v1/books"
```

Q3

An anonymous client POSTs to /pagesco/v1/events WITHOUT a title. Which answer does it get first?

A) 400 rest_missing_callback_param: arguments are validated before the permission check

B) 401 rest_forbidden: the permission check comes first

C) 500: the callback fails on a missing title

D) 201: the event is created without a title

Q4

Maria sends a bad starts_at AND a negative capacity. The answer names only starts_at. Why?

A) capacity is not validated

B) The schema stops at the first error it finds in alphabetical order

C) A negative capacity is turned into 0

D) validate_callbacks run first; the schema checks of the other arguments run in the sanitizing step, which a failed validation never reaches

Q5

Which permission_callback fits a route that DELETES events?

A) '__return_true': the callback checks the ID

B) 'is_user_logged_in'

C) A check of `current_user_can( 'edit_others_posts' )` that returns a WP_Error with rest_authorization_required_code()

D) None: DELETE routes are protected by WordPress

Q6

`GET /pagesco/v1/books/14` asks for the Events page (a page, not a book). Why does the answer say only "No book was found with that ID." and not "that is a page"?

JOURNAL

Design an endpoint the shop does not have yet, for example "reserve a seat at an event" or "a customer's wish list". Write its route and method, every argument with its type and rule, the permission callback in one sentence, the success status and response, and three error responses with code and status. Which of your checks can the schema do, and which need a callback?

7.3 External APIs, Transients & WP-Cron

BY THE END YOU CAN

- Call an external API with wp_remote_get() and wp_remote_post(), and handle each way it can fail: no answer, an error status, or an answer that is not what you expected
- Keep API keys in wp-config.php and choose wp_safe_remote_get() when a URL comes from a user
- Cache answers in transients, explain where they are stored and how a persistent object cache changes that, and cache failures briefly too
- Schedule recurring work with WP-Cron, start and stop it on activation and deactivation, and inspect and run it with WP-CLI
- Register an ability with the Abilities API and run it over the REST API

KEY IDEAS

- **wp_remote_get() / wp_remote_post()** with a timeout you can afford; read answers with `wp_remote_retrieve_response_code()`, `_body()`, `_header()`.
- **Three failures, in order:** a WP_Error (no answer, timeout), a status outside 2xx (still a normal response), a body that is not the data you expect.
- **Keys and URLs live in wp-config.php** (`wp config set`), not in code; **wp_safe_remote_*()** for URLs users supply (SSRF).
- **Transients** are options with an expiry (two non-autoloaded rows, or the object cache); they can vanish early: read through, cache failures briefly, never store false.
- **WP-Cron** runs due events on the next request, as actions, without a user: schedule once on activation with `wp_next_scheduled()`, clear on deactivation, add the callback on every request.
- **The Abilities API** describes units of work (input and output schemas, a permission check) that tools can discover and run, through PHP or /wp-json/wp-abilities/v1/.

WORKED EXAMPLE: WHERE THE NOTE LIVES, AND HOW OFTEN THE JOB RUNS

Where does the supplier note live on this site? (The site after Lesson 7.3, no persistent object cache.)

```
wp db query "SELECT option_name, autoload FROM wp_options WHERE option_name LIKE '
```

```
option_name autoload
_transient_timeout_pagesco_books_supplier_availability  off
_transient_pagesco_books_supplier_availability  off
```

Two rows, not autoloaded: the value and its expiry time. And how often the nightly job repeats:

```
wp eval 'echo wp_get_schedule( "pagesco_books_daily_sync" ), "\n";'
```

```
daily
```

`wp_get_schedule()` names the recurrence of the next event of a hook; false would mean nothing is scheduled (the plugin deactivated, or the activation code never ran on this site).

GUIDED LAB

- From the project folder: `cp labs/module-14/l7_3-external-apis-cron/remote_answers_starter.php wordpress/`, then `wp eval-file --use-include remote_answers_starter.php`.
- Write `pagesco_lab_ask()`: TODO 1 the request, TODOs 2 to 4 the three failures, in order. Run after each TODO.

- Done when your output matches "Expected when done". Then `rm wordpress/remote_answers_starter.php`.
- Finished early? `events_cache_starter.php`, `cron_plan_starter.sh`, then `debug_event_counts.php`.

PREDICT, THEN RUN

Write your prediction on the line first, then run it (`wp ...` from the project folder, curl in the WP-CLI container shell). P1 and P2 need the site after Lesson 7.2; P3 and P4 after Lesson 7.3.

P3

Predict the output, then run it. (The sandbox without a key.)

```
wp eval 'var_dump( is_wp_error( wp_remote_get( "http://localhost:8080/supplier
```

P4

Predict the output, then run it. (A private address, the safe way.)

```
wp eval 'echo wp_safe_remote_get( "http://10.0.0.1/" )->get_error_message(), "
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

The Supplier Sync (lesson page, "Project: The Supplier Sync"). In class: Steps 1 and 2. At home: Steps 3 and 4.

- ☐ Step 1: all eleven books have their ISBN
- ☐ Step 2: Supplier\Client and Supplier\Availability; the Supplier column on the Books screen; a simulated failure cached
- ☐ Step 3: DailySync, the new Activator, the uninstall line; one daily event at 03:00; the past event removed
- ☐ Step 4: pagesco/check-stock answers Maria over REST and refuses Devon; twenty features

SELF-CHECK

Q7

The supplier's host does not resolve, and code calls `wp_remote_retrieve_body( $response )` on the WP_Error. What does it get?

A) A fatal error

B) null and a Warning

C) The error message

D) An empty string

Q8

What does this print, for a transient nobody ever set?

```
wp eval 'var_dump( get_transient( "pagesco_books_nothing_here" ) );'
```

Q9

A plugin fetches a book cover from a URL an editor types into a field. Which function?

A) `wp_safe_remote_get()`: it refuses private networks and odd ports

B) `wp_remote_get()` with a short timeout

C) `file_get_contents()`

D) `wp_remote_head()`

Q10

You want to run the daily sync now, while developing, without moving tomorrow's 03:00 run. What do you type?

A) `wp cron event run pagesco_books_daily_sync`

B) `wp eval 'do_action( "pagesco_books_daily_sync" );'`

C) `wp cron test`

D) `curl -s http://localhost:8080/wp-cron.php`

Q11

Where does the supplier's API key belong?

A) In the plugin's Client class, as a constant

B) In a transient, refreshed daily

C) In wp-config.php (or an environment variable it reads), on each site

D) In the REST API response, for the JavaScript

Q12

On the course site after Lesson 7.3: what does this print?

```
wp cron event list --hook=pagesco_books_daily_sync --fields=hook,recurrence
```

JOURNAL

Pick a real service a client of yours might want on their site (shipping rates, a newsletter list, weather for an outdoor event, exchange rates). Write down: its endpoint, how it authenticates and where the key will live, what you would cache and for how long, what the page shows when the service is down, and whether a cron job, a webhook or a page view should trigger the call.

EXIT TICKET

Three questions, no notes.

E1

Name the four desks a REST request passes, in order.

E2

Name the three ways a call to another server can fail.

E3

Where does a plugin schedule its daily job, and what guards it?

NOTES

HOMEWORK CHECKLIST

- ☐ Finish the Events API (Steps 3 and 4) and the Supplier Sync (Steps 3 and 4)
- ☐ Lesson 7.2 Hands-on Exercises 1 and 2 (Exercise 3 is a stretch)
- ☐ Lesson 7.3 Hands-on Exercises 1 to 3 (Exercise 4 needs the internet)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ Before the Next Lesson: the Supplier column, one daily event, no scratch files left; Node 22.13+ for Lesson 8.1
- ☐ Optional: `schema_rules_starter.php`, `events_cache_starter.php`, `cron_plan_starter.sh`, both debug handouts
- ☐ Bring to Session 15: a site at the end of Lesson 7.3, this workbook

GLOSSARY

TERM	MEANING
`register_rest_route()`	Adds a route under a namespace, on rest_api_init
argument schema	type, default, minimum, maximum, enum, required: what WordPress checks before your callback
validate_callback / sanitize_callback	Your own check (true or a WP_Error) / your own cleaning of one argument
permission_callback	Who may call the endpoint: true, false or a WP_Error
WP_REST_Response	Data plus status and headers
`rest_do_request()`	Runs a REST request inside PHP, without HTTP
HTTP API	`wp_remote_get()`, `wp_remote_post()`, `wp_remote_request()` and the `wp_remote_retrieve_*()` helpers
SSRF	Server-side request forgery: making a server request internal addresses; `wp_safe_remote_*()` refuses them
transient	An option with an expiry: `set_transient()`, `get_transient()` (false = missing), `delete_transient()`
read-through	Read the cache; on a miss, fetch, store, return
WP-Cron	Scheduled events (hooks) run on the next request after their time
ability	A described unit of work for tools and assistants (`wp_register_ability()`)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 15: Debugging, Testing and Performance

Your workbook for Session 15: a warm-up, the key ideas of Lessons 8.1 and 8.2, worked examples, the guided labs, predict-then-run practice, the two projects' checklists, a self-check for each lesson, and your exit ticket.

Lessons 8.1–8.2 | Session 15 · 3 hours | Name: _____

SESSION GOALS

- Debug with evidence: the debug constants, `error_log()` from a hook such as `http_api_debug`, a backtrace on `doing_it_wrong_run`, recovery mode and `--skip-plugins`.
- Know what an integration test is, why it never uses the network, and run the plugin's tests in a wp-env test environment of their own.
- Bring the plugin to zero PHPCS findings, with every exception narrow and explained.
- Measure what a page costs before you change it: SAVEQUERIES and a query report, the object cache, N+1, the browser's Core Web Vitals.
- Cache your own query with a key that knows when the data changed, and remove a query from every request.

WARM-UP: LESSONS 7.2 AND 7.3

Five questions from last session, no notes. Compare with a neighbor, then with the room.

W1

Name the three ways a call to the supplier can go wrong, and what wp_remote_get() returns for each.

W2

The Supplier column reads a transient. What happens when the transient has vanished?

W3

Why does the lesson run the daily job with wp eval 'do_action("pagesco_books_daily_sync");' instead of wp cron event run?

W4

Your REST route returns a WP_Error without a status. What does the client get?

W5

Where do the supplier's address and key live on your site, and why there?

LESSONS

8.1 Debugging & Testing

BY THE END YOU CAN

- Configure WP_DEBUG, the debug log and error display for development and production, and keep the log private
- Trace a problem with error_log(), hooks such as http_api_debug and doing_it_wrong_run, and wp_debug_backtrace_summary()
- Recover from a fatal error with recovery mode and WP-CLI's --skip-plugins, and use Query Monitor to look inside a request
- Write PHPUnit integration tests with WP_UnitTestCase, factories and pre_http_request, and run them in a wp-env test environment
- Set up PHPCS with the WordPress Coding Standards for a plugin, fix what it finds, and justify the exceptions with a ruleset or phpcs:ignore

KEY IDEAS

- **Debugging is detective work:** symptom, suspects, evidence. Make the program tell you what it does; never change code until the symptom goes away.
- **Debug settings:** locally WP_DEBUG, WP_DEBUG_LOG and WP_DEBUG_DISPLAY on; on a live site display always off and the log outside the web root (`wp-content/debug.log` is downloadable).
- **Make code talk:** `error_log()` with `wp_json_encode()`; hooks such as `http_api_debug`; `wp_debug_backtrace_summary()` on `doing_it_wrong_run` names the code behind a notice. Remove debugging code afterwards; never log secrets.
- **When the site goes down:** the fatal error handler and recovery mode (an email for protected pages); `wp --skip-plugins=<slug>` repairs from the command line; a broken must-use plugin is renamed.
- **Query Monitor** shows queries, hooks, HTTP calls and more for each page: install it to investigate, remove it (and its db.php) afterwards.
- **Integration tests** run the plugin in a real, fresh WordPress: `WP_UnitTestCase`, factories, a rollback after each test; `pre_http_request` instead of the network; a wp-env test environment of their own; the project's `vendor/bin/phpunit`.
- **PHPCS with WPCS** reads every line: a phpcs.xml.dist with the text domain, prefixes and scope; every finding is fixed, auto-fixed (`composer lint:fix`) or documented, narrowly and with a reason.

WORKED EXAMPLE: WHAT A TEST ASKS OF A WP_ERROR

Every test of the supplier client asks the same three questions of what the client returned: is it an error, which one, and what is in its data? A WP_Error up close, on your site:

```
wp eval '$e = new WP_Error( "pagesco_supplier_http_error", "The supplier answered
```

```
bool(true)
string(27) "pagesco_supplier_http_error"
int(503)
```

`is_wp_error()` is the first question, the code the second (tests compare it with `assertSame()`: an exact string, not "some error"), the data the third (here the status a 503 left behind). And the fake that answers in place of the network exists only inside the test: on the real site nothing is hooked to `pre_http_request`:

```
wp eval 'var_dump( has_filter( "pre_http_request" ) );'
```

```
bool(false)
```

`has_filter()` returns false when nothing is attached. A filter added inside a WP_UnitTestCase test is removed after that test; a filter added in a lab script ends with the script. Neither ever reaches your visitors.

GUIDED LAB

- From the project folder: `cp labs/module-15/l8_1-debugging-testing/fake_supplier_starter.php wordpress/`, then `wp eval-file --use-include fake_supplier_starter.php`. Open `wordpress/fake_supplier_starter.php` in VS Code.
- Check 1 is done for you: read `pagesco_lab_fake()` (the answer) and `pagesco_lab_check()` (the condition). For checks 2 to 4, write one of each. Run again after each one.
- For each check write one sentence: which bug in Client.php would make it say FAIL?

- Done when the last line says `4 of 4 checks passed, 4 requests answered by the fake, none by the network`. Then `rm wordpress/fake_supplier_starter.php`.
- Finished early? `debug_supplier_log.php` in the same folder, then the folder lab `next_event_starter/` with `test_next_event.sh` (after the project's Step 2).

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P1

Predict the output, then run it. (WP-CLI and PHP's display_errors.)

```
wp eval 'var_dump( WP_DEBUG_DISPLAY, ini_get( "display_errors" ) );'
```

P2

Predict the output, then run it. (A fake network answer, without the plugin.)

```
wp eval 'add_filter( "pre_http_request", fn() => new WP_Error( "http_request_f
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Lint and Test the Plugin (lesson page, "Project: Lint and Test the Plugin"). In class: Steps 1 to 3. At home: Step 4 and the Quality note.

- ☐ Step 1: composer.json with require-dev and three scripts; `composer update`; `composer show --direct` lists PHPUnit, WPCS and the polyfills
- ☐ Step 2: phpcs.xml.dist; the first report: _____ errors and _____ warnings in _____ files
- ☐ Step 3: auto-fixed, fixed, documented; `composer lint; echo "exit status $?"` says 0
- ☐ Step 4: .wp-env.json, phpunit.xml.dist, tests/bootstrap.php, four test classes; `OK (12 tests, 30 assertions)`; one test red on purpose, then green again

SELF-CHECK

Q1

The live Pages & Co. shop should keep a record of PHP warnings for the developer, and show visitors none. Where should `WP_DEBUG_LOG` point?

A) `true`, so the messages go to `wp-content/debug.log`

B) A file outside the web root, such as `/home/pagesco/logs/wordpress.log`

C) `false`: a live site must never record anything

D) The theme's folder, where only the theme developer looks

Q2

On the course site after Lesson 7.3, with WordPress loaded (`wp eval-file`): what does this print?

```php
<?php
add_filter(
    'pre_http_request',
    fn() => array(
        'headers'  => array(),
        'body'     => '<h1>Service Unavailable</h1>',
        'response' => array( 'code' => 503, 'message' => 'Service Unavailable'
        'cookies'  => array(),
        'filename' => null,
    )
);
$result = ( new PagesCo\Books\Supplier\Client() )->availability( array( '97804
echo $result->get_error_code(), ' ', $result->get_error_data()['status'], "\n"
```

Q3

A plugin crashes on `init` and every WP-CLI command now dies with the same fatal error. Which command still deactivates it?

A) `wp plugin deactivate pagesco-crash --force`

B) `wp eval 'deactivate_plugins( "pagesco-crash/pagesco-crash.php" );'`

C) `wp plugin delete pagesco-crash`

D) `wp --skip-plugins=pagesco-crash plugin deactivate pagesco-crash`

Q4

Why does the lesson name its temporary backtrace logger `0-pagesco-debug-backtrace.php`?

A) Must-use plugins load in alphabetical order: it must be there before the plugin that asks too early

B) WordPress ignores must-use plugins whose names start with a letter

C) The 0- tells WordPress to load it only when WP_DEBUG is on

D) So that PHPCS skips the file

Q5

PHPCS reports 30 times `PrefixAllGlobals.NonPrefixedVariableFound` in the blocks' `render.php` files. What did Lesson 8.1 do, and why?

A) Renamed every variable to start with pagesco_books_

B) Turned the rule off for the whole plugin

C) Excluded that rule for those two file patterns in phpcs.xml.dist, with a comment: WordPress includes them inside a function, so the variables are local

D) Added // phpcs:ignore to each of the 30 lines

Q6

In the plugin's folder, with the wp-env test environment started: which command runs the plugin's PHPUnit tests, and why not just `phpunit`?

JOURNAL

Think of the last bug you chased, in any project. How did you find it and how long did it take? Which tool from today (the log, a hook that logs, a backtrace, Query Monitor) would have shortened the hunt, and which test would have caught it before it happened? Then list three behaviors of the Pages & Co. plugin that have no test yet, and the one you would write first.

8.2 Performance & Caching

BY THE END YOU CAN

- Measure what a request costs with SAVEQUERIES and $wpdb->queries, read an EXPLAIN of a meta query, and recognize missing options and N+1 queries
- Explain the object cache, its persistent version and transients, choose between them, and cache your own query with a key that is invalidated when the data changes
- Describe how WordPress 7.1 loads block CSS, script modules and classic scripts, and use wp_preload_resources and fetchpriority where they help
- Explain srcset, sizes, lazy loading and why block templates get no automatic image optimization
- Measure LCP, CLS and interaction timing in a browser, and say honestly what a local lab measurement can and cannot tell you

KEY IDEAS

- **Measure first,** several times, one change at a time. Three places: the server (queries), the network (what the page downloads), the browser (Core Web Vitals).
- **SAVEQUERIES** in wp-config.php keeps every query with its time and caller in `$wpdb->queries`; `$wpdb->num_queries` counts. For measuring sessions only.
- **Missing options** cost a query per request; **N+1** (one meta read per item after `'fields' => 'ids'`) costs a query per item: `update_postmeta_cache( $ids )` loads them in one.
- **The object cache** remembers within a request; a **persistent** one (object-cache.php: Redis, Memcached) between requests, and it moves transients out of wp_options. **Transients** outlive the request on any site.
- **Cache with an invalidation plan:** a key with `wp_cache_get_last_changed( $group )`, renewed by `wp_cache_set_last_changed( $group )` on every write.
- **What the browser downloads:** block CSS only for blocks on the page, inlined up to 40,000 bytes; script modules deferred; speculation rules prefetch the next page.
- **Images:** content images get srcset, sizes, lazy loading and fetchpriority; a block theme's template images get nothing: give them an aspect ratio, and preload the LCP image (`wp_preload_resources`, `fetchpriority`). Never lazy-load the LCP image.
- **Core Web Vitals** (LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1) are judged on field data; lab numbers compare changes.

WORKED EXAMPLE: WHAT EVERY REQUEST LOADS

Every autoloaded option is loaded on every request, used or not (Lesson 2.2). How many does your site have, and which are the biggest?

```
wp option list --autoload=on --format=count; echo
```

```
101
```

The largest three, by the size of their value:

```
wp db query "SELECT option_name, LENGTH(option_value) AS bytes FROM wp_options WHE
```

```
option_name bytes
_transient_wp_core_block_css_files  23778
rewrite_rules   11118
wp_user_roles   3451
```

WordPress's own caches lead (the list of core block stylesheets, the rewrite rules, the roles). Lesson 8.2's settings row is in this list too since Step 2, as a few bytes: one row loaded with all the others costs nothing extra, which is why it is cheaper than a missing option asked for on every request. Site Health warns when autoloaded options pass 800 KB in total.

GUIDED LAB

- From the project folder: `cp labs/module-15/l8_2-performance-caching/query_count_starter.php wordpress/`, then `wp eval-file --use-include query_count_starter.php`.
- Before each TODO, write your two numbers (first time, second time) in the comment. Then write the `pagesco_lab_row()` call and run.
- Finished early? `cheapest_cache_starter.php`, then `page_weight_starter.sh` (run it with `docker compose run --rm wpcli sh page_weight_starter.sh`), then `debug_events_cache.php`.

ROW	MY PREDICTION	REAL	WHY THE DIFFERENCE?
2 missing option			
3 two meta values			
4 WP_Query twice			
5 one by one			
6 update_postmeta_cache			

PREDICT, THEN RUN

Write your prediction on the line first, then run it on your site from the project folder. A wrong prediction you can explain afterward is worth more than a right guess.

P3

Predict the output, then run it. (One post, asked for twice.)

```
wp eval 'global $wpdb; $b = $wpdb->num_queries; get_post( 30 ); get_post( 30 )
```

P4

Predict the output, then run it. (Is there a persistent object cache?)

```
wp eval 'var_dump( wp_using_ext_object_cache() );'
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

A Faster Shop, with Numbers (lesson page, "Project: A Faster Shop, with Numbers"). In class: Steps 1 to 3. At home: Steps 4 and 5.

MEASURED	BEFORE	AFTER
Queries per page (front page, Dune, Find a Book, Events)		
Queries for /pagesco/v1/books		
Queries to collect the ISBNs		
Front page CLS (DevTools, Slow 4G)		
Front page LCP		

- ☐ Step 1: EventsTable caches find(); a past event appears and disappears through the cache
- ☐ Step 2: the settings row: `a:0:{}` and `auto`; pagesco_books_settings gone from the missing options

- ☐ Step 3: update_postmeta_cache(); 12 queries became 2
- ☐ Step 4: the hero image's aspectRatio and preload, measured ☐ Step 5: SAVEQUERIES and the report removed, `composer lint` 0

SELF-CHECK

Q7

On the course site: what does this print?

```
wp eval 'global $wpdb; $b = $wpdb->num_queries; get_option( "pagesco_lab_nope"
```

Q8

On a site without a persistent object cache, these two commands run one after the other. What does the second print?

```
wp eval 'wp_cache_set( "stock", 24, "pagesco_demo" );'
wp eval 'var_dump( wp_cache_get( "stock", "pagesco_demo" ) );'
```

A) `int(24)`

B) `bool(false)`

C) `NULL`

D) A Warning: the group pagesco_demo does not exist

Q9

A shop with 500 books runs `get_posts( array( 'post_type' => 'book', 'fields' => 'ids', 'posts_per_page' => -1 ) )`, then `get_post_meta()` for each ID, with an empty cache. About how many queries is that, and what is the fix?

A) 2 queries: WordPress loads the meta with the IDs

B) 500 queries; the fix is a persistent object cache

C) 501 queries; the fix is update_postmeta_cache($ids) before the loop: 2 queries

D) 1 query; meta is autoloaded like options

Q10

EventsTable::find() treats a cached value as a hit with `if ( is_array( $events ) )`. A colleague suggests the shorter `if ( $events )`. What would break?

A) Nothing: both conditions treat every cached value the same way

B) A cached empty result, array(), would count as a miss: a location without events would query the database on every call

C) A cached WP_Error would reach the shortcode

D) Two different limits would share one cache entry

Q11

In Lesson 8.2's lab measurement the front page had a CLS of 0.101, just over the 0.1 limit. What caused it, and what fixed it?

A) The web fonts; font-display: swap fixed it

B) The speculation rules prefetching the next page

C) The Navigation block's external stylesheet; inlining it fixed it

D) The hero image had no reserved space; the Image block's aspectRatio attribute fixed it

Q12

Which WordPress filter adds a `<link rel="preload">` for the front page's hero image to the head, and which attribute tells the browser to fetch it first?

JOURNAL

Think of a WordPress site you know that feels slow, or open one in Chrome with the Performance panel. Write your guess for why, before measuring. Then measure: its LCP element, its CLS, the size of its HTML, how many stylesheets and scripts it loads. Was your guess right? Write the one change you would make first, and how you would prove that it helped.

EXIT TICKET

Three questions, no notes. Write your answers, hand them in or keep them: they tell you and your instructor what to revisit.

E1

A live site shows "There has been a critical error on this website." Name two places to look, and one command that still works when a plugin is the cause.

E2

Why does `EventsTable::find()` put `wp_cache_get_last_changed( 'pagesco_events' )` into its cache key?

E3

Name one thing a lab measurement on your laptop can tell you, and one it cannot.

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 8.1 project: Steps 1 to 3 finished (`composer lint` exits 0), Step 4 (twelve tests green in wp-env, one red on purpose), the Quality note
- ☐ Lesson 8.2 project: Step 4 (the hero image, measured) and Step 5 (the table, the instruments removed), the Performance note
- ☐ Lesson 8.1 Hands-on Exercises 1 and 3 (Exercise 2 is a stretch)
- ☐ Lesson 8.2 Hands-on Exercises 1 and 2 (Exercise 3 is a stretch)
- ☐ Both Quick Quizzes on the lesson pages
- ☐ Both journal entries
- ☐ 8.2 "Before the Next Lesson": no SAVEQUERIES, no drop-in, the twelve tests pass
- ☐ Optional: `debug_supplier_log.php`, `next_event_starter/`, `cheapest_cache_starter.php`, `page_weight_starter.sh` (before and after Step 4), `debug_events_cache.php`
- ☐ Bring to Session 16: the site as Lesson 8.2 leaves it, `pagesco-live.zip` downloaded at home, Git and Node.js 24 on my computer, pagesco-plan.txt, this workbook

GLOSSARY

TERM	MEANING
`WP_DEBUG_DISPLAY` / `WP_DEBUG_LOG`	Print PHP's messages into the page / write them to a log file (`true` = wp-content/debug.log, or a path)
`http_api_debug`	An action that fires after every HTTP request WordPress makes: a place to log it
backtrace	The chain of calls that led to a line of code; `wp_debug_backtrace_summary()` writes it as one line
recovery mode	After a fatal error in a plugin or theme, a login link (emailed to the admin) that pauses that plugin
`--skip-plugins`	WP-CLI loads WordPress without the named plugins (all of them without a name)
integration test	A test that runs your code inside a real, freshly installed WordPress with a test database
`WP_UnitTestCase`	WordPress's test base class: a rollback after each test, factories, WordPress assertions
`pre_http_request`	A filter that can answer an HTTP request instead of the network: how tests fake an API
PHPCS / WPCS / phpcbf	The code sniffer / the WordPress Coding Standards for it / its automatic fixer
`SAVEQUERIES`	A constant that makes `$wpdb` keep every query of the request with its time and caller
N+1	One query for a list, then one more per item; fixed by loading all items' data in one query
object cache / persistent object cache	WordPress's in-memory cache for one request / the same kept between requests by a drop-in (Redis, Memcached)
last-changed	A group's timestamp in every cache key, renewed on every write, so old entries are never read again
LCP · INP · CLS	Core Web Vitals: when the largest element is drawn · how fast the page responds · how much it jumps

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 16: Distribution and Deployment

Your workbook for Session 16: a warm-up, the key ideas of Lessons 8.3 and 8.4, worked examples, the guided labs, predict-then-run practice, the two projects' checklists, the final project's planning page, a self-check for each lesson, and your exit ticket.

Lessons 8.3–8.4 | Session 16 · 3 hours | Name: _____

SESSION GOALS

- Write strings a translator can translate, and know how PHP and JavaScript translations reach the page.
- Check a theme and its blocks for accessibility three ways, and say what tools miss.
- Prepare a release: versions, readme, a ZIP without development files, an uninstall that deletes only on opt-in.
- Keep the shop's code in Git, build releases from a tag, move a database safely, configure a site for production.
- Read the final project's brief, milestones and rubric, and start your plan.

WARM-UP: LESSONS 8.1 AND 8.2

Five questions from last session, no notes.

W1

On a live site, where do PHP's messages go, and why not to wp-content/debug.log?

W2

Which command runs the plugin's tests, and in which site?

W3

get_posts() with 'fields' => 'ids', then get_post_meta() for each ID: what is that called, and the fix?

W4

What makes a cached result of EventsTable::find() unreachable after a new event is added?

W5

The theme's hero image got two fixes. Which, and what did each change?

8.3 Internationalization, Accessibility & Distribution

BY THE END YOU CAN

- Generate a POT file with wp i18n make-pot, write translator-friendly strings, and produce .mo, .l10n.php and JavaScript .json translations with WP-CLI
- Explain how WordPress 7.1 finds a plugin's translations, for PHP (Domain Path) and for scripts (wp_set_script_translations() and block.json)
- Check a theme and its blocks for accessibility with contrast ratios, a keyboard walk and an automated axe audit, and say what automated checks miss
- Prepare a plugin for distribution: a valid readme.txt, version numbers, the directory guidelines, a release ZIP without development files, and a clean Plugin Check report
- Write an uninstall routine that removes content only when an administrator has opted in

KEY IDEAS

- **i18n** is your code (translation functions, one text domain); **l10n** is translation files; **a11y** is a quality of every page.
- **Translator-friendly strings:** whole sentences, numbered placeholders, a translators comment, `_n()` for counts, `_x()` for ambiguous words, literals only; escape translations too.
- **make-pot → POT → PO → make-mo, make-php, make-json --no-purge.** Since 6.8, Text Domain and Domain Path are enough for PHP; scripts need `wp_set_script_translations()` (block.json's textdomain for blocks) and a JSON named after the MD5 of the script's path.
- **Accessibility:** contrast computed (4.5:1, 3:1), a keyboard walk (order, visible focus), axe or Lighthouse; then a screen reader. Tools find perhaps a third of the problems.
- **Distribution:** a valid readme.txt, consistent versions, honest requirements, ABSPATH guards, documented external services, Plugin Check; a ZIP without development files, built with a production autoloader.
- **Uninstall** deletes content only on the administrator's opt-in, and registers what it needs: the plugin is not loaded.

WORKED EXAMPLE: A PLURAL AND ITS COMMENTS

`_n()` chooses the form for a count; the language's plural rule decides how many forms there are. On your site, in English:

```
wp eval 'echo esc_html( sprintf( _n( "%d seat", "%d seats", 1, "pagesco-books" ),
```

```
1 seat | 20 seats
```

How many strings of the plugin carry a translators comment? Count them in a fresh POT (in /tmp, so nothing changes):

```
wp i18n make-pot wp-content/plugins/pagesco-books /tmp/pagesco-w.pot --exclude=src
grep -c '^#. translators:' /tmp/pagesco-w.pot
rm /tmp/pagesco-w.pot
```

```
14
```

Every one of them has a placeholder; make-pot warns about each placeholder string without one.

GUIDED LAB

- `cp labs/module-16/l8_3-i18n-a11y-distribution/plural_strings_starter.php wordpress/`, then `wp eval-file --use-include plural_strings_starter.php`.
- Read the translator's file at the top: its msgids are the strings your code must use, exactly.
- TODO 1 to 4: `_n()`, numbered placeholders, `_x()`, a whole sentence; a translators comment for each placeholder.

- Finished early? `a11y_scan_starter.sh`, then `debug_seats_spanish.php`.

PREDICT, THEN RUN

Write your prediction first, then run it on your site. A wrong prediction you can explain is worth more than a right guess.

P1

Predict the output, then run it. (Which language is this site in?)

```
wp eval 'echo get_locale(), " ", determine_locale(), "\n";'
```

P2

Predict the output, then run it. (A script path, hashed.)

```
printf '%s' 'build/blocks/book-card/index.js' | md5sum | cut -d' ' -f1
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Release 1.1.0 (lesson page, "Project: Release 1.1.0"). In class: Steps 1 and 2.

- ☐ Step 1: the delete-all setting, off by default
- ☐ Step 2: uninstall tested both ways from a backup: 11 books kept, then 0, then 11 again
- ☐ Step 3: Spanish: POT, PO, MO, l10n.php, three JSON; seen on the front end and in Maria's Dashboard
- ☐ Step 4: the notice a region, the navigation as links, the focus ring
- ☐ Step 5: 1.1.0, readme.txt, the ZIP, Plugin Check without errors, `composer lint` 0

SELF-CHECK

Q1

make-pot prints `Warning: Multiple placeholders should be ordered.` for `__( 'Showing %s of %s books', 'pagesco-books' )`. What is the fix?

A) Nothing: it is only a warning, and the POT file was written

B) Split it into two strings, one per number

C) Numbered placeholders, `'Showing %1$s of %2$s books'`, with a translators comment naming both

D) Use `_n()` instead of `__()`

Q2

The Book Card's editor script is `build/blocks/book-card/index.js`. In the WP-CLI container this prints its hash. Which file holds its Spanish strings?

```
printf '%s' 'build/blocks/book-card/index.js' | md5sum | cut -d' ' -f1
```

Q3

The contrast script printed `slate on paper 5.8:1 AA`. What does that allow?

A) Normal-size text in slate on paper meets WCAG AA (4.5:1), but not AAA (7:1)

B) Only large text: normal text needs 7:1

C) Nothing: anything under 7:1 fails WCAG

D) Slate is safe on every background of the palette

Q4

Why does the new uninstall.php read the opt-in with `get_option( 'pagesco_books_settings', array() )` and not with `Settings::get( 'delete_data' )`?

A) get_option() is faster

B) Settings::get() would translate the value

C) uninstall.php runs after the option is deleted

D) During uninstall the plugin is not loaded: its classes, post type and taxonomy are not there

Q5

Which of these must NOT be in the release ZIP of Pages & Co. Books?

A) `build/blocks/blocks-manifest.php`

B) `phpcs.xml.dist`

C) `languages/pagesco-books-es_ES.l10n.php`

D) `vendor/autoload.php`

Q6

What does `--no-purge` change in `wp i18n make-json pagesco-books-es_ES.po --no-purge`?

JOURNAL

Use one site you visit every day with the keyboard only for five minutes. Can you always see where you are, and reach everything? Then think of a client who sells abroad: which parts of the site would a translation file cover, and which would need a multilingual content plugin?

8.4 Deployment & Maintenance

BY THE END YOU CAN

- Choose a hosting type for a WordPress project by what it needs, and explain how code, content and configuration move between development, staging and production
- Keep a WordPress project's own code in Git with a .gitignore, and build release files from a tagged commit with composer install --no-dev, npm ci, npm run build and the plugin ZIP
- Move a database with wp db export, wp db import and wp search-replace, including serialized values and a plugin's own tables
- Configure WordPress for production: debugging and development mode off, the environment type, secrets outside the code, a real cron job, file editing disabled and a least-privilege database account
- Back up a site and prove the backup with a restore, deploy an update, and keep a maintenance checklist

KEY IDEAS

- **Code goes up, content comes down, configuration stays.** A Site Editor change on the live site is content.
- **Production:** WP_DEBUG off and errors only in a log, WP_ENVIRONMENT_TYPE production, no WP_DEVELOPMENT_MODE, DISALLOW_FILE_EDIT, DISABLE_WP_CRON with a real cron job, secrets on the server only; raise the theme's Version with each release.
- **Git** tracks only your own source: a .gitignore that ignores everything and lets your folders back in; tag every release; never deploy the repository.
- **Release builds** from a tag in a clean folder: `composer install --no-dev --optimize-autoloader`, `npm ci`, `npm run build`, `npm run plugin-zip`.
- **Migrations:** `wp search-replace` (serialized data), `--all-tables-with-prefix`, `--skip-columns=guid`, `--dry-run` first, `--export` for a launch file; after importing a development database, reset passwords and remove application passwords.
- **Running a live site:** a cron job, daily backups kept elsewhere and proven by a restore, updates through staging, a least-privilege database account, a maintenance checklist.

WORKED EXAMPLE: PRODUCTION SETTINGS, ASKED

Three of the production settings, asked of your development site:

```
wp eval 'printf( "%s | %s | %s\n", wp_get_environment_type(), wp_get_development_m
```

```
local | theme | file editor on
```

And one that core derives from the environment: application passwords work without HTTPS only on local sites:

```
wp eval 'var_dump( wp_is_application_passwords_available() );'
```

```
bool(true)
```

On the rehearsal server the same commands print production, an empty development mode, no file editor, and false (recorded in the lesson, Step 5).

GUIDED LAB

- `cp labs/module-16/l8_4-deployment-maintenance/migration_preview_starter.sh wordpress/`, then `docker compose run --rm wpcli sh migration_preview_starter.sh`.
- TODO 1 to 4: never a search-replace without `--dry-run` or `--export`.
- Finished early? `restore_drill_starter.sh`, `cron_audit_starter.php`, then `debug_launch_check.sh`.

QUESTION	YOUR ANSWER
Which table is missing without --all-tables-with-prefix?	
What is the 34?	
What did the database say at the end, and why?	

PREDICT, THEN RUN

Write your prediction first, then run it on your site. A wrong prediction you can explain is worth more than a right guess.

P3

Predict the output, then run it. (The tables WordPress registers.)

```
wp db tables | wc -l
```

P4

Predict the output, then run it. (A move to the real address, counted.)

```
wp search-replace 'http://localhost:8080' 'https://www.pagesco.example' --all-
```

LESSON PROJECT: START IN CLASS, FINISH AT HOME

Launch Day, Rehearsed (lesson page). In class: Step 3. At home: Steps 1 and 2 (after 8.3's Step 5), then 4 to 7.

- ☐ Step 3: pagesco-live unzipped, .env with my passwords, `docker compose up -d`
- ☐ Step 1: commit, tag release-1, the release files built from the tag
- ☐ Step 2: the launch database: 13 tables
- ☐ Step 4: launched at http://localhost:8081, new passwords saved
- ☐ Step 5: the production settings checked
- ☐ Step 6: a backup, a deleted book, restored
- ☐ Step 7: release 2 deployed without touching the database

SELF-CHECK

Q7

Maria changed the header in the live site's Site Editor. You deploy theme 1.2.0 with a new parts/header.html. What do visitors see?

A) The new header from the theme's file

B) Maria's header: Site Editor changes are saved in the database, and they win over the theme's files

C) Both headers, one above the other

D) An error until someone resets the template part

Q8

On the course site after Lesson 8.3: what does this print, and which table makes the difference?

```
echo "$(wp db tables | wc -l) $(wp db tables --all-tables-with-prefix | wc -l)
```

Q9

Why does every migration command in Lesson 8.4 say `--skip-columns=guid`?

A) The GUID is an identifier feed readers use to recognize posts, not a link: it must not change

B) GUID columns are serialized and would break

C) The guid column is too long for search-replace

D) WordPress rewrites GUIDs itself on the first request

Q10

The project's .gitignore says `!/themes/`, then `/themes/*`, then `!/themes/pagesco/`. Why not just `!/themes/pagesco/` after `/*`?

A) Git reads .gitignore from the bottom up

B) The order does not matter; it is only for readability

C) Without the first line Git would also track Twenty Twenty-Five

D) Git cannot let a file back in when its folder is ignored: the folder is let in, its contents ignored again, then the one subfolder let in

Q11

In a clean copy of the tagged plugin (git archive), which Composer command builds the production autoloader, and what does it leave out?

Q12

On the rehearsal server the account pagesco_app has no LOCK TABLES right. What happens to `wp db export "$BACKUP"`?

A) It works: WP-CLI locks nothing

B) It asks for the root password

C) It fails with error 1044, and leaves a small header-only file under the backup's name

D) It exports only the tables the account may lock

◆ JOURNAL

The live shop goes down at 9 p.m. on a Thursday, during the book club. Write, step by step, what you would do in the first fifteen minutes: where you look first, how you roll back, which backup you use, whom you tell. Which part of today's rehearsal made that plan possible?

THE FINAL PROJECT: RESERVE & PICK UP

◆ YOUR PLAN (START IN CLASS, FINISH TONIGHT)

Read Lesson 9.1's Brief again at home. Underline the nouns (the data) and the verbs (the actions). Then fill the table: for each of Maria's requirements, where the rule is enforced and which doors reach it.

REQUIREMENT (MARIA'S WORDS)	THE RULE, AND WHERE IT LIVES	THE DOORS (FORM, REST, BLOCK, SCREEN)
1.		
2.		
3.		
4.		

REQUIREMENT (MARIA'S WORDS)	THE RULE, AND WHERE IT LIVES	THE DOORS (FORM, REST, BLOCK, SCREEN)
5.		
6.		
7.		
8.		

Three questions I still have:

MILESTONES

#	MILESTONE	USES LESSONS	TIME
1	Plan, and measure before you build	1.1, 8.2	15 min
2	The data: a holds table and two settings	5.1, 5.4, 5.5	25 min
3	The rules, and the email	1.3, 2.3, 5.3, 1.2	30 min
4	REST routes, form handlers and the staff screen	5.3, 5.4, 7.1, 7.2	30 min
5	Two blocks: Reserve for Pickup and My Holds	3.2, 6.1, 6.3, 6.4	45 min
6	Live prices on the front page	3.3, 6.3	15 min
7	Quality: standards, tests, speed, accessibility	8.1, 8.2, 8.3	30 min
8	Release 1.2.0	8.3, 8.4	20 min
9	Deploy to the rehearsal server	8.4, 7.3	25 min
10	Write-up and showcase	all	15 min

HOW IT IS ASSESSED

AREA	POINTS
Features	20

AREA	POINTS
Data and rules	15
Security and privacy	20
Blocks and experience	15
Code quality	10
Testing	10
Release and deploy	5
Write-up and showcase	5

EXIT TICKET

E1

Give one example each of code, content and configuration on the Pages & Co. site, and how each moves to the live site.

E2

Name two translator-friendly rules for strings.

E3

Which two Reserve & Pick Up requirements do you expect to be the hardest, and which layer handles each?

NOTES

HOMEWORK CHECKLIST

- ☐ Lesson 8.3 project Steps 3 to 5 and the Release note
- ☐ Lesson 8.4 project Steps 1, 2, 4 to 7 and the Deployment note
- ☐ 8.3 Exercises 1 and 2; 8.4 Exercises 1 and 2
- ☐ Both Quick Quizzes, both journal entries
- ☐ The final project's plan: the requirements table and three questions (no building yet)
- ☐ Optional: `a11y_scan_starter.sh`, `release_check_starter.sh`, `debug_seats_spanish.php`, `restore_drill_starter.sh`, `cron_audit_starter.php`, `debug_launch_check.sh`

BRING LIST FOR SESSION 17

- The site as Lesson 8.4 leaves it: `wordpress/wp-content` is a Git repository with a clean working copy and the tags release-1 and release-2 (`git status`, `git tag -n`), plugin and theme 1.1.1, `composer lint` exits 0.
- The rehearsal server `~/pagesco-live` launched, with plugin and theme 1.1.1, its new passwords in your password manager, `LOCK TABLES` granted, and one backup in `backups/` (Lesson 8.4, Steps 3 to 7). `docker compose up -d` in its folder before the session.
- The test environment works: `npx @wordpress/env start` in the plugin's folder, the twelve tests green (Node.js 24; the node:24 and composer:2 images pulled).
- An internet connection for npx and `npm ci` (Lesson 9.1's Milestones 5 and 8).
- Your plan for Reserve & Pick Up from tonight's homework (the requirements-to-layers table, the questions you still have), Lesson 9.1 open in a browser tab, pagesco-plan.txt, the journal and the workbook.

GLOSSARY

TERM	MEANING
i18n · l10n · a11y	Internationalization (your code) · localization (translation files) · accessibility

TERM	MEANING
POT · PO · MO · l10n.php	The template · one language's translation · compiled for PHP · the PHP-array format (6.5+)
text domain	The name that ties a plugin's strings to its translations (`pagesco-books`)
WCAG 2.2 AA	The usual accessibility target: contrast 4.5:1, keyboard access, visible focus, ...
Plugin Check	The plugin review team's checker (a plugin with a WP-CLI command)
staging	A copy of production on the same kind of server, for trying releases with real data
release tag	An annotated Git tag on the commit that went live (`release-1`)
`git archive`	Writes exactly the files of a commit, nothing else: the clean copy a build starts from
`wp search-replace`	Replaces text in the database, including inside serialized values
GUID	A post's permanent identifier; looks like a URL, never changed by a migration
least privilege	An account with only the rights it needs (pagesco_app: its own database)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 17: Final Project Kickoff and Studio I

Your workbook for Session 17: the readiness check, the brief by layer, the milestone tracker for the whole project, the guided lab, predict-then-run practice, a self-check and your exit ticket.

Lesson 9.1 · Milestones 1–4 | Session 17 · 3 hours | Name: _____

SESSION GOALS

- Start the final project from a clean site: release-2 in Git, the rehearsal server ready.
- Turn Maria's brief into data, rules and doors, and know which layer handles each requirement.
- Know how the project is scored: the security and privacy checklist and the 100-point rubric.
- Build Milestones 1 to 3 (the plan and baseline, the holds table and settings, the rules and the email) and start Milestone 4 (the doors).
- Leave with a plan for Milestones 4 to 6 at home.

START HERE: READY FOR THE FINAL PROJECT?

READINESS CHECK

- ☐ In wordpress/wp-content: `git tag -n` lists release-1 and release-2; `git status --short | wc -l` prints 0
- ☐ `wp plugin list --name=pagesco-books --fields=name,status,version` says active, 1.1.1
- ☐ In pagesco-live: `docker compose ps` shows db, wordpress and cron running
- ☐ I read The Brief, the Milestones, the Security and Privacy Checklist and the rubric on lesson_30.html
- ☐ My question about the brief: _____

THE BRIEF IN ONE LINE EACH (FILL IN THE LAYER)

#	REQUIREMENT	DATA, RULES OR DOOR? WHICH CLASS?
1	Reserve, three days, expired	
2	Limits, duplicates, sold out, settings	
3	My holds, cancel, only your own	
4	Staff list and "collected"	

#	REQUIREMENT	DATA, RULES OR DOOR? WHICH CLASS?
5	The email	
6	No JavaScript, screen readers, no reload	
7	Live prices on the home page	
8	Personal data, tested, safe deploys	

LESSON

9.1 Final Project: Launch the Pages & Co. Website

BY THE END OF THE PROJECT YOU CAN

- Turn a client's brief into data, rules, routes, blocks and screens, and decide where each rule is enforced
- Build a feature through every layer of the course: a custom table, settings, a service class with a custom hook, REST routes with real permission callbacks, form handlers, a staff screen, a dynamic and an interactive block, and a theme pattern with live data
- Make it secure and private by default: nonces and capabilities on every change, ownership in every query, escaping, and personal data kept no longer than needed
- Prove the feature with coding standards, PHPUnit tests, a smoke test from outside, accessibility checks and a performance measurement
- Release it from a tagged commit and deploy it as code only, then present your work against a rubric

KEY IDEAS

- **A feature is a stack of layers:** the data (HoldsTable only stores), the rules (Holds only decides), the doors (form handlers, REST routes, blocks only check who is asking). A rule changes in one place.
- **A custom table** for holds: typed columns, indexes for "this customer's holds" and "what is waiting", a clean delete. `maybe_upgrade()` on init creates it on an update, where activation never runs.
- **"Expired" is computed,** never stored: waiting and `pickup_by` before today.
- **Ownership goes into the query** (`WHERE id = %d AND user_id = %d` , the user from the login); another customer's hold is "not found", so its existence is never confirmed.
- **Errors carry their HTTP status:** 401 not logged in, 404 no such thing (or not yours), 409 the shop's state says no. Every door passes it on.
- **Your own action** (`pagesco_books_hold_placed`) is the extension point: the email listens to it; the rules know nothing about email.
- **Measure first** (the query report), **prove last** (lint, tests, a smoke test, the numbers again).

WORKED EXAMPLE: WHAT A RELEASE ZIP WOULD CONTAIN

Lesson 9.1's Common Mistake 1, met before it happens. In the plugin's folder, with Node as in Lesson 6.1, a stand-in file in the folder Milestone 2 creates, then npm's own list of what a package would hold:

```
mkdir -p src/Holds
printf '<?php\n' > src/Holds/HoldsTable.php
npm pack --dry-run 2>&1 | grep -c ' src/'
npm pack --dry-run 2>&1 | grep -c ' src/Holds/'
rm -r src/Holds
```

```
24
0
```

24 PHP files from src/, none from src/Holds/: package.json's "files" list names the folders that existed in Lesson 8.3. Nothing warns you; the live site would answer "There has been a critical error". The rule: a new top-level folder in src/ goes into "files" in the same commit, and Milestone 8 counts the new folder's files in the ZIP before upload.

◆ MILESTONE TRACKER

MILESTONE	DONE WHEN (FROM THE LESSON)	IN CLASS / HOME	DONE
1 Plan and baseline	pagesco-plan.txt has the plan; three query counts written down: / ___ · /books/dune/ ___ · /find-a-book/ ___	class	☐
2 The data	the maybe_upgrade check: no table, then six columns and version 1; the "Reserve & Pick Up" settings section	class	☐
3 The rules and the email	holds-check.php prints the lesson's eleven lines; the Animal Farm email is in Mailpit	class	☐
4 The doors	two routes; a visitor 401; Jordan 409; Priya 201; Jordan's DELETE of hold 5 is 404; Devon gets no Holds menu; the forged POST 403; commit	start in class, finish at home	☐
5 Two blocks	the four states; holds-smoke.sh: eight ok lines, exit status 0; the JavaScript path in a browser	home	☐
6 Live prices	the staff picks follow Dune's price; without the plugin: "Ask us for the price"; commit	home	☐
7 to 9	quality, release 1.2.0, the code-only deploy	Session 18	☐
10 Write-up	RESERVE-AND-PICKUP.md	home, after Session 18	☐

◆ GUIDED LAB

- From the project folder: `cp labs/module-17/l9_1-final-project/ownership_sql_starter.php wordpress/`, then `wp eval-file --use-include ownership_sql_starter.php`. Open `wordpress/ownership_sql_starter.php` in VS Code.
- TODO 1: the prepared SELECT. TODO 2: the update with the owner in the WHERE. TODO 3: the door, with the user from the login and ONE error for every refusal. Run again after each.
- Done when your output matches "Expected when done" at the end of the file. Then `rm wordpress/ownership_sql_starter.php`.
- Finished early? `rsvp_rules_starter.php`, then `debug_trash_door.php` in the same folder.

QUESTION	YOUR ANSWER
Devon tries wish 1 (Maria's). What would 403 instead of 404 tell him?	
Which line of Reserve & Pick Up does the same as your TODO 2?	

PREDICT, THEN RUN

Write your prediction first, then run it from the project folder with the `wp` shortcut (on your site at the end of Lesson 8.4, or later).

P1

Predict the output, then run it. (Devon is an author. Post 15 is Maria's.)

```
wp eval 'var_dump( current_user_can( "edit_post", 15 ), current_user_can( "edi
```

P2

Predict the output, then run it. (One line per feature in Plugin::feature_classes().)

```
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

P3

Predict the output, then run it. (The label of a hold prints its date like this.)

```
wp eval 'echo get_option( "date_format" ), " | ", wp_date( get_option( "date_f
```

P4

Predict the output, then run it. (The filter trick of HoldsTable::waiting(), with a real ID.)

```
wp eval 'global $wpdb; echo $wpdb->prepare( "WHERE ( %d = 0 OR user_id = %d )"
```

SELF-CHECK

Q1

Why does the reference solution keep holds in a custom table rather than a post type?

A) A post type cannot store a date

B) A hold is one customer, one book, a status and a date: typed columns with indexes answer "this customer's holds" and "what is waiting" directly, and old rows can be deleted cleanly

C) Custom tables are always faster than wp_posts

D) Post types cannot be private

Q2

Where does the reference solution keep the status `expired`?

A) In the status column, set by the daily cron job

B) In a post meta field on the book

C) In a transient that expires after hold_days

D) Nowhere: `Holds::status()` computes it from `pickup_by` and today's date every time a hold is shown

Q3

On the course site at the end of Lesson 8.4, what does this print? And after Milestone 4?

```
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

Q4

The plugin is already active on your development site. You add `Holds\HoldsTable::install()` to `Activator::activate()`. When is the holds table created on YOUR site?

A) On the next request that loads WordPress: `maybe_upgrade()` runs on init, finds no version option and installs it

B) Never, until you deactivate and reactivate the plugin

C) When you run `wp db query`

D) At 3 a.m., with the daily job

Q5

A reviewer finds these lines in Holds::cancel() (the reference has one call on one line). What can a logged-in customer now do?

```
$done = HoldsTable::finish(
    $hold_id,
    'cancelled'
);
if ( ! $user_id || ! $done ) {
```

A) Nothing new: the $user_id check still stops it

B) Cancel only their own holds, as before

C) Cancel ANY customer's waiting hold by sending its ID

D) Collect holds as staff

Q6

Why does `Holds::place()` answer the limit, a duplicate and a sold-out book with status 409, not 400 or 403?

A) 409 is the only status WordPress allows in a WP_Error

B) The request is well formed and the customer is allowed to ask; the shop's current state says no (Conflict)

C) 403 would log the customer out

D) 409 makes the browser retry automatically

Q7

Common Mistake 2, run as Devon (an author) about Maria's post 15. What does it print, and which check belongs in a handler?

```
wp eval 'var_dump( current_user_can( "delete_posts" ), current_user_can( "dele
```

Q8

Why does the confirmation email live in `HoldMail`, listening to `pagesco_books_hold_placed`, instead of a `wp_mail()` call inside `Holds::place()`?

A) wp_mail() cannot be called from a class

B) Actions run faster than function calls

C) Emails must be sent from a must-use plugin

D) The rules class knows nothing about email: others (an SMS, a staff notice in Exercise 2) attach to the same moment without changing the rules

Q9

Which line computes the last day of pickup, three days from today, the way the reference does?

A) `wp_date( 'Y-m-d', time() + $days * DAY_IN_SECONDS )`

B) `date( 'Y-m-d', time() + $days * DAY_IN_SECONDS )`

C) `current_time( 'mysql', true )`

D) `gmdate( 'Y-m-d' ) . ' + 3'`

Q10

`HoldsTable::waiting()` uses this trick. What does it print, and what does it mean for the query?

```
wp eval 'global $wpdb; echo $wpdb->prepare( "WHERE ( %d = 0 OR user_id = %d )"
```

Q11

Milestone 4 sends a forged "Mark collected" from Maria's own logged-in session, but without the form's nonce. What does admin-post.php answer?

A) 303, back to Books > Holds with ?pagesco_hold=collected

B) 401: Maria is not logged in

C) 403 with `The link you followed has expired.`

D) 400: the action is unknown

Q12

The lesson's mental model is the shop's counter. Name its three layers and the one job of each.

◆ PROJECT LOG

Three lines tonight, in pagesco-plan.txt under "Reserve & Pick Up": the milestone you finished last and its done-when output; one decision you made differently from the reference, and why; the one thing you are least sure about. (The lesson's journal prompt comes after Session 18, when the feature is finished.)

EXIT TICKET

Three questions, no notes.

E1

Name the three layers of Reserve & Pick Up and one class in each.

E2

Priya sends `DELETE /pagesco/v1/holds/5` for Jordan's hold. What does she get, and which line of code decides it?

E3

Which milestone is the next one you will finish, and what is its done-when?

NOTES

HOMEWORK CHECKLIST

- ☐ Milestone 4 finished: the REST checks, the staff screen, the forged request, truncate, passwords revoked, commit
- ☐ Milestone 5: both blocks, the My holds page, the four states, holds-smoke.sh with eight ok lines
- ☐ Milestone 6: live prices, with and without the plugin; commit

- ☐ `npx @wordpress/env start` once at home (then `stop`), pagesco-live starts
- ☐ The project log in pagesco-plan.txt
- ☐ Optional: `rsvp_rules_starter.php`, the own_action folder lab, `debug_trash_door.php`
- ☐ Bring to Session 18: the site at Milestone 6, holds-smoke.sh, the plan with the baseline numbers, this workbook

GLOSSARY

TERM	MEANING
hold	One customer, one book, a status (waiting, collected, cancelled) and a last day of pickup; a row in `wp_pagesco_holds`
`pickup_by`	The last day of pickup, a date in the site's time zone: today + `hold_days` when placed
expired	A waiting hold whose `pickup_by` is before today: computed by `Holds::status()`, never stored
`maybe_upgrade()`	Runs on init; installs the table when the version option is older than the code (the update path)
door	Anything that receives a request: a form handler, a REST route, a block; it checks who is asking and passes the question to the rules
IDOR	Insecure direct object reference: acting on someone else's record by sending its ID; stopped by ownership in the query
409 Conflict	The request is fine, the current state says no (limit, duplicate, sold out)
`pagesco_books_hold_placed`	The plugin's own action after a hold is saved; HoldMail listens to it
baseline	The numbers before a change (Milestone 1's query counts), compared after (Milestone 7)

INSTRUCTOR KIT · PARTICIPANT WORKBOOK

Session 18: Final Project Studio II, Showcase and Next Steps

Your workbook for Session 18, the last one: the rebuild check, Milestones 7 to 10 with their done-when, the showcase checklist and the submission checklist, Lesson 9.2's key ideas and guided lab, a self-check for each lesson and your exit ticket.

Lessons 9.1–9.2 | Session 18 · 3 hours | Name: _____

SESSION GOALS

- Prove the feature four ways: standards, tests, numbers, accessibility.
- Release 1.2.0 from a tag, check the ZIP, deploy code only to the rehearsal server.
- Show Reserve & Pick Up in five minutes, and defend one decision.
- See what comes next: headless and CORS, AI in core, abilities as tools, contributing, a portfolio.
- Leave with the submission checklist and a plan for your own next project.

START HERE: THE REBUILD CHECK

BEFORE THE STUDIO

- ☐ `sh holds-smoke.sh http://localhost:8080 jordan jordan-pass dune; echo "exit status $?"`: eight ok lines, exit status 0
- ☐ Live prices: the staff picks follow Dune's price; without the plugin, "Ask us for the price"
- ☐ wp-env started once at home; pagesco-live starts
- ☐ Committed: `git status --short | wc -l` prints 0

9.1 Final Project: Launch the Pages & Co. Website

BY THE END YOU CAN

- Turn a client's brief into data, rules, routes, blocks and screens, and decide where each rule is enforced
- Build a feature through every layer of the course: a custom table, settings, a service class with a custom hook, REST routes with real permission callbacks, form handlers, a staff screen, a dynamic and an interactive block, and a theme pattern with live data
- Make it secure and private by default: nonces and capabilities on every change, ownership in every query, escaping, and personal data kept no longer than needed
- Prove the feature with coding standards, PHPUnit tests, a smoke test from outside, accessibility checks and a performance measurement
- Release it from a tagged commit and deploy it as code only, then present your work against a rubric

KEY IDEAS

- **Prove it four ways:** coding standards (a documented exception, not a silenced sniff), unit tests that would catch the dangerous bug, a smoke test from outside, numbers before and after.
- **A new feature is a minor version:** 1.2.0 everywhere a version lives; the theme too when its templates or patterns change (the live pattern cache).
- **Release from a tag,** build in a clean copy, and **check the ZIP** before upload: the new folder is in it and its autoloader finds the new classes.
- **Code only:** back up, install the ZIPs, let `maybe_upgrade()` create the table; create on the live site the content the feature needs (My holds, a test customer). Never the development database.
- **Showcase:** the problem, the browser, the rules, the proof, one decision you would defend, one thing next.

WORKED EXAMPLE: CHECK THE ZIP BEFORE IT LEAVES

Milestone 8's last check before anything leaves your machine: is the new folder in the ZIP, and does the ZIP's own autoloader find the new classes with nothing else around it?

```
mv release/pagesco-books/pagesco-books.zip release/
rm -r release/pagesco-books
unzip -l release/pagesco-books.zip | grep -c 'src/Holds/'
unzip -l release/pagesco-books.zip | grep -c 'build/blocks/reserve/'
unzip -q release/pagesco-books.zip -d /tmp/check
php -r 'require "/tmp/check/pagesco-books/vendor/autoload.php"; var_dump( class_ex
rm -r /tmp/check
unzip -p release/pagesco.zip pagesco/style.css | grep '^Version'
ls release
```

```
4
8
bool(true)
bool(true)
Version: 1.2.0
pagesco-books.zip  pagesco.zip
```

4 files from src/Holds and 8 from the Reserve block's build: package.json's files list has "src/Holds". The two `bool(true)` lines come from PHP loading only the ZIP's vendor/autoload.php: the classes are where the class map says. A ZIP that fails this check never reaches the rehearsal server.

GUIDED LAB

- In the studio, the project is the lab: Milestones 7, 8 and 9 with their done-when checks (the tracker below).
- Stuck or waiting? `labs/module-18/l9_1-final-project/version_audit_starter.php` (the five places a version lives), `files_list_starter.php` (what a ZIP would leave out), `smoke_rest_starter.sh` (a smoke test of your own), `debug_release_check.php`.

MILESTONE	DONE WHEN	DONE
7 Quality	`composer lint` exit status 0 (one documented exception); 17 tests OK in wp-env; the three numbers again; button names checked	☐
8 Release 1.2.0	five version places; the POT; `release-3` tagged; the ZIP check: src/Holds files, two bool(true)	☐
9 Deploy	1.1.1 kept; a backup; both ZIPs installed; the holds table; My holds and shoptest on live; the live smoke test: eight ok	☐
10 Write-up	RESERVE-AND-PICKUP.md with the outline of the lesson (at home)	☐

PREDICT, THEN RUN

Write your prediction first, then run it from the project folder (`wp` commands) or in the WP-CLI container (`curl`).

P1

Predict the output, then run it. (The reference after Lesson 9.1: holds still waiting, from a date long ago.)

```
wp eval 'echo count( PagesCo\Books\Holds\HoldsTable::waiting( "2000-01-01" ) )
```

P2

Predict the output, then run it. (The schema version maybe_upgrade() compares with.)

```
wp option get pagesco_books_holds_db_version
```

◆ PROJECT: IN CLASS, THEN AT HOME

Showcase (five minutes). The lesson's checklist; tick what you showed:

1. **The problem** (30 seconds): the sticky notes, in one sentence each from the brief.
2. **A visitor** opens a book: "Log in to reserve". Then **a customer** logs in and reserves it: no reload, the message is announced. Show the email in Mailpit.
3. **The rules**: reserve the same book again, then a sold-out one, then one too many. Change "Holds per customer" in the settings and try again.
4. **My holds**: the list, a cancel. Then turn JavaScript off in the browser and reserve again: it still works.
5. **The counter**: Books > Holds as Maria, "Mark collected".
6. **The home page**: change Dune's price in the editor, reload the home page.
7. **The proof**: `composer lint`, the PHPUnit summary, the smoke test's output on the rehearsal server, `git tag -n`.
8. **One decision you would defend** (why a custom table, why "not found" instead of "forbidden", why

expired is computed) and **one thing you would do next**.

CLASSMATE	ONE STRENGTH	ONE QUESTION

Submission (one week from today):

- ☐ `git tag -n` in `wordpress/wp-content` lists `release-3`, and `git status` is clean
- ☐ `pagesco-books.zip` (1.2.0) and `pagesco.zip` (1.2.0) are kept with your release notes
- ☐ The output of `composer lint`, the PHPUnit run and both smoke tests is saved in your write-up or next to it
- ☐ The rehearsal server runs 1.2.0, has a My holds page, and its backup from before the deploy is in `backups/`
- ☐ `RESERVE-AND-PICKUP.md` maps every requirement to the code, lists your decisions and next steps
- ☐ If you share the project publicly (a portfolio, a Git host), share the code and screenshots only: never `.env`, database dumps or passwords

SELF-CHECK

Q1

Milestone 7: `composer lint` finds 11 warnings, all in HoldsTable.php (DirectQuery and NoCaching). What does the reference do?

A) Adds a phpcs:ignore comment to every query

B) Turns the two sniffs off in phpcs.xml.dist for the whole plugin

C) Documents ONE exception in phpcs.xml.dist, for that file, with its reason: a custom table has no WordPress API, and holds are per user and change with every click, so a cache would mostly be invalidated before it is read

D) Moves the holds into post meta so the warnings go away

Q2

On the reference after Milestone 8, what does this print? Why did the THEME's version change too?

```
wp eval 'echo wp_get_theme( "pagesco" )->get( "Version" ), " ", PAGESCO_BOOKS_
```

Q3

Milestone 9, on the rehearsal server: right after `wp plugin install /deploy/pagesco-books-1.2.0.zip --force`, `SHOW TABLES LIKE` finds no holds table. Why not yet?

A) `wp plugin install` ran with the OLD plugin in memory, and `wp db query` does not start WordPress: no new code has run init yet

B) The ZIP was built without src/Holds

C) The database user may not create tables

D) Tables are only created by the activation hook, so it never will be

Q4

After the deploy, the rehearsal server answers 404 for /my-holds/. What is the right fix?

A) Import the development database on the live site

B) Deploy the plugin again with --force

C) Flush the permalinks on the development site

D) Create the page ON the live site (it is content: it lives in the database), with `wp post create`, as the readme's upgrade notice says

Q5

Milestone 7 measures the front page at 31 queries for a visitor; the baseline was 22. Where do the nine come from, and why does the reference accept them?

Q6

Why does each Cancel button on My holds carry an aria-label that names its book?

```
<button type="submit" class="wp-element-button" aria-label="Cancel the hold or
<button type="submit" class="wp-element-button" aria-label="Cancel the hold or
```

A) Search engines index aria-labels

B) A screen reader would otherwise read "Cancel, Cancel, Cancel" with no way to tell which hold each one cancels

C) WordPress requires an aria-label on every button

D) It makes the button work without JavaScript

JOURNAL

Open pagesco-plan.txt and your first journal entry, both from Lesson 1.1. Compare the site you planned with the site you launched. Which part of WordPress felt like magic then and feels like ordinary code now? Which module was the hardest, and what made it click? Write one paragraph to yourself on the day you started Lesson 1.1: what should you have known?

9.2 Next Steps: Headless, AI in Core & Your Career

BY THE END YOU CAN

- Explain what "headless" WordPress means, when it is worth its costs, and build a small front end on your own REST routes that pages through results, handles HTML fields correctly and works across origins
- Describe what WordPress 7.0 and 7.1 ship for AI (the AI client, wp_supports_ai(), the Connectors screen) and what they do not (a model, a provider, an AI feature), and write code that degrades gracefully when no provider is set up
- Show how an ability such as pagesco/check-stock becomes a tool an AI model can call, and why its permission callback still decides
- Find your way into contributing to WordPress: the Make teams, Trac and GitHub, Learn WordPress, meetups and WordCamps
- Choose a career direction, turn the Pages & Co. project into a portfolio piece without leaking secrets, and keep up with WordPress releases

KEY IDEAS

- **Headless** keeps WordPress for content and rules and moves the front end elsewhere; it pays for apps and many front ends and costs previews, plugins' front ends and a second deployment.
- **A front end** reads every page (X-WP-TotalPages), knows which fields are HTML, puts text on the page with textContent, and relies on permission callbacks, not CORS.
- **WordPress 7 ships AI plumbing:** `wp_ai_client_prompt()`, `wp_supports_ai()`, the Connectors screen; no model, no provider, no core feature. Every result is text or a WP_Error.
- `using_abilities()` turns abilities into tools a model may call; the resolver runs them as the current user, so permission callbacks and input schemas still decide.
- **Contribute with evidence** (make.wordpress.org, Trac and GitHub, Learn WordPress, meetups); **publish code, never secrets** (search the whole history first).

WORKED EXAMPLE: WHAT ANOTHER ORIGIN MAY READ

A front end on another origin asks the shop's API. What may that page read?

```
curl -s -I -H 'Origin: http://localhost:3000' http://localhost:8080/wp-json/pagesc
```

```
HTTP/1.1 200 OK
Access-Control-Expose-Headers: X-WP-Total, X-WP-TotalPages, Link
Access-Control-Allow-Headers: Authorization, X-WP-Nonce, Content-Disposition, Con
X-WP-Total: 11
X-WP-TotalPages: 2
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: OPTIONS, GET, POST, PUT, PATCH, DELETE
Access-Control-Allow-Credentials: true
```

WordPress repeats the page's origin in Access-Control-Allow-Origin, so any origin may read public data, and exposes the paging headers (X-WP-Total, X-WP-TotalPages) so the page can read every page of results. None of this lets a page do what its visitor may not: the permission callbacks decide.

GUIDED LAB

- From the project folder: `cp labs/module-18/l9_2-next-steps/cors_headers_starter.sh wordpress/`, then `docker compose run --rm wpcli sh cors_headers_starter.sh`.
- Each TODO is the same grep and cut on another header; TODO 4 reads the status line of a protected route.
- Done when your output matches "Expected when done". Then `rm wordpress/cors_headers_starter.sh`.
- Finished early? `ai_readiness_starter.php`, then `debug_headless_client.php`.

QUESTION	YOUR ANSWER
The holds route said 401 and sent Allow-Origin. Is that a hole?	
Why does the browser need Expose-Headers to page through the books?	

PREDICT, THEN RUN

Write your prediction first, then run it from the project folder (`wp` commands) or in the WP-CLI container (`curl`).

P3

Predict the output, then run it. (The REST index, as a front end gets it.)

```
curl -s 'http://localhost:8080/wp-json/?_fields=name'
```

P4

Predict the output, then run it. (Two AI questions to the course site.)

```
wp eval 'var_dump( function_exists( "wp_ai_client_prompt" ), wp_supports_ai()
```

PROJECT: IN CLASS, THEN AT HOME

Pages & Co., What Comes Next (lesson page). In class: step 4 and the start of step 5. At home: steps 1 to 3 and the rest of 5.

- ☐ Step 1: the storefront lists all eleven books, shows "Pages & Co." correctly, works from a second origin
- ☐ Step 2: model-check.php: data for Maria, a refusal for Devon
- ☐ Step 3: the release-watch commands, and what I do with an update line
- ☐ Step 4: zero secrets in the history; README.md committed
- ☐ Step 5: next-plan.txt with the lessons to reread

SELF-CHECK

Q7

A front end on another origin asks the shop's API. What does this print, and does it make the holds routes public?

```
curl -s -I -H 'Origin: https://shop.example' http://localhost:8080/wp-json/pag
  | tr -d '\r' | grep -i '^access-control-allow-origin'
```

Q8

A headless storefront shows the heading "Pages & Co." (the entity, as text). What is the right fix?

A) Switch the heading to innerHTML

B) Change the site title in Settings to "Pages and Co."

C) Know which fields are HTML: decode the stored-escaped name once to text (DOMParser), then put it on the page with textContent

D) Escape it again with encodeURIComponent()

Q9

What does WordPress 7.1 ship for AI?

A) Plumbing: the AI client (wp_ai_client_prompt()), the Connectors screen and a switch to turn AI off; no model, no provider and no core feature

B) A small built-in model for alt text

C) A paid OpenAI integration enabled on every site

D) Nothing: AI is only in the separate AI plugin

Q10

What does this print on the course site, and what does it tell you?

```
grep -rl 'wp_ai_client_prompt *(' wp-admin wp-includes
```

Q11

Lesson 9.2's model-check.php lists the pagesco/check-stock ability as a tool for a model. Which name does the model see?

A) `pagesco/check-stock`

B) `check_stock`

C) `pagesco-check-stock`

D) `wpab__pagesco__check-stock`

Q12

Before the repository goes public, Lesson 9.2 searches with `git log --all -S`, not only the current files. Why?

JOURNAL

Go back to the very first page of this journal, Lesson 1.1's entry, and answer its two questions again, today. Why do you want to do WordPress development, and has the answer changed? What will you build next (step 5's next-plan.txt is the start)? Then three short lists: three things you can do now that you could not, three you want to learn next, and three people or places you will ask.

EXIT TICKET

Three questions, no notes.

E1

Which of your checks proves that another customer's hold cannot be cancelled, and which one cannot prove it?

E2

Code goes up, content stays: name one thing of Reserve & Pick Up for each.

E3

What will you build next with WordPress, and which lesson will you reread first?

NOTES

HOMEWORK CHECKLIST

- ☐ The final project: Milestones 7 to 10 and the submission checklist, due one week from today
- ☐ Lesson 9.1: Exercises 1 to 3, the Quick Quiz, the journal

- ☐ Lesson 9.2: project steps 1 to 3 and 5, Exercises 1 to 3 (4 is a stretch), the Quick Quiz, the last journal entry
- ☐ Optional: `ai_readiness_starter.php`, `publish_scan_starter.sh`, `debug_headless_client.php`, `version_audit_starter.php`, `smoke_rest_starter.sh`
- ☐ Stop what you do not use: `npx @wordpress/env stop`, `docker compose stop` in pagesco-dev and pagesco-live

GLOSSARY

TERM	MEANING
release-3	The Git tag of Pages & Co. Books 1.2.0 and the theme 1.2.0: what the release ZIPs are built from
smoke test	A short script that checks a running site from outside, through the same doors a browser uses (holds-smoke.sh)
code-only deploy	Installing new release ZIPs on the live site without touching its database; content the feature needs is created there
headless	WordPress keeps content and rules; another front end draws the pages from its API
CORS	The headers that tell a browser which other origins may read a response (Access-Control-Allow-Origin ...)
AI client	`wp_ai_client_prompt()` and friends (7.0): a provider-neutral way for plugins to call models; needs a provider plugin and a key
Connectors	Settings > Connectors (7.0): where a site owner connects services that need keys
`using_abilities()`	Offers abilities to a model as tools; the resolver runs them as the current user

Complete Participant Workbook · all 18 sessions · v1.0 · © 2026 Ray de la Paz